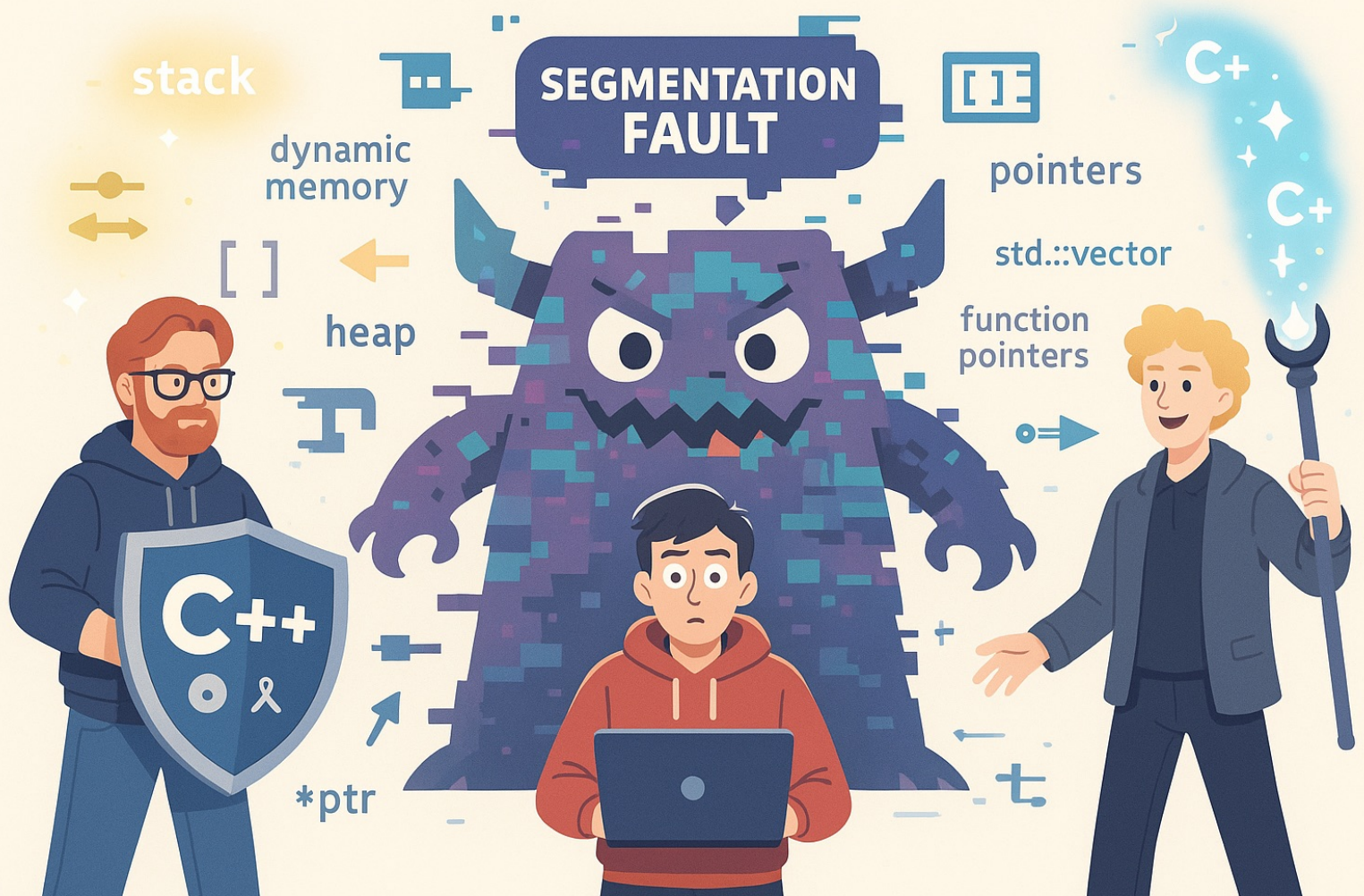




Программирование на языке C++

1 семестр

Автор: **Дмитриенко Еремей Алексеевич**

Дата: 15.12.2025



Стандарт языка по умолчанию: C++20

Оглавление

От автора	1
Несколько слов перед началом	1
Обратная связь	1
Благодарности	1
 0 Как мы будем учить C++ и что вообще происходит	 2
Глава 0 Как мы будем учить C++ и что вообще происходит	4
0.0 Что это за текст	4
0.1 Где вообще живёт C++ и зачем его учить	4
0.2 Как устроен курс/пособие	4
0.3 Немного формальностей: стандарт, компилятор, cppreference	4
0.4 Что от тебя требуется	5
0.5 Чем C++ отличается от Python	5
0.6 Настраиваем окружение	5
0.7 Твоя первая программа Hello, world!	6
0.8 Как мы будем смотреть на код в пособии	6
0.9 Ошибки: compile-time vs runtime (очень коротко)	8
0.10 Как читать это пособие с максимальной пользой	8
 Глава 1 Инженерный минимум: сборка, заголовки, диагностика	 9
1.1 Компиляция и линковка: два разных этапа	9
1.2 .h и .cpp: что куда класть	10
1.3 Include guards: защита от двойного подключения	10
1.4 Предупреждения компилятора: включаем по максимуму	11
1.5 Санитайзеры: ловим ошибки в рантайме	11
1.6 Под капотом: символы, манглинг, ABI (по желанию)	12
 Глава 2 Структура программы: объявления, функции и пространства имён	 14
2.1 Объявление и определение	14
2.2 Область видимости и перекрытие имён	15
2.3 Пространства имён (namespace)	16
2.4 Предварительные объявления (forward declaration)	16
2.5 Функции	17
 1 Базовые типы и выражения	 19
 Глава 3 Типы и литералы	 21
3.1 Что такое тип	21
3.2 Базовые встроенные типы C++	21

3.3	Что такое литерал	24
3.4	Целочисленные литералы	24
3.5	Вещественные литералы	25
3.6	Символьные и строковые литералы	26
3.7	Литералы <code>bool</code> и <code>nullptr</code>	27
3.8	<code>auto</code> и тип литерала	28
3.9	Константы и литералы: <code>const</code>	29
Глава 4	Операторы и приоритеты	30
4.1	Что такое оператор	30
4.2	Классификация операторов	30
4.3	Приоритет и ассоциативность	30
4.4	Арифметические операторы	31
4.5	Операторы присваивания	33
4.6	Операторы сравнения	34
4.7	Логические операторы и короткое замыкание	35
4.8	Инкремент и декремент	36
4.9	Побитовые операторы (быстрое знакомство)	37
4.10	Оператор запятой	38
4.11	Скобки и стиль: как не сойти с ума	38
2	Управление потоком и как всё может пойти не так	40
Глава 5	Условия и циклы	42
5.1	Что вообще такое «управление потоком»	42
5.2	Булевы выражения и сравнения	42
5.3	Логические операторы <code>&&</code> , <code> </code> , <code>!</code> и ленивость	43
5.4	Конструкция <code>if / if-else</code>	44
5.5	<code>switch</code> и <code>case</code>	45
5.6	Тернарный оператор <code>?:</code>	45
5.7	Цикл <code>while</code>	46
5.8	Цикл <code>do-while</code>	46
5.9	Цикл <code>for</code>	47
5.10	Операторы <code>break</code> и <code>continue</code>	48
5.11	Бесконечные циклы	48
5.12	Range-based <code>for</code> (современный C++)	49
5.13	Частые ошибки в условиях и циклах	49
Глава 6	Ошибки и неопределённости	51
6.1	Какие вообще бывают ошибки	51
6.2	Ошибки по времени обнаружения	51
6.3	Поведение программы по стандарту C++	53
6.4	Undefined behavior (UB) — тёмная магия	54

6.5	Как живёт стек при вызове функции	57
6.6	Unspecified behavior	59
6.7	Implementation-defined behavior	60
6.8	Почему вся эта неопределённость — это и плюс, и минус	60
6.9	Стратегии борьбы с ошибками	60
6.10	Где ошибка программы, а где просто «плохие данные»	62
6.11	Типичные ошибки новичка (и где тут неопределённость)	62
6.12	Практические правила, которые реально спасают от ошибок	64
Глава 7	Исключения	65
7.1	Зачем нужны исключения	65
7.2	Минимальный синтаксис	65
7.3	Стандартные исключения, которые встречаются в курсе	66
7.4	Раскрутка стека и RAII	66
7.5	noexcept и terminate	67
7.6	Связь с предыдущими главами	67
7.7	Что важно помнить	68
3	Данные посложнее: массивы, строки, векторы, указатели	69
Глава 8	Массивы и циклы по массивам	71
8.1	Зачем нужны массивы	71
8.2	Объявление и инициализация массивов	71
8.3	Индексация: от 0 до N–1 и почему a[10] — это зло	72
8.4	Размер массива и sizeof	73
8.5	Цикл for по массиву через индексы	73
8.6	Циклы while и do while по массиву	74
8.7	Массивы разных типов и многомерные массивы	75
8.8	Указатель на массив: int (*)[N]	76
8.9	Массивы и функции: почему внутри функции sizeof не спасёт	77
8.10	Циклы по массиву через указатель	77
8.11	Типичные ошибки при работе с массивами	78
Глава 9	std::string и std::vector	80
9.1	C-строки против std::string	80
9.2	Базовые операции с std::string	81
9.3	std::vector: динамический массив	86
9.4	Циклы по std::vector	88
9.5	std::vector и функции	89
9.6	Важные детали: память и сломанные указатели	90
9.7	Вектор векторов: почти двумерные массивы	91
9.8	Сравнение: массив, std::string, std::vector	92
Глава 10	Указатели и ссылки	94

10.1	Адрес в памяти и оператор &	94
10.2	Указатель: объявление, инициализация, nullptr	94
10.3	Разыменование: оператор *	96
10.4	Указатели и массивы	96
10.5	Массивы и функции: «декей» к указателю	97
10.6	Указатели и const	98
10.7	Ссылки: «кличка» для объекта	101
10.8	Параметры функций: указатели vs ссылки	102
10.9	Возврат по ссылке	106
10.10	Указатели, ссылки и std::vector/std::string	107
10.11	Хардкор: указатели на функции	108
10.12	Типичные ошибки с указателями и ссылками	111
4	Собственные типы и ООП	114
Глава 11	struct, выравнивание и работа с памятью	116
11.1	Свой тип данных: что такое struct	116
11.2	Размер и раскладка структуры в памяти	117
11.3	Выравнивание и паддинг	118
11.4	Пустые классы, sizeof и оптимизация пустой базы (EBO)	119
11.5	Заглядываем внутрь: reinterpret_cast и байтики	120
11.6	Где живёт структура: стек и куча	121
11.7	new, delete и массивы структур	122
11.8	Классические грабли работы с памятью	123
11.9	Как думать о struct и памяти	124
Глава 12	Классы, инкапсуляция и жизненный цикл	125
12.1	От struct к class: идея	125
12.2	Инкапсуляция и модификаторы доступа	125
12.3	Устройство класса: поля, методы, this	126
12.4	Статические поля класса (static data member)	127
12.5	Перегрузка операторов и friend (семинар + экзаменационный минимум)	128
12.6	Конструкторы: старт объекта	131
12.7	Деструктор: финал объекта	133
12.8	Жизненный цикл и где живут объекты	134
12.9	Копирование и присваивание: две разные операции	135
12.10	Когда дефолтное копирование ломает всё	135
12.11	Copy elision (RVO/NRVO) и возврат по значению	137
12.12	Правило трёх	138
12.13	Умные указатели: RAII для динамической памяти	139
12.14	Инкапсуляция и инварианты	140
Глава 13	Наследование и виртуальные функции	142

13.1	Зачем нужно наследование	142
13.2	Базовый синтаксис наследования	143
13.3	Что происходит с объектом при наследовании	144
13.4	Конструкторы и деструкторы при наследовании	145
13.5	Перекрытие методов без <code>virtual</code>	147
13.6	<code>virtual</code> и полиморфизм	147
13.7	Как это устроено внутри (vtable, но без фанатизма)	148
13.8	<code>override</code> и почему его нужно писать всегда	148
13.9	Виртуальный деструктор: <code>must-have</code> для полиморфной иерархии	150
13.10	Чисто виртуальные функции и абстрактные классы	151
13.11	Наследование интерфейса и реализации	152
13.12	Срезка объектов (object slicing)	153
13.13	Запрет переопределения и наследования: <code>final</code>	154
13.14	Скрытие перегрузок и <code>using Base::f</code> (семинар)	155
13.15	Наследование конструкторов: <code>using Base::Base</code>	155
13.16	Значения по умолчанию и <code>virtual</code>	156
13.17	Виртуальные вызовы в конструкторах и деструкторах	157
13.18	Множественное и виртуальное наследование (очень кратко)	158
Глава 14	Динамическое приведение типов и RTTI	160
14.1	Зачем вообще нужны приведения в ООП	160
14.2	Напоминание: статический и динамический тип	161
14.3	Четыре оператора приведения в C++	161
14.4	RTTI: что это и когда оно есть	161
14.5	<code>dynamic_cast</code> : безопасный даункаст	162
14.6	Сравнение <code>static_cast</code> и <code>dynamic_cast</code> при наследовании	163
14.7	<code>dynamic_cast</code> и разные направления	163
14.8	<code>typeid</code> и <code>std::type_info</code>	164
14.9	Когда RTTI не работает или опасен	165
14.10	Где <code>dynamic_cast</code> и RTTI уместны, а где — тревожный звоночек	166
14.11	Главное по главе	166
5	Введение в шаблоны	168
Глава 15	Шаблоны: основы	170
15.1	Идея шаблонов	170
15.2	Функции-шаблоны	170
15.3	Классы-шаблоны	172
15.4	Где «живут» шаблоны и почему они обычно в <code>.h</code>	174
15.5	Когда возникают ошибки	174
Глава 16	Шаблоны: специализация, нетиповые параметры и метапрограммирование	175

16.1	Полная специализация	175
16.2	Частичная специализация (только для классов)	176
16.3	Нетиповые параметры шаблона	177
16.4	Шаблоны от шаблонов: <code>template template parameters</code>	177
16.5	Compile-time вычисления: шаблонная рекурсия	178
16.6	<code><type_traits></code> : <code>is_same</code> , <code>is_class</code> , <code>conditional</code> и <code>if constexpr</code>	178
16.7	Заметка про <code>std::vector<bool></code>	180
Глава 17	Variadic templates, fold expressions и зависимые имена	181
17.1	Variadic templates: параметры-пакеты	181
17.2	Рекурсивная обработка пакета	181
17.3	Fold expressions (C++17)	182
17.4	Двухфазная проверка шаблонов	182
17.5	Зависимые имена: <code>typename</code> и <code>template</code>	183
17.6	Зависимые базовые классы и <code>this-></code>	183

От автора

Несколько слов перед началом

Если ты держишь в руках (или открываешь на экране) это пособие, значит, впереди у тебя первый серьёзный заход в мир C++. Здесь не будет попытки «выучить весь язык» — это вообще невозможно. Вместо этого мы аккуратно пройдемся по тем кускам C++, которые реально нужны живому человеку на первом курсе.

Курс вырос из лекций, где мы много писали код, ловили странные ошибки, спорили о стиле и регулярно наступали на одни и те же грабли. Пособие сохраняет тот же подход: минимум теории ради теории, максимум примеров, комментариев и честных пояснений, «почему тут всё так странно устроено».

По пути нам придётся подружиться с компилятором, научиться понимать его сообщения, разобраться, где заканчивается просто ошибка, а где начинается `undefined behavior`, и почему иногда программа «вроде работает», но на самом деле уже всё плохо.

Я очень рекомендую не просто читать код, а запускать его у себя, ломать, дописывать, смотреть, что говорит компилятор, и задаваться вопросом «почему?». Если по ходу чтения у тебя периодически возникает лёгкое ощущение ужаса от указателей и памяти — это нормально. Главное, чтобы вместе с этим появлялось и понимание, как этим аккуратно пользоваться.

Обратная связь

Если по ходу курса возникнут вопросы или захочется написать отзыв, можно обратиться в мой личный Telegram-канал: https://t.me/eremei_dmitrienko. Лучше всего писать в сообщения канала — я читаю их регулярно.

Удачи. Пусть C++ будет для тебя не только источником боли, но и инструментом, с помощью которого можно делать быстрые и красивые вещи.

Благодарности

Особую благодарность хочу выразить лектору курса Павлу Александровичу Попову (@cpptopchik) за структурированную подачу материала и вдохновение для работы над этим пособием. Также благодарю Илью Семировича Мещерина (@mesyarik) за создание фундаментальной методической базы курса C++ на ФПМИ, которая легла в основу многих объяснений в данной книге. Также благодарю моего семинариста Николая Алексеевича Гарбузова (@nemoumbra) за подробный разбор сложных тем, внимательное отношение к вопросам и проработку множества нюансов, существенно повлиявших на глубину и ясность изложения материала.

Отдельное спасибо спонсорам за поддержку: Алексею Дмитриенко, Егору Демидову, Любови Васильевой, Илье Донову, Коту, Николаю Матвееву, Вадиму М. и Владиславу Х.

Часть 0

**Как мы будем учить C++ и что вообще
происходит**

Эта часть — короткий разгон: настроим окружение, договоримся о терминах и потренируемся читать ошибки. Дальше почти каждая тема будет опираться на привычку *проверять гипотезы кодом*.

Глава 0 Как мы будем учить C++ и что вообще происходит

0.0 Что это за текст

Это учебное пособие по первому семестру C++. Оно устроено так, чтобы ты мог:

- понять логику языка (почему компилятор «придирается» именно к этому);
- быстро проверять идеи короткими примерами;
- научиться читать сообщения компилятора и связывать их с кодом.

Часть 0 — короткий «разгон»: как запускать компилятор, какие слова мы будем использовать (compile-time, runtime, UB), и как извлекать пользу из примеров.

Если что-то непонятно — это нормально. В C++ многое становится ясным только после нескольких запусков кода и пары осмысленных ошибок.

0.1 Где вообще живёт C++ и зачем его учить

C++ чаще всего встречается там, где важны контроль над ресурсами и предсказуемая производительность: движки, высоконагруженные сервисы, библиотеки, системные компоненты, инструменты разработки.

Главная особенность C++ для новичка: язык даёт много свободы, но взамен требует аккуратности. Поэтому в этом пособии мы будем часто говорить не только «как написать», но и «почему так безопаснее».

0.2 Как устроен курс/пособие

Пособие разбито на части и главы. Каждая глава — это:

- короткая теория;
- несколько «минимальных» примеров;
- важные нюансы (почему код может быть ошибочным или опасным).

Лучший режим работы:

1. прочитал раздел;
2. скопировал примеры и запустил;
3. попробовал чуть «сломать» код (поменять тип, удалить строку);
4. прочитал ошибку компилятора и сопоставил её с тем, что изменил.

Если застрял — используй поиск по PDF: по названию раздела или по ключевому слову.

0.3 Немного формальностей: стандарт, компилятор, cppreference

Стандарт C++ — это документ, который описывает правила языка: что означает код и какое поведение считается корректным.

Компилятор (clang++, g++) — программа, которая переводит твой C++-код в машинный код. Компилятор старается следовать стандарту, но у него есть свои диагностики и расширения.

cppreference — удобный справочник по стандартной библиотеке и правилам языка. Это не учебник: там часто сухо, но очень точно. Если хочешь уточнить синтаксис/детали — cppreference почти всегда лучший быстрый источник.

Практическое правило на курс: всегда собирай код в одном и том же стандарте (например, C++20) и с предупреждениями.

0.4 Что от тебя требуется

В этом курсе важно не просто «прочитать», а постоянно проверять понимание кодом.

Минимум, который реально работает:

- запускать примеры самостоятельно;
- не бояться ошибок компилятора и читать их;
- аккуратно относиться к памяти и времени жизни объектов (это будет повторяться часто).

Если что-то не получается — обычно помогает упростить пример до 10–15 строк и воспроизвести проблему в маленьком коде.

0.5 Чем C++ отличается от Python

Три главных отличия, которые будут влиять на обучение:

1. C++ — компилируемый язык: ошибки часто ловятся до запуска программы.
2. Типы важны: компилятор проверяет соответствие типов и не делает «магических» преобразований без правил.
3. Управление ресурсами ближе к «железу»: время жизни объектов, владение памятью и стоимость операций имеют значение.

Это не «хуже» и не «лучше», просто другой стиль: больше контроля — больше ответственности.

0.6 Настраиваем окружение

0.6.1 Вариант 1: онлайн-компилятор (быстрый старт)

Если ты не хочешь ничего ставить прямо сейчас, можно:

- использовать онлайн-компилятор:
 - **Compiler Explorer (godbolt.org)** — идеален для экспериментов с короткими примерами;
 - любой «online C++ compiler» — главное, чтобы можно было указать стандарт `-std=c++17` или `c++20`.

Минусы: неудобно писать большие проекты, нет нормального дебаггера, зависимости и файлы — боль.

0.6.2 Вариант 2: нормальная IDE

Рекомендации:

- **Windows**

- Visual Studio (не Code!) с компонентом «Desktop development with C++»;
- либо VS Code + расширения (C/C++, CMake Tools)
+ установленный компилятор (MinGW-w64 или MSVC).

- **Linux / macOS**

- CLion (бесплатен для некоммерческого использования и по студенческой лицензии; для коммерческих проектов нужна платная подписка);
- или VS Code + g++/clang.

Главное: чтобы ты мог (1) создать `main.cpp`, (2) собрать и запустить программу, (3) видеть сообщения компилятора.

Быстрый «стандартный» вариант сборки

Обычно для учебных примеров хватает одной команды:

Фрагмент

```
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O0 -g main.cpp -o app
```

Коротко про флаги:

- `-std=c++20` — какой стандарт языка использовать;
- `-Wall -Wextra` — включить больше предупреждений;
- `-Wpedantic` — быть строже к нестандартным вещам;
- `-O0 -g` — без оптимизаций и с отладочной информацией (удобно учиться и дебажить).

0.7 Твоя первая программа Hello, world!

Полный пример

```
#include <iostream> // подключаем объявления для std::cout

int main() {          // точка входа: отсюда начинается выполнение программы
    std::cout << "Hello, world!\n";
    return 0;         // 0 обычно означает «успешно»
}
```

Важный нюанс: В C++ можно не писать `return 0;`; если выполнение дошло до конца `main`, это эквивалентно `return 0;`.

0.8 Как мы будем смотреть на код в пособии

0.8.1 Правила листингов: *полный пример* vs *фрагмент*

В книге встречаются два типа блоков кода — и **каждый листинг** явно помечен одним из них:

- **Полный пример** — помечен заголовком «*Полный пример*» в рамке листинга. Такой код можно копировать в отдельный файл и собирать **как есть**.
- **Фрагмент** — кусок кода для иллюстрации идеи: часть функции/класса/файла. Фрагмент может не содержать `#include`, `main` или вспомогательный код, который уже был показан выше.

Важный нюанс: «*Фрагмент*» — это не «плохой пример», а просто кусок кода без каркаса (`#include`, `main`, вспомогательных объявлений). Если хочется быстро проверить фрагмент — используй шаблон ниже.

0.8.2 Мини-шаблон для запуска фрагментов

Если хочешь быстро проверить фрагмент (а это почти всегда полезно) — вставляй его в такой каркас:

Полный пример

```
#include <iostream>

int main() {
    // Вставляй сюда фрагменты и экспериментируй.
    // Если нужен std::vector/std::string - добавь #include <vector>/<string>.
}
```

0.8.3 Про компилируемость и «честность» примеров

- Все *полные примеры* рассчитаны на сборку в стандарте **C++20** с предупреждениями (см. команду сборки выше).
- Если пример в тексте помечен как `CE`, `UB` или `runtime error` — это не «случайно сломанный код», а **намеренная демонстрация**. Мы будем явно это подписывать.
- При подготовке PDF полные примеры дополнительно **прогоняются компилятором** (внутренняя проверка качества), чтобы в книге не оставалось «битых» листингов.

0.8.4 Единый стиль примеров

Чтобы примеры было проще читать и сравнивать, по книге стараемся держать один стиль:

- без `using namespace std`; (лучше видеть, откуда берётся `std::`);
- отступы 4 пробела, `{` на той же строке;
- для перевода строки чаще используем `'\n'` или `"\n"`, а `std::endl` — только когда реально нужен принудительный *flush*;
- в учебных «мини-проверках» предпочитаем маленькие самодостаточные примеры **до 10–20 строк**.

Максимум пользы обычно получается так:

1. копируешь код к себе;
2. запускаешь сам, пробуешь **испортить** (удалить строчку, поменять тип, дописать что-то);
3. читаешь, что говорит компилятор, и стараешься понять, что ему не нравится.

0.9 Ошибки: compile-time vs runtime (очень коротко)

Мы будем постоянно говорить про разные виды ошибок (подробно позже, но минимальный словарь нужен уже сейчас):

- **Compile-time error (CE)** — ошибка компиляции. Пример: забыл `;`, написал неверный тип, пробуешь вызвать несуществующую функцию. Программу даже не соберут.
- **Runtime error** — программа собралась и запустилась, но аварийно завершилась в процессе работы (например: стек переполнился при глубокой рекурсии, сработал `assert/std::terminate`, не поймали исключение, операционная система убила процесс и т. п.).
- **Undefined behavior (UB)** — ты сделал что-то, что стандарт запрещает (например, вышел за границы массива, разыменовал `nullptr`, поделил целые на ноль, полез в чужую память и т. д.). Компилятор не обязан тебя защищать: программа может упасть, может «работать», а может тихо ломать данные. UB может не проявиться сразу: программа иногда «работает», а ломается позже — поэтому UB важно избегать, даже если «на моём компьютере всё нормально».

В курсе/книге мы будем регулярно подчёркивать:

- «вот тут UB»;
- «здесь CE»;
- «тут runtime-ошибка».

0.10 Как читать это пособие с максимальной пользой

Рекомендуемый режим:

1. читаешь раздел (например, про указатели);
2. сразу прогоняешь примеры у себя;
3. если что-то не понятно — смотри по оглавлению/поиском по PDF или на `srpreference`.

Нормальный темп на семестр: 1–2 теоретические части в неделю + практика (решение задач). Не копи всё «на потом» — с плюсами так не работает.

Окружение настроено, термины зафиксированы, и мы понимаем, что такое ошибка компиляции и ошибка во время выполнения. Теперь можно переходить к коду: начнём с того, как исходники превращаются в исполняемый файл, а затем разберём структуру программы, объявления и области видимости — это быстро окупается, когда код перестаёт помещаться в один файл.

Глава 1 Инженерный минимум: сборка, заголовки, диагностика

Эта глава короткая и очень практичная: она экономит часы (и нервы) на всём дальнейшем материале. Здесь разберёмся, что именно делает компилятор, почему иногда ошибка вылезает только на этапе линковки, как правильно раскладывать код по `.h` и `.cpp`, и какие флаги резко снижают число случайных ошибок и UB в учебных проектах.

1.1 Компиляция и линковка: два разных этапа

В самом грубом приближении путь от текста к программе выглядит так:

1. **Препроцессор** разворачивает `#include`, `#define` и условную компиляцию.
2. **Компиляция** переводит каждый `.cpp` в объектный файл (`.o` / `.obj`).
3. **Линковка** склеивает объектные файлы и библиотеки в исполняемый файл.

Отсюда два классических вида ошибок:

- **Compile-time error**: синтаксис/типы/доступы — компилятор не смог собрать `.o`.
- **Linker error**: компилятор собрал `.o`, но линковщик не нашёл реализацию функции/переменной или нашёл её несколько раз.

1.1.1 Практикум: как «посмотреть внутрь» сборки (семинары)

Почти любой современный компилятор позволяет останавливать сборку на промежуточной стадии:

Фрагмент

```
# 1) Только препроцессор: получаем «чистый» .cpp без #include
clang++ -std=c++20 -E main.cpp > main.i

# 2) До ассемблера
clang++ -std=c++20 -S main.cpp -o main.s

# 3) Только компиляция в объектник (без линковки)
clang++ -std=c++20 -c main.cpp -o main.o

# 4) Линковка из .o
clang++ main.o other.o -o app
```

Под капотом: На семинарах часто полезно посмотреть `.i` (после препроцессора): это помогает отлаживать «странные» ошибки из-за макросов/`#include`. А `.s` хорошо объясняет, что именно оптимизирует компилятор.

1.1.2 Мини-проект: несколько файлов

Классический учебный формат:

- `app.h` — объявления;
- `app.cpp` — определения;
- `main.cpp` — точка входа.

Сборка одной командой:

Фрагмент

```
clang++ -std=c++20 -Wall -Wextra -Wpedantic -O0 -g \
main.cpp app.cpp -o app
```

Если вы забыли добавить `app.cpp` в команду, то часто получите ошибку линковщика вида:

Фрагмент

```
undefined reference to 'foo(...)'
```

1.2 .h и .cpp: что куда класть

Удобная (и почти всегда правильная) договорённость:

- в `.h` лежат **объявления** (interfaces): сигнатуры функций, объявления классов, структуры, enum, constexpr/inline мелочи;
- в `.cpp` лежат **определения** (implementations): тела функций, методы классов, «тяжёлый» код.

Почему так? Потому что `#include` — это *буквальная вставка текста*. Если в заголовке разместить определения обычных функций/глобальных переменных и подключить этот заголовок в двух `.cpp`, линковщик честно скажет: **multiple definition**. Это связано с правилом **ODR** (one definition rule): у большинства сущностей в программе должна быть ровно одна “настоящая” реализация.

1.2.1 Шаблоны — исключение

Шаблон (*template*) — это код с параметрами (обычно по типам): одна и та же функция или класс, который может работать с разными типами данных. Шаботонный код обычно живёт в заголовках, потому что компилятору нужно *видеть* определение при инстанцировании. В конце книги мы ещё вернёмся к этому в части про шаблоны.

1.3 Include guards: защита от двойного подключения

Так как `#include` вставляет текст, один и тот же заголовок может оказаться подключённым несколько раз (напрямую или транзитивно). Чтобы компилятор не видел повторные объявления, заголовок защищает **include guards**:

Фрагмент

```
// my_header.h
#ifndef MY_HEADER_H
#define MY_HEADER_H
```

```
struct Point {
    int x;
    int y;
};

#ifdef MY_HEADER_H
```

Альтернатива, которую поддерживают почти все современные компиляторы: `#pragma once`. Это проще, но не является частью стандарта (хотя де-факто распространено).

1.4 Предупреждения компилятора: включаем по максимуму

Компилятор умеет находить подозрительные места ещё до запуска программы. В учебном коде это маст-хэв: лучше получить 10 предупреждений сейчас, чем 1 странный баг на защите.

Рекомендованный минимум для GCC/Clang:

Фрагмент

```
-std=c++20 -Wall -Wextra -Wpedantic
```

Важный нюанс: Мы считаем, что код и листинги в этом пособии должны компилироваться **без предупреждений** как минимум под набором выше. Именно его и используйте как «базовый режим».

Чуть строже (опционально, «параноидальный режим»):

Фрагмент

```
-Wconversion -Wsign-conversion -Wshadow
```

Частая ловушка: `-Wconversion` и `-Wsign-conversion` могут давать много предупреждений даже на идиоматичном коде (например, при работе с `size_t` и индексами). Это полезный режим, чтобы *научиться* видеть потенциально опасные места, но не обязательно держать его включённым всегда.

Опционально можно временно включать `-Werror` (превращает предупреждения в ошибки), но аккуратно: в чужих библиотеках и на разных платформах это иногда мешает жить.

1.5 Санитайзеры: ловим ошибки в рантайме

Санитайзеры — это режим сборки, в котором компилятор добавляет проверки и помогает поймать типичные проблемы: выход за границы массива, `use-after-free`, двойное освобождение памяти, некоторые виды UB и т.д.

Самые полезные для старта:

- **ASan** (AddressSanitizer) — ошибки памяти;
- **UBSan** (UndefinedBehaviorSanitizer) — некоторые случаи UB;

Типичная команда для GCC/Clang в режиме отладки:

Фрагмент

```
clang++ -std=c++20 -O1 -g -fno-omit-frame-pointer \
-fsanitize=address,undefined main.cpp -o main
```

Важно: санитайзеры включают *другую* модель исполнения, поэтому их обычно держат отдельной конфигурацией (Debug/ASan). А когда ты ловишь баг, очень помогает правило: **воспроизводимость > скорость**.

1.6 Под капотом: символы, манглинг, ABI (по желанию)

Этот раздел не обязателен для первых программ, но он помогает объяснить, *почему* некоторые вещи в C++ устроены именно так.

1.6.1 Символы и манглинг имён

Под капотом: Линковщик работает не с «функциями из исходника», а с **символами** (именами в объектниках). В C++ есть **перегрузка функций**, поэтому компилятор обычно кодирует типы параметров прямо в имени символа. Это и есть **name mangling**.

Посмотреть символы можно инструментом `nm`:

Фрагмент

```
clang++ -std=c++20 -c main.cpp -o main.o
nm -C main.o | head
```

Флаг `-C` у `nm` просит «деманглинг» символы обратно в читаемый вид.

1.6.2 Зачем нужен `extern "C"`

Иногда нужно вызвать C++-функцию из C (или из другого языка через C ABI). Тогда используют:

Фрагмент

```
extern "C" int c_api_sum(int a, int b);
```

В таком режиме перегрузка не работает, а имена символов становятся совместимыми с C.

1.6.3 Calling convention и ABI

Под капотом: **Calling convention** определяет, как именно параметры передаются в функцию (через регистры/стек), кто чистит стек и где лежит возвращаемое значение. **ABI** (application binary interface) фиксирует набор таких договорённостей для платформы/компилятора. На практике ABI зависит от ОС и архитектуры, поэтому «как именно» передаются аргументы лучше воспринимать как деталь реализации.

Теперь у нас есть базовая инженерная инфраструктура: мы понимаем, где искать ошибки компиляции/линковки и как включать диагностику. Можно спокойно переходить к «скелету» программы: объявлениям, областям видимости, функциям и пространствам имён.

Глава 2 Структура программы: объявления, функции и пространства имён

Мы настроили окружение и договорились, как запускать компилятор. Теперь разберём «скелет» C++-программы: как компилятор узнаёт про имена, чем объявление отличается от определения, как устроены области видимости и зачем нужны пространства имён. Это снимет много вопросов, которые обычно возникают при разнесении кода по файлам и при первой встрече с линковщиком.

2.1 Объявление и определение

Объявление (declaration) вводит имя и тип сущности (переменной, функции, типа).

Определение (definition) — это объявление, которое ещё и «создаёт сущность»:

- для **переменной** выделяет память (задаёт объект хранения);
- для **функции** содержит тело (реализацию).

Важный нюанс: Важно не запутаться в терминологии: **любое определение является объявлением, но не любое объявление является определением**. Поэтому фраза «объявим переменную» в разговорной речи иногда означает и определение тоже.

2.1.1 Переменные

Полный пример

```
int g = 10;           // определение (и инициализация)
int h;                // тоже определение (глобальная переменная будет 0)
extern int ext;        // только объявление: память здесь НЕ выделяется

int main() {
    int x = 5;         // определение (локальная переменная)
    int y;             // определение (локальная переменная, неинициализирована!)

    (void)x;
    (void)&y;
}
```

Частая ловушка: В C++ строка `int h;` является определением и на глобальном уровне создаёт переменную, которая будет **нулём инициализирована**. Локальная переменная `int y;` — тоже определение, но **значение не задано** (использование без инициализации часто приводит к UB).

2.1.2 Функции

Фрагмент

```
int sum(int a, int b);           // объявление (прототип)
int sum(int, int);              // повторное объявление (разрешено)

int sum(int a, int b) {         // определение
    return a + b;
}
```

Важный нюанс: Возвращаемый тип **не входит в сигнатуру** (нельзя перегрузить функции, отличающиеся только return). См. также раздел про перегрузку ниже.

2.1.3 ODR в одном абзаце

ODR (One Definition Rule) в упрощённой формулировке:

- в рамках *одной единицы трансляции* (одного .cpp после препроцессора) имя должно быть определено однозначно;
- во всей программе (после линковки) для большинства сущностей допустимо **ровно одно определение**.

Если нарушить ODR, часто получится **linker error**: «multiple definition of ...». Практику с несколькими файлами см. в главе 1.

2.2 Область видимости и перекрытие имён

Область видимости (scope) — участок кода, где имя доступно. Типичный случай: локальная переменная перекрывает (*shadowing*) глобальную с тем же именем.

Полный пример

```
#include <iostream>

int x = 100; // глобальная

int main() {
    int x = 5;           // локальная перекрывает глобальную
    std::cout << x << "\n"; // 5
    std::cout << ::x << "\n"; // 100 (оператор области видимости ::)
}
```

Частая ловушка: Не злоупотребляйте теневыми именами: это типичный источник багов. Если очень нужно — выбирайте сильно отличающиеся имена (global_x, local_x).

2.3 Пространства имён (namespace)

Namespace — способ группировать имена и избегать конфликтов.

Полный пример

```
namespace math {
    int add(int a, int b) { return a + b; }
}

namespace math { // пространство имён можно «дополнять»
    int sub(int a, int b) { return a - b; }
}

int main() {
    int x = math::add(2, 3);
    return x;
}
```

Важный нюанс: Директива `using namespace std;` в маленьких учебных программах кажется удобной, но в реальных проектах её стараются избегать, особенно в заголовках: она «вытаскивает» много имён в глобальную область и может создавать конфликты. Часто безопаснее писать `std::` или делать точечные `using std::cout;` внутри функции.

2.3.1 Анонимное пространство имён

Фрагмент

```
namespace {           // «видно» только в этом .cpp
    int internal = 42;
}
```

Это удобно для «внутренних» функций и переменных, которые не должны быть доступны из других файлов.

Важный нюанс: Исторически похожую роль играло `static` на уровне файла (*internal linkage*), но в современном C++ чаще используют анонимный namespace: он лучше сочетается с типами и шаблонами.

2.4 Предварительные объявления (forward declaration)

Иногда нам нужно сообщить компилятору «такой тип/функция существует», но не раскрывать детали прямо сейчас. Это называется **предварительным объявлением**.

Фрагмент

```
struct Node;           // Node объявлен, но не определён (incomplete type)
```

```

struct List {
    Node* head = nullptr; // OK: указатель на неполный тип разрешён
};

struct Node {           // теперь определение
    int value = 0;
    Node* next = nullptr;
};

```

Частая ловушка: Неполный тип можно использовать только ограниченно: можно объявлять указатели/ссылки, но нельзя делать `sizeof(Node)` или хранить `Node` «по значению» до определения.

2.5 Функции

2.5.1 Перегрузка функций

Функции можно **перегружать** (одинаковое имя, разные параметры):

Фрагмент

```

void f(int);
void f(double);

int main() {
    f(10);    // вызов f(int)
    f(1.5);   // вызов f(double)
}

```

Важный нюанс: Перегрузка выбирается в **compile-time** по типам аргументов (см. также главу про выражения и приведения типов).

2.5.2 Значения по умолчанию

Полный пример

```

#include <iostream>

void print_n(int x = 10) {
    std::cout << x << "\n";
}

int main() {
    print_n();    // 10
    print_n(7);   // 7
}

```

Частая ловушка: Значения по умолчанию **подставляются в месте вызова** (в compile-time). Поэтому для **виртуальных** функций значение по умолчанию определяется **статическим типом** ссылки/указателя, а не динамическим типом объекта (к этому вернёмся в главе про полиморфизм).

2.5.3 Передача аргументов

На этом этапе полезно знать минимум:

- **по значению** (`f(int x)`) — копия аргумента;
- **по ссылке** (`f(int& x)`) — можно изменить исходный объект;
- **по константной ссылке** (`f(const std::string& s)`) — без копии и без изменения.

Детально про ссылки, указатели, времена жизни и «когда что выбирать» поговорим в главе *Указатели и ссылки*.

2.5.4 Указатели на функции и компаратор для сортировки

Иногда функцию нужно передать как аргумент другой функции. Например, компаратор для сортировки:

Полный пример

```
#include <algorithm>
#include <cstdlib>    // std::abs(int)
#include <vector>

bool by_abs(int a, int b) {
    return std::abs(a) < std::abs(b);
}

int main() {
    std::vector<int> v = {3, -10, 7, -2};
    std::sort(v.begin(), v.end(), by_abs); // передали функцию как объект
}
```

Важный нюанс: Указатель на функцию можно записать явно: `bool (*cmp)(int, int) = by_abs;`, но обычно компилятор сам всё выводит. Позже вместо «чистых» указателей на функции часто используют **лямбды** (они короче) и `std::function` (если нужна динамическая полиморфность).

Теперь мы понимаем, как устроен «скелет» программы: объявления и определения, области видимости и пространства имён. Дальше разберём типы и литералы — базовые кирпичики языка.

Часть 1

Базовые типы и выражения

Эта часть — фундамент: мы договоримся, что такое типы и литералы, как из них собираются выражения, и почему «одна и та же цифра» может означать разные вещи в зависимости от суффикса и контекста. Когда база будет в голове, дальше станет проще: все условия, циклы, функции и классы в конечном счёте состоят из выражений.

Глава 3 Типы и литералы

3.1 Что такое тип

Тип — это информация о том, какие значения может принимать объект и что с ним можно делать.

C++ — **статически типизированный язык**: тип переменной известен уже во время компиляции и не меняется в рантайме. Если мы написали:

Фрагмент

```
int a = 5;
```

то `a` — это `int` и сделать из него внезапно `double` без явного приведения нельзя. В Python переменная может «изображать из себя» то число, то строку — интерпретатор подстраивается на ходу.

Зачем нам статическая типизация?

- Компилятор раньше находит ошибки.
- Можно генерировать более быстрый код (меньше проверок в runtime).
- Проще понимать, что происходит с памятью и значениями.

3.2 Базовые встроенные типы C++

Разберём самые базовые кирпичики.

3.2.1 Целочисленные типы

Классика:

Фрагмент

```
short    // "маленький" целый
int       // обычный целый
long     // больше int
long long // ещё больше
```

Каждый из них может быть:

Фрагмент

```
signed // со знаком (по умолчанию)
unsigned // без знака (только >= 0)
```

Примеры:

Фрагмент

```
int x = -10;
unsigned int y = 10u;
long long big = 1'000'000'000LL;
```

Размеры зависят от платформы, но в типичной реальности: **int** — 4 байта, **long long** — 8 байт.

В стандартной библиотеке есть фиксированные типы (ровно N байт):

Фрагмент

```
#include <cstdint>

int8_t  a; // 1 байт
int16_t b; // 2 байта
int32_t c; // 4 байта
int64_t d; // 8 байт
```

Чтобы не путаться с размером int и друзей.

Ещё есть полезный тип:

Фрагмент

```
#include <cstddef>

size_t n; // используется для размеров, индексов
```

Он **беззнаковый** и достаточно большой, чтобы описать размер любого объекта в памяти.

Важный нюанс: Как прикинуть диапазон «в уме». Если у **беззнакового** целого типа n битов, то значения лежат в диапазоне $0 \dots (2^n - 1)$.

Для **знакового** целого (на практике почти всегда — двоичное дополнение) диапазон обычно $-2^{n-1} \dots (2^{n-1} - 1)$. Например, для 8 бит: unsigned 0..255, signed -128..127.

Но размер типа в C++ не «прошит»: выясняйте число битов через `sizeof(T) * CHAR_BIT` или через `std::numeric_limits<T>::digits`, а точные границы — через `std::numeric_limits<T>::min()/max()`.

Фрагмент

```
#include <climits>
#include <limits>
#include <iostream>

std::cout << "bits in int = " << sizeof(int) * CHAR_BIT << "\n";
std::cout << "int min = " << std::numeric_limits<int>::min() << "\n";
std::cout << "int max = " << std::numeric_limits<int>::max() << "\n";
```

3.2.2 Вещественные типы (с плавающей точкой)

- float
- double
- long double

Хранят числа вида: знак + мантисса + экспонента (примерно как научная нотация $1.23 \cdot 10^9$, только в двоичной системе).

Главная идея:

- `float` — 4 байта, около 7 значащих цифр;
- `double` — 8 байт, около 15–16 значащих цифр;
- `long double` — ещё точнее (зависит от компилятора).

Из-за конечной точности:

Фрагмент

```
double x = 0.1;
double y = 0.2;
std::cout << (x + y == 0.3); // часто выведет 0 (false)
```

Числа в двоичной системе не всегда представимы точно, возникают ошибки округления.

Частая ловушка: У вещественных типов есть маленькая ловушка: `std::numeric_limits<double>::min()` — это **минимальное положительное нормализованное** число, а не «самое отрицательное».

Если нужен «самый маленький по значению» (то есть наиболее отрицательный) — берите `lowest()`.

3.2.3 Символьный тип

Фрагмент

```
char c = 'A';
```

`char` — ровно 1 байт. Чаще всего используется для хранения одного символа (но с Unicode нужно аккуратно — это отдельная история).

Есть ещё `signed char`, `unsigned char`, `wchar_t`, `char16_t`, `char32_t`, но пока достаточно знать, что `char` — «маленький» целый, который мы интерпретируем как символ.

3.2.4 Логический тип

Фрагмент

```
bool ok = true;
```

Может быть только `true` или `false`. Внутри обычно занимает 1 байт (хотя по ощущениям просится 1 бит).

3.2.5 Указатели и ссылки (тизер)

Пока только тизер, детали будут в отдельной главе.

- **Указатель** (`int* p`) — хранит адрес объекта в памяти.
- **Ссылка** (`int& r`) — псевдоним уже существующей переменной.

В этой главе они нужны только для понимания литерала `nullptr`.

3.3 Что такое литерал

Литерал — это значение, написанное прямо в коде:

Фрагмент

```
42          // целочисленный литерал
3.14        // вещественный
'a'         // символьный
"hello"     // строковый
true        // булев
nullptr     // нулевой указатель
```

Литерал — это не переменная, это «голое значение». В упрощённой модели: литералы — «временные значения» (rvalue). **Очень грубо:** lvalue — это выражение, которое обозначает конкретный объект в памяти (у него есть адрес), а rvalue — временное значение. Полная картина категорий выражений (lvalue/prvalue/xvalue и почему это важно) — в главе про ссылки и перемещение.

3.4 Целочисленные литералы

3.4.1 Десятичные

Фрагмент

```
int a = 42;          // десятичный
int b = 1'000'000;   // разделитель разрядов
```

Число 42 по умолчанию имеет тип int (пока влезает в диапазон int).

3.4.2 Другие системы счисления

В C++ можно записывать литералы в нескольких системах:

Фрагмент

```
int dec = 42;        // десятичное
int hex = 0x2A;      // шестнадцатеричное (0x)
int oct = 052;       // восьмеричное (0 в начале)
int bin = 0b101010;  // двоичное (0b, C++14+)
```

Все они представляют одно и то же число — 42. Можно использовать ' для удобства:

Фрагмент

```
int mask = 0b1111'0000;
int color = 0xFF'00'FF;
```

3.4.3 Суффиксы для целых

Чтобы явно указать тип, используем суффиксы:

Фрагмент

```
42u    // unsigned int
42l    // long
42ul   // unsigned long
42ll   // long long
42ull  // unsigned long long
```

Компилятор выбирает минимальный тип, в который число помещается. Если хотите контролировать тип — используйте суффиксы.

Осторожно с signed/unsigned:

Фрагмент

```
int a = -1;
unsigned int b = 1u;

std::cout << (a < b); // часто неожиданно выведет 0
```

При сравнении `a` неявно приводится к `unsigned`, и `-1` преобразуется к беззнаковому по модулю 2^N , превращаясь в огромное положительное число.

3.5 Вещественные литералы

Формы записи:

Фрагмент

```
double x = 3.14;
double y = .5;      // 0.5
double z = 1.;      // 1.0

double e = 1e-3;     // 0.001 (1 * 10^-3)
double big = 1.23e8; // 123000000
```

3.5.1 Суффиксы

По умолчанию вещественный литерал без суффикса — это `double`.

Фрагмент

```
float f = 3.14f;      // суффикс f/F
long double ld = 3.14L; // суффикс l/L
```

Чем больше тип, тем выше точность (и дороже операции).

3.6 Символьные и строковые литералы

3.6.1 Символьные литералы

Фрагмент

```
char c1 = 'a';
char c2 = 'Z';
char c3 = '\n'; // перевод строки
char c4 = '\t'; // табуляция
char c5 = '\''; // одинарная кавычка
char c6 = '\\'; // обратный слэш
```

Есть ещё `\x41` — шестнадцатеричный код ('A') и `\101` — восьмеричный код:

Фрагмент

```
char c7 = '\x41'; // 'A'
char c8 = '\101'; // тоже 'A'
```

Для Unicode-типов:

Фрагмент

```
char16_t c16 = u'λ';
char32_t c32 = U'λ';
wchar_t wc = L'Ж';
```

Мультисимвольные литералы вроде `'ab'` существуют, но ведут себя странно (это один `int` с закодированными символами) — не используем.

3.6.2 Строковые литералы

Самый классический вариант:

Фрагмент

```
const char* s = "Hello, world!\n";
```

Здесь `const` означает «только чтение»: через такой указатель мы не должны менять символы. Строковый литерал — это **массив** `const char`, в конце которого лежит нулевой байт `'\0'` — терминатор строки.

Строковые литералы нельзя менять:

Фрагмент

```
const char* s = "hi"; // ОК: читать можно, менять нельзя
// s[0] = 'H';        // ошибка компиляции (нельзя менять литерал)

// Если попытаться <<обойти>> запрет (например, сняв const приведением типов),
// вы попадёте в UB: строковый литерал может лежать в памяти только для чтения.

// Если нужна изменяемая строка - делай массив:
```

```
char a[] = "hi";
a[0] = 'H'; // OK
```

Так что литералы читаем через `const char*`, а для изменяемых строк используем массив или `std::string` (о нём позже).

Конкатенация строковых литералов

Компилятор умеет автоматически склеивать литералы, стоящие подряд:

Фрагмент

```
const char* s = "Hello, "
                "world!"; // то же, что "Hello, world!"
```

Удобно для длинных строк.

Префиксы строк

Разные варианты кодировок:

Фрагмент

```
const char*    s1 = "text"; // обычная (узкая) строка
const char8_t* s2 = u8"text"; // UTF-8 (C++20)
const char16_t* s3 = u"text"; // UTF-16
const char32_t* s4 = U"text"; // UTF-32
const wchar_t* s5 = L"text"; // широкая строка
```

Raw-строки

Чтобы не писать миллион обратных слэшей, есть сырые строки:

Фрагмент

```
const char* path = R"(C:\Users\Student\Documents\file.txt)";
```

Внутри `R"( ... )"` почти всё воспринимается буквально — можно писать `\`, `"` и многострочный текст:

Фрагмент

```
const char* html = R"(
<html>
  <body>Привет</body>
</html>
)";
```

3.7 Литералы bool и nullptr

3.7.1 bool

Фрагмент

```
bool ok = true;
bool fail = false;
```

Неявные преобразования:

Фрагмент

```
bool b1 = 0;      // false
bool b2 = 42;     // true
bool b3 = -10;    // true
```

Обратно:

Фрагмент

```
int x = true;    // 1
int y = false;   // 0
```

Лучше не злоупотреблять такими преобразованиями и явно приводить типы.

3.7.2 Нулевой указатель: nullptr

Раньше было модно писать `int* p = NULL;` или `0`. Сейчас в C++ есть специальный литерал:

Фрагмент

```
int* p = nullptr;
```

Он имеет тип `std::nullptr_t` и однозначно означает нулевой указатель.

Почему `nullptr` лучше:

- не путается с целыми числами;
- перегрузки функций с `nullptr` выбираются корректнее;
- читается очевиднее.

3.8 auto и тип литерала

Тип, который получит переменная с `auto`, зависит от литерала справа:

Фрагмент

```
auto a = 42;      // int
auto b = 42u;     // unsigned int
auto c = 42LL;    // long long

auto d = 3.14;    // double
auto e = 3.14f;   // float

auto f = 'A';     // char
auto g = "A";     // const char*: строковый литерал - это массив, но в выражениях почти всегда
                  // «деградирует» до указателя на первый элемент
// Если нужно сохранить именно массивный тип: auto& r = "A"; // r имеет тип const char (&)[2]
```

```
auto h = true;    // bool
auto p = nullptr; // std::nullptr_t
```

Важно помнить: auto просто подставляет тип на основе выражения справа — и литералы здесь играют ключевую роль.

3.9 Константы и литералы: const

Литерал — это значение в коде. Часто удобно «дать ему имя»:

Фрагмент

```
const int DAYS_IN_WEEK = 7;
const double PI = 3.141592653589793;
```

- const запрещает менять значение.
- Компилятор может оптимизировать такие константы (часто подставляет значение прямо в код).

Литерал сам по себе неизменяем, но если вы присвоите его неконстантной переменной — переменную уже можно менять:

Фрагмент

```
int x = 10;    // 10 --- литерал, x --- изменяемая переменная
x = 20;
```

Мы разобрались, какие бывают значения и как их записывать. Следующий шаг — научиться из этих значений собирать выражения: операторы, их приоритеты и места, где без скобок легко получить не то, что задумал.

Глава 4 Операторы и приоритеты

4.1 Что такое оператор

Оператор — это специальный символ (или набор символов), который говорит компилятору: «сделай вот такую операцию над вот этими значениями».

Примеры:

Фрагмент

```
a + b      // сложение
x = y      // присваивание
x < y      // сравнение "меньше"
!flag      // логическое отрицание
++i        // инкремент
```

Операторы в C++ — это грамматика математики и логики языка. Без них пришлось бы писать что-то вроде `add(a, b)` вместо привычного `a + b`.

4.2 Классификация операторов

Операторов в C++ много, но для старта достаточно держать в голове базовые группы.

4.2.1 По количеству операндов

- **Унарные** — один операнд: `+x`, `-x`, `!x`, `~x`, `++x`, `x++`, `--x`, `x--`, `*p`, `&x`.
- **Бинарные** — два операнда: `a + b`, `a - b`, `a * b`, `a / b`, `a % b`, `a && b`, `a || b`, `a & b`.
- **Тернарный** — три операнда: `condition ? expr1 : expr2`. Название «тернарный» относится к числу операндов, а не к числу символов.

4.2.2 По смыслу (основные группы)

- **Арифметические:** `+`, `-`, `*`, `/`, `%`.
- **Присваивания:** `=`, `+=`, `-=`, `*=`, `/=`, `%=` и т. п.
- **Сравнения:** `==`, `!=`, `<`, `>`, `<=`, `>=`.
- **Логические:** `!`, `&&`, `||`.
- **Побитовые:** `&`, `|`, `^`, `~`, `<<`, `>>`.
- **Инкремент/декремент:** `++`, `--`.
- **Прочее:** `?:`, оператор `,` (запятая), `sizeof`, `new`, `delete`, `.`, `->`, `[]`, `()` (вызов функции).

Мы не ныряем сразу во всё, но картину полезно держать в голове.

4.3 Приоритет и ассоциативность

Когда в выражении много операторов, компилятору нужно понять, в каком порядке всё вычислять.

Пример:

Фрагмент

```
int x = 1 + 2 * 3;
```

Сначала умножение или сложение? Правильный ответ: сначала $2 * 3$, потому что $*$ приоритет выше, чем $+$.

4.3.1 Приоритет

Приоритет — это «старшинство» операторов. Чем выше приоритет, тем раньше оператор срабатывает.

Очень упрощённая лестница (сверху — самое сильное):

- Скобки: ().
- Унарные: + - ! ++ -- (префиксные), приведения типов.
- * / %.
- + -.
- Сдвиги: << >>.
- Сравнения: < <= > >=.
- Равенство: == !=.
- Побитовые: &, |, ^.
- Логические: &&, затем ||.
- Тернарный ?:.
- Присваивания: =, += и т. д.
- Запятая (оператор ,).

Необязательно зубрить всю таблицу. Достаточно помнить базовые уровни (* /, потом + -, потом сравнения, потом логика) и не стесняться ставить скобки.

4.3.2 Ассоциативность

Если у операторов одинаковый приоритет, важен порядок применения: слева направо или справа налево.

- Бинарное вычитание ассоциируется слева направо:

Фрагмент

```
int x = 10 - 5 - 2; // (10 - 5) - 2
```

- Присваивание ассоциируется справа налево:

Фрагмент

```
a = b = c = 0; // a = (b = (c = 0))
```

4.4 Арифметические операторы

Частая ловушка: Переполнение знаковых целых (int, long и т. п.) — это **undefined behavior**. У беззнаковых (unsigned) арифметика идёт **по модулю 2^n** : после максимума снова 0.

Поэтому unsigned удобно для битовых масок и модульной арифметики, но опасно, если вы ожидаете «ошибку при переполнении».

4.4.1 Базовый набор

Фрагмент

```
+ // сложение
- // вычитание
* // умножение
/ // деление
% // целочисленный остаток
```

Работают с целыми и вещественными типами (но % — только с целыми).

Примеры:

Фрагмент

```
int a = 5 + 3;    // 8
int b = 5 - 10;   // -5
int c = 2 * 3;    // 6
int d = 7 / 2;    // 3 (целочисленное деление)
int e = 7 % 2;    // 1

double x = 7.0 / 2.0; // 3.5
double y = 7 / 2.0;   // 3.5 (int приводится к double)
```

Важно: 7 / 2 даст 3, а не 3.5, потому что оба операнда целые.

4.4.2 Деление и остаток

Для целых a / b — деление с отбрасыванием дробной части, $a \% b$ — остаток, такой, что примерно выполняется $a == (a / b) * b + (a \% b)$ (для положительных чисел).

С отрицательными числами поведение / и % тоньше, так что на старте лучше избегать подобных комбинаций.

4.4.3 Переполнение

Целочисленные типы имеют ограниченный диапазон. Если его превысить:

- для знаковых типов (int) получаем undefined behavior;
- для беззнаковых (unsigned) происходит «кольцевое» переполнение по модулю 2^N .

Почему для знаковых переполнение — UB? Стандарт позволяет компилятору исходить из того, что переполнения не случается, и на этой базе агрессивно оптимизировать: убирать ветки вроде `if (x + 1 < x)`, переупорядочивать вычисления в циклах и т.п. У беззнаковых типов правило проще: арифметика идёт по модулю, поэтому никакой UB нет, просто происходит wraparound.

Пример плохого кода:

Фрагмент

```
int x = 2'000'000'000;
int y = 2'000'000'000;
int z = x + y; // переполнение
```

Компилятор честно сложит, но результат будет мусор.

4.5 Операторы присваивания

4.5.1 Обычное присваивание

Фрагмент

```
int x = 5;
x = 10;
```

Оператор = не сравнивает, а присваивает. Выражение `x = 10` возвращает значение 10 и является выражением категории lvalue (может стоять слева от следующего присваивания).

Цепочка:

Фрагмент

```
int a, b, c;
a = b = c = 0; // допустимо, работает справа налево
```

4.5.2 Составные операторы присваивания

Вместо:

Фрагмент

```
x = x + 5;
```

обычно пишут:

Фрагмент

```
x += 5;
```

Аналогично:

Фрагмент

```
x -= 3; // x = x - 3;
x *= 2; // x = x * 2;
x /= 10; // x = x / 10;
x %= 2; // x = x % 2;
```

Есть ещё `&=`, `|=`, `^=`, `<<=`, `>>=`.

Плюсы составных операторов: короче запись, иногда эффективнее для сложных типов, выглядит привычно для C/C++.

4.6 Операторы сравнения

Список:

Фрагмент

```
== // равно
!= // не равно
< // меньше
> // больше
<= // меньше или равно
>= // больше или равно
```

Результат — bool (true или false).

Примеры:

Фрагмент

```
int x = 5, y = 10;
bool a = (x < y); // true
bool b = (x == y); // false
bool c = (x != y); // true
```

4.6.1 Важное предупреждение

Классический баг новичка:

Фрагмент

```
if (x = 0) { // ой
    // ...
}
```

Здесь стоит присваивание, а не сравнение. `x = 0` вернёт 0, а `if (0)` — это false.

Правильно:

Фрагмент

```
if (x == 0) {
    // ...
}
```

Некоторые пишут `if (0 == x)`, чтобы забытый знак `=` превратился в ошибку компиляции.

4.6.2 Сравнение вещественных чисел

С double и float лучше не сравнивать на точное равенство:

Фрагмент

```
double a = 0.1 + 0.2;
if (a == 0.3) { /* ... */ } // может не сработать
```

Обычно сравнивают с учётом погрешности:

Фрагмент

```
double eps = 1e-9;
if (std::abs(a - 0.3) < eps) {
    // считаем, что равны
}
```

Функция `std::abs` для `double` лежит в `<cmath>`.

4.7 Логические операторы и короткое замыкание

4.7.1 Основные операторы

Фрагмент

```
!x      // логическое НЕ
x && y   // логическое И
x || y   // логическое ИЛИ
```

Работают с `bool`, но любое значение неявно приводится: ноль $\rightarrow$ `false`, всё остальное $\rightarrow$ `true`.

Примеры:

Фрагмент

```
bool a = true;
bool b = false;

bool c = !a;      // false
bool d = a && b;   // false
bool e = a || b;  // true
```

4.7.2 Короткое замыкание (short-circuit)

Операторы `&&` и `||` вычисляют операнды не всегда.

- `x && y`: если `x` — `false`, `y` не вычисляется.
- `x || y`: если `x` — `true`, `y` не вычисляется.

Классика:

Фрагмент

```
int a = 10;
int b = 0;

if (b != 0 && a / b > 1) {
    // безопасно: деление выполняется только если b != 0
}
```

Если поменять порядок, можно поймать UB (деление на ноль):

Фрагмент

```
int a = 10;
```

```
int b = 0;

if (a / b > 1 && b != 0) { // плохо: a / b вычислится раньше проверки
    // ...
}
```

4.7.3 Логические vs побитовые

Не путать:

- `&&` и `||` — логические, результат `bool`, есть короткое замыкание.
- `&`, `|`, `^` — побитовые, работают с целыми побитно, вычисляют оба операнда.

4.8 Инкремент и декремент

Частая ловушка: Классическая «мина»: не модифицируйте одну переменную несколько раз в одном выражении без явного порядка. До C++17 многие комбинации вроде `i = i++, a[i] = i++` или `f(i, i++)` попадали в UB из-за неупорядоченных побочных эффектов; в новых стандартах часть случаев стала определённой, но стиль всё равно плохой. Если хочется и увеличить, и использовать — разнесите на две строки.

Операторы `++` и `--` увеличивают/уменьшают переменную на 1. Есть две формы:

Фрагмент

```
++i; // префиксная
i++; // постфиксная
```

Точно так же для `--`.

4.8.1 Разница между префиксной и постфиксной формой

- **Префиксная** (`++i`): сначала увеличивает `i`, потом возвращает новое значение.
- **Постфиксная** (`i++`): возвращает старое значение, потом увеличивает `i`.

Пример:

Фрагмент

```
int i = 5;

int a = ++i; // i = 6, a = 6
int b = i++; // i = 7, b = 6
```

4.8.2 Когда какую использовать

Если результат оператора не нужен, обычно используют префикс: `++i`; . Для сложных типов она может быть чуть эффективнее. Постфикс полезен, когда нужно старое значение, например в цикле:

Фрагмент

```
for (int i = 0; i < n; i++) {
    // ...
}
```

4.8.3 Чего делать не стоит

Не модифицируйте одну и ту же переменную несколько раз в одном выражении без чётких скобок/порядка:

Фрагмент

```
int i = 0;
int x = i++ + ++i; // так нельзя, это UB
```

Операнды + могут вычисляться в любом порядке, поэтому побочные эффекты этих инкрементов не упорядочены. Правило выживания: не меняйте переменную больше одного раза в одном выражении.

4.9 Побитовые операторы (быстрое знакомство)

Работают с числами на уровне битов:

Фрагмент

```
& // побитовое И
| // побитовое ИЛИ
^ // XOR (исключающее ИЛИ)
~ // побитовое НЕ
<< // сдвиг влево
>> // сдвиг вправо
```

Пример:

Фрагмент

```
unsigned int x = 0b0011; // 3
unsigned int y = 0b0101; // 5

unsigned int a = x & y; // 0b0001 (1)
unsigned int b = x | y; // 0b0111 (7)
unsigned int c = x ^ y; // 0b0110 (6)
unsigned int d = ~x;    // инверсия
```

Сдвиги:

Фрагмент

```
unsigned int z = 1; // 0b0001
z = z << 3;         // 0b1000, z = 8
```

Ими удобно работать с флагами/масками, но пока важно просто не путать их с логическими `&&` и `||`.

Мини-правило про сдвиги (важно!):

1. если количество сдвига < 0 или $\geq$ разрядности типа — поведение не определено (UB);
2. для `unsigned` сдвиги описываются как операции над битами (с отбрасыванием/добавлением нулей);
3. для `signed` исторически было больше скользких мест: до C++20 правый сдвиг отрицательных был `implementation-defined`, а начиная с C++20 стандарт описывает его как деление на 2^k с округлением вниз (арифметический сдвиг).

4.10 Оператор запятая

Странный зверёк вида `expr1, expr2`.

- Сначала вычисляется `expr1`.
- Потом `expr2`.
- Значение всего выражения — результат `expr2`.

Пример:

Фрагмент

```
int a = 0;
int b = (a = 5, a + 10); // a = 5, b = 15
```

Часто встречается в заголовке `for`:

Фрагмент

```
for (int i = 0, j = 10; i < j; ++i, --j) {
    // ...
}
```

Важно не путать оператор запятая с запятой-разделителем в списках аргументов, инициализаторах и т. п.: там запятая лишь разделяет элементы и не образует новое выражение. Например, `f(a, b)` — это два аргумента, а `f((a, b))` — один аргумент со значением `b`. В обычном коде оператор запятая используют редко. Если без него понятнее — пишите без него.

4.11 Скобки и стиль: как не сойти с ума

Да, таблица приоритетов огромна, но нормальные люди не держат её всю в голове.

Золотые правила:

- Если не уверен — поставь скобки.
- Не пытайся писать «крутые» однострочники, которые никто не поймёт. Лучше 2–3 простых строки.
- Сложные логические условия группируй скобками:

Фрагмент

```
if ((age >= 18 && age <= 65) && hasJob) {
```

```
// ...  
}
```

- Не модифицируй одну переменную несколько раз в одном выражении.
- Не путай = и ==, && и &, || и |.

Скобки — не для слабых, а для тех, кто не любит баги.

Теперь выражения у нас под контролем. Пора управлять тем, когда они выполняются: ветвления, циклы и базовые приёмы управления потоком исполнения.

Часть 2

**Управление потоком и как всё может
пойти не так**

Выражения сами по себе — это «математика». А программа начинается там, где появляется выбор: `if`, `switch` и циклы. В этой части мы научимся управлять потоком выполнения *и* обсудим, почему некоторые ошибки не просто «красные подчёркивания», а реальные ловушки языка.

Глава 5 Условия и циклы

5.1 Что вообще такое «управление потоком»

C++-программа состоит из трёх больших типов конструкций:

- **declarations** — объявления (переменные, функции, типы);
- **expressions** — выражения ($a + b$, $x * 2$, вызов функции и т. п.);
- **control statements** — управляющие конструкции: `if`, `switch`, `for`, `while` и компания.

Нас сейчас интересуют именно **control statements** — конструкции, которые позволяют:

- выполнить код только при выполнении условия;
- повторять одни и те же действия много раз.

5.2 Булевы выражения и сравнения

5.2.1 Тип `bool`

Тип `bool` хранит всего два значения:

Фрагмент

```
bool ok = true;
bool fail = false;
```

Но любое число тоже можно неявно превратить в `bool`:

- $0 \rightarrow \text{false}$;
- любое ненулевое $\rightarrow \text{true}$.

И наоборот, `bool` можно неявно превращать в `int`:

- `false` $\rightarrow 0$;
- `true` $\rightarrow 1$.

Поэтому вот так — законно:

Фрагмент

```
int x = true;    // x = 1
if (42) {        // 42 -> true
    // сюда зайдём
}
```

5.2.2 Операторы сравнения

Основные операторы сравнения:

Фрагмент

```
==  равно
!=  не равно
<   меньше
>   больше
```

```
<=  меньше или равно
>=  больше или равно
```

Все они возвращают `bool`. Например:

Фрагмент

```
int a = 5;
int b = 7;

bool eq = (a == b); // false
bool lt = (a < b);  // true
bool ge = (a >= 5); // true
```

Сравнения — базовый кирпич для условий в `if`, `while` и т. п.

5.3 Логические операторы &&, ||, ! и ленивость

Чтобы собирать более сложные условия, есть логические операторы:

Фрагмент

```
&&  логическое И
||   логическое ИЛИ
!    логическое НЕ
```

5.3.1 Короткое замыкание (short-circuit)

Фишка `&&` и `||` — ленивость (короткое замыкание).

- Для `&&` (и): как только первое условие даёт `false`, остальные даже не вычисляются.
- Для `||` (или): как только находится первое `true`, остальные тоже не вычисляются.

Это важно и полезно:

Фрагмент

```
if (ptr != nullptr && ptr->value > 0) {
    // безопасно разыменовывать ptr
}
```

Если `ptr == nullptr`, вторая часть вообще не будет выполнена, и программа не упадёт.

5.3.2 Оператор !

Просто инвертирует `bool`:

Фрагмент

```
bool ok = true;
if (!ok) {
    // сюда не зайдём
}
```

5.4 Конструкция if / if-else

Синтаксис:

Фрагмент

```
if (условие) {
    // ветка "да"
} else {
    // ветка "нет"
}
```

Пример:

Фрагмент

```
int x;
std::cin >> x;

if (x % 2 == 0) {
    std::cout << "чётное\n";
} else {
    std::cout << "нечётное\n";
}
```

5.4.1 else if цепочки

Фрагмент

```
if (x < 0) {
    std::cout << "отрицательное\n";
} else if (x == 0) {
    std::cout << "ноль\n";
} else {
    std::cout << "положительное\n";
}
```

5.4.2 Область видимости и фигурные скобки

Формально можно писать без {} одну строку:

Фрагмент

```
if (x > 0)
    std::cout << "positive\n";
```

Но по код-стилю почти всегда ставим фигурные скобки, даже для одной строки, чтобы не сломать классический баг вида:

Фрагмент

```
if (x > 0)
```

```
std::cout << "positive\n";
std::cout << "still here\n"; // уже вне if!
```

5.5 switch и case

switch — когда есть одна переменная и много вариантов:

Фрагмент

```
int command;
std::cin >> command;

switch (command) {
    case 1:
        std::cout << "start\n";
        break;
    case 2:
        std::cout << "stop\n";
        break;
    default:
        std::cout << "unknown command\n";
        break;
}
```

Особенности:

- switch работает с целочисленными типами и перечислениями (enum, enum class).
- Без break исполнение «протекает» дальше в следующий case — иногда это делают специально, но чаще это баг. Если протекание сделано намеренно, в C++17+ обычно пишут `[[fallthrough]]`; , чтобы это было видно и компилятор не ругался.
- default выполняется, если никакой case не подошёл.

5.6 Тернарный оператор ?:

Тернарный оператор — компактный брат if-else в виде выражения:

Фрагмент

```
условие ? выражение1 : выражение2
```

Пример:

Фрагмент

```
int a = ...;
int b = ...;

int mx = (a > b) ? a : b; // максимум из двух чисел
```

Тип результата всего выражения выбирается по правилам приведения: если ветки разных типов, они приводятся к общему (например, `cond ? 1 : 2.5` даст `double`). Если обе ветки дают `lvalue` одного типа, результатом тоже будет `lvalue`, и в него можно присваивать.

- Сначала вычисляется условие.
- Если оно true — вычисляется только первое выражение.
- Если false — только второе. Обе ветки никогда не считаются одновременно.

Его удобно использовать, когда нужно именно значение, а не просто ветвление:

Фрагмент

```
int x = ...;
int abs_x = (x >= 0) ? x : -x;
```

Полезный пример — выбрать максимум/минимум из двух значений без отдельного if-else.

Важный момент. if — это statement, а тернарный оператор — expression. Поэтому if нельзя подставить туда, где ожидается выражение (например, внутри заголовка for), а тернарный можно.

5.7 Цикл while

Самый простой цикл:

Фрагмент

```
while (условие) {
    // тело цикла
}
```

Алгоритм:

1. Проверяется условие.
2. Если оно true — выполняется тело.
3. Возврат к пункту 1.

Пример: считывать числа, пока не встретили ноль:

Фрагмент

```
int x;
std::cin >> x;

while (x != 0) {
    std::cout << "x = " << x << '\n';
    std::cin >> x;
}
```

Любимый классический баг — забыли менять переменную в условии, получили вечный цикл.

5.8 Цикл do-while

Похож на while, но тело выполняется хотя бы один раз:

Фрагмент

```
do {
    // тело
```

```
} while (условие);
```

Пример «меню»:

Фрагмент

```
int cmd;
do {
    std::cout << "1 - play, 2 - settings, 0 - exit\n";
    std::cin >> cmd;

    if (cmd == 1) {
        // играть
    } else if (cmd == 2) {
        // настройки
    }
} while (cmd != 0);
```

5.9 Цикл for

Классический for:

Фрагмент

```
for (init; condition; step) {
    // тело
}
```

- `init` — выполняется один раз перед началом цикла;
- `condition` — проверяется перед каждой итерацией;
- `step` — выполняется после каждой итерации.

Пример:

Фрагмент

```
for (int i = 0; i < 10; ++i) {
    std::cout << i << '\n';
}
```

Логически это примерно то же самое, что:

Фрагмент

```
int i = 0;                // init
while (i < 10) {           // condition
    std::cout << i << '\n';
    ++i;                  // step
}
```

В заголовке `for` — три выражения: начальное, условие и шаг. После выполнения тела выполняется именно третье выражение.

5.9.1 Пре- и пост-инкремент в for

В заголовке цикла обычно пишут ++i или i++.

Для простых типов (int) всё равно — и то и то увеличивает i на 1. Но:

- ++i — пре-инкремент: увеличивает и возвращает новое значение;
- i++ — пост-инкремент: возвращает старое значение, а потом увеличивает.

Для простого счётчика разницы почти нет, но для итераторов контейнеров привычно и иногда выгоднее писать ++it.

5.10 Операторы break и continue

5.10.1 break

Прерывает ближайший цикл или switch:

Фрагмент

```
for (int i = 0; i < n; ++i) {
    if (a[i] == 0) {
        break; // выходим из for
    }
}
```

5.10.2 continue

Прерывает текущую итерацию, сразу переходит к проверке условия/шагу:

Фрагмент

```
for (int i = 0; i < n; ++i) {
    if (a[i] < 0) {
        continue; // пропускаем отрицательные
    }
    sum += a[i];
}
```

5.11 Бесконечные циклы

Иногда бесконечный цикл — нормальная ситуация:

Фрагмент

```
while (true) {
    // сервер, который что-то обрабатывает
}
```

Ещё стильный вариант:

Фрагмент

```
for (;;) {
    // тоже бесконечный цикл
}
```

Главное — следить, что внутри есть путь выхода (`break`, `return`, `throw`), иначе можно запустить вечный обогреватель CPU.

5.12 Range-based for (современный C++)

Современный удобный цикл по элементам **диапазона**: обычного массива или контейнера.

Фрагмент

```
int a[] = {1, 2, 3, 4};

for (int x : a) {
    std::cout << x << ' ';
}
```

Такой `for` работает и с массивами (компилятор знает их границы), и с контейнерами стандартной библиотеки (для них используются `begin()` и `end()`). С контейнерами вроде `std::string` и `std::vector` мы познакомимся в следующих главах.

Часто используют `auto`:

Фрагмент

```
for (auto x : a) {           // копия элемента
    ...
}

for (auto& x : a) {          // ссылка на элемент, можно менять
    x *= 2;
}
```

Этот цикл разворачивается в обычный `for` с вызовами `begin()` и `end()`, и вы сами будете реализовывать подобные вещи для своих контейнеров.

5.13 Частые ошибки в условиях и циклах

5.13.1 Присваивание вместо сравнения

Фрагмент

```
if (x = 0) {    // ОШИБКА: здесь присваивание, не сравнение
    ...
}
```

Такое условие почти всегда `false` (если 0) или `true` (если другое значение). Пишите `==`.

Совет: в сложных условиях ставьте скобки и не стесняйтесь разбивать на переменные:

Фрагмент

```
bool is_zero = (x == 0);  
if (is_zero) { ... }
```

5.13.2 Переполнение счётчика и бесконечный цикл

Пример: в цикле счётчик умножают на большое число, он переполняется (`int` выходит за границы), и программа переходит в состояние **undefined behavior** — после чего может зациклиться навсегда.

Суть:

- переполнение знакового `int` — это UB в C++;
- значит, после этого компилятор ничего не гарантирует — в том числе поведение цикла.

Мы ещё подробно поговорим об этом в следующей главе про ошибки и неопределённости.

5.13.3 Непонимание разницы между `statement` и `expression`

Пример: попытались засунуть `if` внутрь заголовка `for` туда, где ожидается выражение — и получили ошибку компиляции «ожидалось выражение».

Важно держать в голове простую границу: `if`, `switch` и циклы — это `statements` (они управляют потоком выполнения), а `?:` — `expression`, поэтому его можно использовать там, где нужно именно значение.

На этом Глава 4 «Условия и циклы» заканчивается.

Как только появляются ветвления и циклы, сразу становится заметно: ошибки бывают не только синтаксические. Дальше разберёмся, что такое UB, переполнения, выходы за границы и почему иногда программа “работает” ровно до первой оптимизации.

Глава 6 Ошибки и неопределённости

В прошлой главе мы научились управлять потоком выполнения: `if`, `for`, `while` и всё такое. Теперь поговорим о том, как всё это может пойти не так.

Эта глава знакомит тебя с тем, какие бывают ошибки и когда мы их обнаруживаем, что такое *undefined behavior* и другие «серые зоны» стандарта, почему иногда программа «работает», но по правилам языка уже в аду, и какие привычки помогают ловить такие вещи раньше.

6.1 Какие вообще бывают ошибки

Слово «ошибка» в программировании используют для разных вещей. Иногда код даже не собирается — компилятор сразу выдаёт диагностическое сообщение, и это ошибка компиляции. Бывает, что каждый файл по отдельности скомпилировался, но линковщик не смог собрать их в один исполнимый файл: например, вы объявили функцию и забыли её определить, или определили одну и ту же сущность в двух единицах трансляции. Это ошибки линковки.

Если программа собралась и запускается, но падает или ведёт себя странно во время работы (деление на ноль, выход за границы массива, сегфолт), это ошибки времени выполнения. А когда всё компилируется и не падает, но ответ неправильный из-за ошибки в логике или алгоритме, мы говорим о логическом баге.

Плюс отдельно стоит мир, о котором обычно говорят с нервным смехом: **undefined / unspecified / implementation-defined behavior**. Это «поведение, которое может быть чем угодно».

6.2 Ошибки по времени обнаружения

6.2.1 Ошибки компиляции

Самые приятные (да-да, именно они).

Примеры:

Фрагмент

```
int x = "hello";      // типы не совпадают
int y = z + 5;        // z не объявлена
if (x = 5) { ... }    // тут компилятор может (и часто будет) ругаться
```

Компилятор видит, что правила языка нарушены, и честно об этом сообщает. Такие программы называются **ill-formed** (неправильно сформированные).

Хорошая новость: вы их даже не запустите — значит, баг пойман раньше.

6.2.2 Ошибки линковки

Все файлы по отдельности компилируются, а вот связать их воедино не получается:

Фрагмент

```
// a.cpp
```

```
void foo();

int main() {
    foo();
}

// b.cpp
// забыл определить foo()
```

Компиляция a.cpp и b.cpp пройдёт, но на этапе линковки: вы получите сообщение вроде `unresolved external symbol (MSVC)` или `undefined reference to ... (GCC/Clang)`: линковщик не нашёл определения `foo`.

Или наоборот:

Фрагмент

```
// a.cpp
int value;

// b.cpp
int value; // в двух единицах трансляции одинаковое определение
```

Будет `multiple definition`. **Почему так:** это нарушение ODR (One Definition Rule) — на программу должна быть ровно одна дефиниция не-`inline` сущности, которая реально используется. Если нарушить, линковщик закономерно ругается.

6.2.3 Ошибки времени выполнения

На практике программа часто падает из-за переполнения стека при глубокой рекурсии, не пойманных исключений и т. п. А вот деление на ноль для **целых** типов, выход за границы массива и разыменование `nullptr` — это **undefined behavior**: иногда это выглядит как `segmentation fault/access violation`, иногда «работает», а иногда ломает данные тихо и проявляется позже.

6.2.4 Логические ошибки

Самые подлые.

Код компилируется, программа не падает, но делает не то — перепутали формулу, неверно реализовали алгоритм или забыли включить одно из условий.

Пример:

Фрагмент

```
int sum = 5;
int n = 2;

double avg = sum / n; // 2.0, а ожидали 2.5
double avg_ok = 1.0 * sum / n; // 2.5
```

Чтобы получить дробный результат, хотя бы один операнд должен быть вещественным.

Формально ошибок нет, всё строго по стандарту, но результат не тот.

6.3 Поведение программы по стандарту C++

Стандарт C++ вводит несколько «режимов» поведения программы:

- **well-defined behavior** — всё хорошо, результат однозначен;
- **implementation-defined behavior** — компилятор выбирает один из вариантов и обязан задокументировать;
- **unspecified behavior** — стандарт перечислил несколько допустимых вариантов, но какой из них будет, не фиксирует;
- **undefined behavior (UB)** — стандарт вообще ничего не гарантирует.

6.3.1 Терминологическая таблица: UB / unspecified / implementation-defined

Термин	Что гарантирует стандарт	Как с этим жить (в учебном коде)
implementation-defined (ID)	Компилятор/платформа выбирают один вариант и обязаны задокументировать , какой именно. На одной реализации поведение, как правило, стабильно (при фиксированных флагах/ABI).	Старайся не строить логику на таких деталях, если нужна переносимость. Если всё же опираешься — явно зафиксируй предпосылки (компилятор/архитектура/флаги), добавь <code>static_assert</code> и комментариев.
unspecified	Стандарт разрешает несколько вариантов, но не обещает , какой выберется (и может выбирать по-разному от запуска к запуску, от оптимизации к оптимизации).	Пиши код так, чтобы он был корректен при любом допустимом варианте. Практический способ: избегай «тонких» побочных эффектов в одном выражении, явно задавай порядок вычислений (разбей на несколько строк).
UB (undefined)	Никаких гарантий: программа нарушила правила стандарта, и дальше компилятор волен делать что угодно (включая агрессивные оптимизации, исходя из предположения «UB не бывает»).	Избегать как класса. Если видишь риск UB — перепиши код. Для поиска используй предупреждения компилятора, санитайзеры (ASan/UBSan) и минимальные воспроизводимые примеры.

Важный нюанс: Пример «unspecified» обычно связан с **порядком вычислений**. В C++17+ часть правил стала строже, но всё равно встречаются места, где порядок не

зафиксирован. Поэтому безопасный навык: **не полагаться** на порядок вычислений, если он явно не задан стандартом.

И отдельно:

- **ill-formed** — программа нарушает правило, которое обязан диагностировать компилятор (ошибка компиляции);
- **ill-formed, no diagnostic required** — нарушено правило, но компилятор может не ругаться (самый любимый ужас).

6.4 Undefined behavior (UB) — тёмная магия

Вот здесь начинается самое весёлое.

Undefined behavior — это ситуация, когда программа делает что-то, что стандарт запрещает, и дальше язык снимает с себя все обязательства.

Может случиться всё что угодно:

- программа «как бы работает»;
- программа падает;
- программа печатает разные ответы от запуска к запуску;
- компилятор выкидывает куски кода при оптимизации, потому что «такое никогда не бывает».

Важно: UB — не «плохо определено», а вообще никак не определено.

6.4.1 Почему UB вообще существует?

Звучит дико: язык, который официально разрешает демону вылетать из носа. Но у этого есть причины:

- высокая производительность;
- возможность эффективно использовать железо;
- совместимость с C и старым кодом;
- свобода для оптимизатора (компилятор может делать очень агрессивные преобразования, предполагая, что UB нет).

Цена: ответственность за отсутствие UB — на программисте.

6.4.2 Классические примеры UB

1. Переполнение знакового целого

Фрагмент

```
int x = 2'000'000'000;
int y = x * 2;    // потенциальное переполнение
```

Если результат не помещается в `int`, это UB. Иногда вы увидите «циклический» результат, иногда — что-то ещё; полагаться нельзя.

2. Использование неинициализированной переменной

Фрагмент

```
int x;           // не инициализирована
int y = x + 1;  // UB
```

Не «мусорное значение», а именно undefined behavior. Компилятор может вообще убрать этот код.

3. Выход за границы массива

Фрагмент

```
int a[10];
a[10] = 42;  // индексация от 0 до 9, 10 -- за границами, UB
```

Даже если «оно работает у меня», это никакой не аргумент.

4. Висячие указатели и ссылки (dangling)

Фрагмент

```
int* foo() {
    int x = 10;
    return &x;  // возвращаем адрес локальной переменной
}

int main() {
    int* p = foo(); // p указывает в никуда
    std::cout << *p << '\n'; // UB
}
```

Переменная x умерла при выходе из foo, указатель — висячий.

5. Использование объекта после delete

Фрагмент

```
int* p = new int(5);
delete p;
std::cout << *p;  // UB
```

6.4.3 Как ловить UB и ошибки памяти санитайзерами (семинары)

Большая часть «болевых» ошибок первого семестра связана с памятью: выходы за границы, use-after-free, double free, утечки. Хорошая новость: **санитайзеры** (ASan/UBSan) умеют ловить многие такие вещи в момент, когда они происходят.

Фрагмент

```
// 1) double free
int* p = new int(1);
delete p;
delete p; // UB: ASan почти наверняка поймает

// 2) use-after-free
int* q = new int(2);
```



```
delete q;
*q = 5;    // UB: use-after-free

// 3) heap-buffer-overflow (выход за границы в куче)
int* a = new int[3]{1,2,3};
a[3] = 10; // UB: пишем за пределы массива
delete[] a;
```

Частая ловушка: Санитайзер — это не «панацея» и не доказательство корректности. Он ловит много, но не всё. Однако для учебных проектов это один из лучших способов быстро находить причины крашей и странного поведения. Команды для сборки с ASan/UBSan см. в главе 1.

6.4.4 Несколько модификаций за раз

Классика из старых стандартов:

Фрагмент

```
int i = 0;
i = i++;    // до C++17 было UB
```

Или:

Фрагмент

```
int a[10]{};
int i = 0;
a[i] = i++; // до C++17 было UB
```

До C++17 стандарт действительно не задавал порядок между такими чтениями и модификациями, поэтому результат считался undefined behavior. Начиная с C++17 для присваивания порядок вычисления ужесточили: правый операнд вычисляется раньше левого, так что эти выражения теперь имеют определённое поведение (хотя читаемость у них всё равно плохая).

Правило: не изменяйте переменную более одного раза в одном выражении, если при этом читаете её старое или новое значение.

6.4.5 UB и оптимизатор

Фрагмент

```
int f(int x) {
    if (x + 1 < x) {
        // ...
    }
}
```

Для знакового int выражение $x + 1 < x$ невозможно без переполнения. А переполнение — UB, значит компилятор может считать, что оно никогда не происходит. Следовательно, условие всегда ложно, и он может просто выкинуть ветку (и само условие).

То есть «но у меня же когда-то было true!» — аргумент не для стандарта, а для кофеев.

6.5 Как живёт стек при вызове функции

Мы уже несколько раз говорили слова «стек», «локальная переменная живёт до конца блока», «висячий указатель на уничтоженный объект». Давай теперь аккуратно представим, что именно происходит со стеком при вызове функции.

6.5.1 Что такое стек в двух словах

Можно думать так:

- **стек** — это область памяти, в которой хранятся локальные переменные функций и адрес возврата (куда прыгнуть после `return`);
- каждый вызов функции создаёт **кадр стека** (stack frame);
- внутри кадра живут локальные переменные и служебная информация.

Когда функция заканчивает работу, её кадр стека просто выбрасывается, как будто мы откатываемся назад.

6.5.2 Один вызов — один кадр

Полный пример

```
void foo() {
    int x = 10;
    (void)x;
    // ...
} // здесь x перестаёт существовать

int main() {
    foo();
}
```

Стек можно представить так:

1. запускается `main` — создаётся кадр `main`, в нём будут лежать локальные переменные `main`;
2. `main` вызывает `foo` — поверх стека кладётся новый кадр `foo`: туда попадают параметры `foo` (если есть) и его локальные переменные, такие как `x`;
3. внутри `foo` мы что-то делаем с `x`;
4. при выходе из `foo` его кадр стека удаляется целиком, вместе со всеми локальными переменными;
5. управление возвращается в `main`, стек выглядит так, как будто `foo` никогда и не существовал.

Важно: память под `x` освобождена, а сам `x` перестаёт существовать.

6.5.3 Рекурсия и несколько кадров одной и той же функции

Если функция вызывает саму себя (рекурсия), в стеке может быть несколько кадров **одной и той же функции**.

Фрагмент

```
int fact(int n) {
    if (n <= 1) return 1;
    int sub = fact(n - 1);
    return n * sub;
}
```

При вызове `fact(3)` стек будет примерно таким (сверху вниз):

- `fact(1)` — кадр с `n = 1`;
- `fact(2)` — кадр с `n = 2`, `sub` ещё не вычислен;
- `fact(3)` — кадр с `n = 3`, `sub` ещё не вычислен;
- `main` — кадр с локалками `main`.

По мере возврата:

- `fact(1)` возвращает 1, его кадр выбрасывается;
- продолжаем `fact(2)`, вычисляем `sub = 1`, возвращаем 2, кадр `fact(2)` выбрасывается;
- продолжаем `fact(3)`, вычисляем `sub = 2`, возвращаем 6, кадр `fact(3)` выбрасывается.

В конце остаётся только кадр `main`.

6.5.4 Откуда берутся висячие указатели

Теперь видно, почему такие вещи опасны:

Полный пример

```
int* get_ptr() {
    int x = 42;
    return &x;    // возвращаем адрес локальной переменной
}

int main() {
    int* p = get_ptr(); // p хранит адрес x, но x уже не существует!
    *p = 10;           // UB
}
```

Что происходит по шагам:

1. входим в `get_ptr`, создаётся кадр `get_ptr`, в нём живёт `x`;
2. `return &x`; возвращает адрес памяти внутри этого кадра стека;
3. `get_ptr` заканчивается → её кадр выбрасывается → память «под `x`» считается свободной;
4. в `main` переменная `p` всё ещё содержит тот старый адрес;
5. попытка `*p = 10`; — разыменование указателя, который указывает «в никуда». Итог — `undefined behavior`.

Такой указатель называется **висячим** (*dangling*): он указывает на память, где когда-то что-то жило, но уже умерло.

6.5.5 Время жизни локальных переменных

Можно сформулировать правило:

Локальная переменная живёт от начала своего объявления до конца блока, в котором объявлена.

Блок — это всё между { и }:

Фрагмент

```
{
    int x = 10; // x жив
    // ...
}                // x уничтожается здесь
```

И это напрямую связано со стеком:

- вход в блок/функцию — на стек добавляются новые переменные;
- выход из блока/функции — эта часть стека срезается, переменные уничтожаются.

6.5.6 Как это помогает понимать ошибки

Теперь легко связать:

- висячие указатели/ссылки — это когда мы храним адрес или ссылку на переменную из кадра стека, который уже уничтожен;
- выход за границы массива на стеке — это запись в память чужого кадра или вообще за пределы стека;
- рекурсивная функция без базового случая создаёт новые кадры стека до тех пор, пока стек не переполнится (stack overflow).

Когда мы говорим «эта переменная уже умерла», это не метафора — её кадр стека действительно выброшен, и её памяти «как бы больше нет».

6.6 Unspecified behavior

Unspecified behavior — стандарт говорит: программа должна сделать что-то из списка вариантов, но какой именно — не важен.

Компилятор не обязан документировать, что именно он выбирает, и может менять выбор от случая к случаю.

Классический пример (в духе до C++17): порядок вычисления аргументов функции:

Фрагмент

```
int g();
int h();

f(g(), h());
```

Стандарт говорит: будут вызваны и `g()`, и `h()`, каждый ровно один раз. Но в каком порядке — не фиксирует. Может быть `g→h`, может быть `h→g`.

Если вам от этого порядка что-то зависит, у вас уже логическая ошибка.

6.7 Implementation-defined behavior

Implementation-defined behavior — компилятор выбирает один из вариантов и обязан его задокументировать.

Примеры:

- размер `int` (на практике 16, 32 или 64 бита — как повезёт, но чаще всего 32);
- подписанность `char` (`signed` или `unsigned`);
- представление `bool` (обычно 0 и 1, но стандарт не навязывает конкретный формат в памяти);
- (до C++20) поведение `>>` для отрицательных `signed` было `implementation-defined`; начиная с C++20 семантика правого сдвига для `signed` описана стандартом (арифметический сдвиг / деление на 2^k с округлением вниз).

Почему так сделано?

- поддержка разных архитектур;
- возможность использовать «родной» формат железа.

Если вы пишете по-настоящему переносимый код, нельзя делать предположений вроде «`int` точно 32 бита». Используйте типы вроде `std::int32_t`, если вам важно именно 32 бита.

6.8 Почему вся эта неопределённость — это и плюс, и минус

Минус для программиста:

- надо помнить о куче тонкостей;
- нельзя уткнуться в конкретное поведение одного компилятора и считать его «законом».

Плюс для языка и компиляторов:

- можно оптимизировать очень агрессивно;
- можно поддерживать разные процессоры и ОС;
- можно позволить низкоуровневые трюки (системное программирование, драйверы, ядро и т.п.).

Главное правило: не опирайтесь на UB или `unspecified behavior`. Даже если «на моём компе всё ок».

6.9 Стратегии борьбы с ошибками

6.9.1 Пусть компилятор работает за вас

Включайте максимально строгие предупреждения:

- для `g++/clang++`: `-Wall -Wextra -Wpedantic` (и нередко ещё `-Werror`).

Не игнорируйте предупреждения — часто это либо UB, либо логическая ошибка.

Фрагмент

```
g++ -std=c++20 -Wall -Wextra -pedantic main.cpp
```

Предупреждения — бесплатный статический анализ.

6.9.2 assert и проверка инвариантов

Есть макрос `assert` из `<cassert>`:

Фрагмент

```
#include <cassert>

int divide(int a, int b) {
    assert(b != 0); // наше внутреннее предположение
    return a / b;
}
```

- Если условие в `assert` ложно, программа упадёт с сообщением.
 - В release-сборках часто определяют `NDEBUG`, и тогда все `assert` просто выкидываются.
- `assert` хорош для:
- внутренних инвариантов;
 - вещей, которые «никогда не должны случаться», если программа написана правильно.

Но он не для обработки пользовательского ввода. Если пользователь ввёл ноль, это обычная ситуация, а не баг программы:

Фрагмент

```
int safe_divide(int a, int b) {
    if (b == 0) {
        // обработать ошибку: вернуть код, бросить исключение и т.д.
    }
    ...
}
```

6.9.3 Код возврата и bool-результаты

Самый простой способ сообщить об ошибке — вернуть флаг или код:

Фрагмент

```
bool read_int(std::istream& in, int& value) {
    if (!(in >> value)) {
        return false; // данные некорректные
    }
    return true;
}
```

Использование:

Фрагмент

```
int x;
if (!read_int(std::cin, x)) {
    std::cout << "Ошибка ввода!\n";
    return 1;
}
```

Это уже аккуратнее, чем просто `std::cin >> x;` и надежда на лучшее.

6.9.4 Немного про исключения (только тизер)

В C++ есть полноценный механизм:

- `throw` — «бросить» исключение;
- `try` — блок, внутри которого что-то может полететь;
- `catch` — блоки обработки.

Пример идеи:

Фрагмент

```
#include <stdexcept>

int safe_divide(int a, int b) {
    if (b == 0) {
        throw std::runtime_error("division by zero");
    }
    return a / b;
}
```

Подробно про исключения обычно рассказывают позже, когда уже есть классы и RAII. Сейчас главное: исключения — это тоже способ обработки ошибок, но более мощный, чем коды возврата.

6.10 Где ошибка программы, а где просто «плохие данные»

Важное разделение:

1. **Ошибка в коде** (bug, UB, неверный алгоритм) — надо чинить программу.
2. **Неправильный ввод / внешние условия:**
 - пользователь ввёл "abc" вместо числа;
 - файл не открылся;
 - сервер недоступен.

Это не баг в коде (если, конечно, это предусмотрено), а ситуация, которую программа должна аккуратно обработать:

Фрагмент

```
int x;
if (!(std::cin >> x)) {
    std::cout << "Вы ввели не число\n";
}
```

Программа не должна падать от того, что пользователь нажал не туда.

6.11 Типичные ошибки новичка (и где тут неопределённость)

Небольшое замечание по примерам: дальше иногда встретятся `std::string` и `std::vector` из стандартной библиотеки. Пока достаточно знать, что `std::string` — это *строка*, а `std::vector`

— *динамический массив* (контейнер с индексацией). Подробно мы разберём их в следующей главе.

Список «популярных хитов»:

1. Неинициализированные переменные

Фрагмент

```
int x;
if (x > 0) { ... } // UB
```

Всегда инициализируйте локальные переменные.

2. Возврат ссылки или указателя на локальную переменную

Фрагмент

```
const int& foo() {
    int x = 42;
    return x;    // висячая ссылка, UB
}
```

То же самое случится с любым типом (включая `std::string`): объект уничтожается при выходе из функции.

3. Смешивание `signed` и `unsigned`

Фрагмент

```
int i = -1;
unsigned int n = 10;

if (i < n) { ... } // сюрприз: i приводится к unsigned
```

`i` станет огромным положительным числом, условие будет ложным. Это не UB, но очень частая логическая ошибка. А вот выход за границы массива потом уже будет UB.

4. Отрицательный индекс

Фрагмент

```
int a[10]{};
int i = -1;
a[i] = 5;    // UB: обращение за границы (индекс < 0)
```

То же относится и к `std::vector`: его `operator[]` не проверяет границы, а отрицательный индекс после неявных преобразований превращается в огромный положительный.

5. Переполнение счётчика цикла

Из предыдущей главы: если в цикле счётчик растёт, пока меньше `n`, но при этом может переполниться, то в какой-то момент:

- значение перескакивает через много чисел;
- условие никогда не становится ложным;
- компилятор, зная, что переполнения быть не должно, может вообще переупорядочить ваш код.

6. Разыменование `nullptr`

Фрагмент

```
int* p = nullptr;
*p = 10;    // UB
```

6.12 Практические правила, которые реально спасают от ошибок

Несколько коротких правил из практики. Они не гарантируют идеальный код, но заметно снижают шанс UB и логических багов.

- **Инициализируйте переменные при объявлении.**

Фрагмент

```
int x = 0;
```

- **Не прошивайте размеры магическими числами.** Вместо

Фрагмент

```
for (int i = 0; i < 10; ++i)
```

лучше привязываться к реальному размеру:

Фрагмент

```
for (int i = 0; i < n; ++i)
```

- **Не изменяйте переменную несколько раз в одном выражении.**

Фрагмент

```
// плохо
int a[10]{};
int i = 0;
a[i] = ++i;

// хорошо
++i;
a[i] = ...;
```

- **Включайте предупреждения компилятора и исправляйте их.**
- **Проверяйте входные данные.**

Фрагмент

```
if (!(std::cin >> x)) {
    // сообщить об ошибке
}
```

- **Используйте assert для своих инвариантов.**

На этом Глава 5 «Ошибки и неопределённости» заканчивается.

Теперь мы знаем, как всё может сломаться. Следующий логичный шаг — научиться хранить и обрабатывать наборы данных. Начнём с массивов и циклов по ним, а потом перейдём к более удобным контейнерам.

Глава 7 Исключения

В прошлой главе мы классифицировали ошибки и познакомились с UB. Теперь добавим ещё один важный инструмент: **исключения** (*exceptions*). Они позволяют сигнализировать об ошибке, не протаскивая коды возврата через каждый вызов, и автоматически раскручивают стек, вызывая деструкторы.

7.1 Зачем нужны исключения

У нас есть два частых подхода к обработке ошибок:

- **Коды возврата** (или специальные типы «успех/ошибка», например `std::optional`).
- **Исключения**: функция *бросает* ошибку (`throw`), а обработка происходит там, где это логично (`try/catch`).

Важный нюанс: В курсе мы используем исключения как стандартный механизм для случаев вроде `v.at(i)` или `dynamic_cast<Derived&>`: ошибки редкие, но критичные, и удобнее обработать их на более верхнем уровне.

7.2 Минимальный синтаксис

7.2.1 `throw, try, catch`

Полный пример

```
#include <iostream>
#include <stdexcept>

int safe_div(int a, int b) {
    if (b == 0) {
        throw std::runtime_error("division by zero");
    }
    return a / b;
}

int main() {
    try {
        std::cout << safe_div(10, 0) << "\n";
    } catch (const std::runtime_error& e) {
        std::cout << "error: " << e.what() << "\n";
    }
}
```

Правила «по умолчанию», которые почти всегда правильные:

- ловим исключения как `const T&` (не по значению) — так мы избегаем копирования и не теряем полиморфизм;

- ловим более конкретные типы раньше, а `catch (const std::exception&)` — ближе к концу;
- `catch (...)` — самый последний вариант («поймать всё»).

7.2.2 Повторный бросок (rethrow)

Иногда мы хотим «подлогировать» ошибку и пробросить дальше:

Фрагмент

```
try {
    // ...
} catch (const std::exception& e) {
    std::cerr << "failed: " << e.what() << "\n";
    throw; // пробрасываем то же самое исключение дальше
}
```

7.3 Стандартные исключения, которые встречаются в курсе

- `std::out_of_range` — например, `std::vector::at` и `std::string::at`;
- `std::bad_cast` — если `dynamic_cast<Derived&>` невозможен;
- `std::bad_typeid` — если сделать `typeid(*p)` при `p == nullptr` для полиморфного типа;
- базовый класс `std::exception` почти для всех стандартных исключений.

7.4 Раскрутка стека и RAII

Когда исключение «летит вверх», C++ гарантирует **раскрутку стека**: все уже созданные локальные объекты будут корректно разрушены.

Полный пример

```
#include <iostream>
#include <stdexcept>

struct Guard {
    Guard() { std::cout << "acquire\n"; }
    ~Guard() { std::cout << "release\n"; }
};

void f() {
    Guard g;
    throw std::runtime_error("boom");
}

int main() {
    try {
        f();
    } catch (...) {
```

```

        std::cout << "caught\n";
    }
}

```

Вывод будет таким:

Фрагмент

```

acquire
release
caught

```

Важный нюанс: Это и есть одна из причин, почему в C++ так важен подход **RAII**: ресурс (память, файл, мьютекс) привязан к объекту, а объект гарантированно разрушится даже при исключении.

7.5 noexcept и terminate

Если функция помечена noexcept и внутри всё же произошло исключение, стандарт требует вызвать std::terminate (обычно программа аварийно завершится).

Фрагмент

```

#include <stdexcept>

void g() noexcept {
    throw std::runtime_error("oops"); // приведёт к std::terminate
}

```

Частая ловушка: Поэтому конструкторы перемещения (move ctor) часто помечают noexcept: это даёт контейнерам право перемещать элементы при реаллокации. Но если вы поставили noexcept, то должны быть уверены, что реально не бросаете.

7.6 Связь с предыдущими главами

В прошлой главе мы говорили про CE/RE/UB и про то, что не все ошибки удобно тащить через коды возврата. Исключения дают ещё один «канал» передачи ошибки: throw прерывает обычное выполнение, а обработка происходит там, где это действительно логично.

Дальше по книге мы будем встречать исключения в стандартной библиотеке: например, std::string::at и std::vector::at при выходе за границы бросают std::out_of_range. А в главе про RTTI и dynamic_cast появятся std::bad_cast и typeid, которые тоже связаны с исключениями и диагностикой типов.

7.7 Что важно помнить

Исключения — это способ сигнализировать об ошибке через `throw` и обработать её через `try/catch`, не перегружая каждую функцию «кодами ошибок». При исключении стек корректно «раскручивается», деструкторы вызываются — поэтому RAII работает надёжно. Почти всегда исключения ловят по `const&` и начинают с конкретных типов. А поехсерт стоит писать осознанно: если из поехсерт-функции всё же вылетит исключение, программа завершится через `std::terminate`.

Теперь можно переходить к данным посложнее: массивам, строкам и вектору. Там будет много индексов, границ и ситуаций, где стандартная библиотека предпочитает «не молчать», а бросить исключение — и теперь мы понимаем, что именно это означает.

Часть 3

**Данные посложнее: массивы, строки,
векторы, указатели**

Здесь мы переходим от одиночных значений к наборам и более «живым» данным: массивам, строкам и вектору. Главное чувство, которое должно появиться по итогу: *размер и границы — это часть данных*. А дальше мы аккуратно подведём к указателям и ссылкам, потому что именно они объясняют, что происходит «под капотом» у массивов и контейнеров.

Глава 8 Массивы и циклы по массивам

В прошлых главах мы уже умели хранить отдельные значения в переменных и гонять их по условным операторам и циклам. Но жизнь программиста жестока: очень быстро выясняется, что одного `int x`; недостаточно — нужно 10, 100, 1000 чисел подряд.

Хранить `x1`, `x2`, `x3`, ... в отдельных переменных — путь страданий. Для таких задач в C++ и придумали массивы.

В этой главе мы разберёмся, что такое массивы в C++, как их объявлять и инициализировать разными способами, и как аккуратно пробегать по ним циклами `for` и `while`.

Заодно заглянем под капот и посмотрим, как массив связан с указателями (без фанатизма — полноценные указатели будут в главе 10), и разберём несколько типичных ошибок, чтобы ты их не повторял.

8.1 Зачем нужны массивы

Типичная ситуация:

Фрагмент

```
int a0, a1, a2, a3, a4, a5, a6, a7, a8, a9;  
// хотим найти максимум из этих десяти
```

Считать максимум вручную по десяти переменным — уже неприятно. А если их тысячи?

Массив — это набор значений одного типа, лежащих подряд в памяти, к которым можно обращаться по индексу.

В C++ есть классический сишный синтаксис:

Фрагмент

```
тип имя[размер];
```

Примеры:

Фрагмент

```
int    a[10];      // массив из 10 int  
double b[5];       // 5 даблов  
char   letters[4]; // 4 символа
```

Размер — **компиляторно известное целое** (обычно константа). То есть `int n`; `int a[n]`; в стандарте C++ нельзя (это расширение некоторых компиляторов). Если размер нужно узнать только в рантайме — это уже задача для `std::vector` (динамического массива), о нём в следующей главе.

8.2 Объявление и инициализация массивов

Важный нюанс: Про размер массива. У «сырого» массива `int a[10]`; размер известен компилятору, поэтому снаружи функции можно писать `sizeof(a) / sizeof(a[0])`. Начиная с C++17 есть ещё удобнее: `std::size(a)` (нужен `#include <iterator>`).

А если хочется «массив фиксированного размера, но по-современному» — присмотритесь к `std::array<T, N>` (нужен `#include <array>`): он хранит элементы подряд, знает свой размер и **не** превращается сам по себе в указатель.

Самое базовое:

Фрагмент

```
int a[10]; // 10 int'ов, НО значения не определены!
```

Локальный массив, объявленный без инициализации, содержит мусор. Не «нули по умолчанию», а именно что угодно.

8.2.1 Инициализация всеми нулями

Удобный трюк из C/C++:

Фрагмент

```
int a[10]{}; // все 10 элементов = 0
int b[10] = {}; // то же самое
```

Компилятор просто заполняет весь массив нулями.

8.2.2 Частичная инициализация

Можно задать первые несколько значений, а остальные автоматически станут нулями:

Фрагмент

```
int a[10] = {1, 2, 3, 4};
// дальше фактически: {1, 2, 3, 4, 0, 0, 0, 0, 0, 0}
```

Это очень удобно, когда массив «почти нулевой, но с парой осмысленных значений в начале».

8.3 Индексация: от 0 до N-1 и почему a[10] — это зло

К массиву обращаемся через оператор `[]`:

Фрагмент

```
int a[10]{};
a[0] = 42; // первый элемент
a[1] = 7;
int x = a[0] + a[1];
```

Важно: индексация начинается с 0.

Если размер массива N, то допустимые индексы: 0, 1, 2, ..., N - 1. Для `int a[10];`:

- минимальный индекс: 0;
- максимальный индекс: 9.

Запись:

Фрагмент

```
a[10] = 5; // UB --- undefined behavior
```

— обращение за границы массива. Это undefined behavior: программа может «как будто сработать», может испортить чужие данные в памяти, может упасть или начать вести себя странно ещё через полчаса.

Компилятор иногда умеет это ловить, если индекс константный. Например, в лекциях показывали массив на 10 элементов и обращение по индексу 10 — компилятор честно выдавал ворнинг.

Правило номер один:

Последний элемент массива размера N — это $a[N - 1]$, а обращение к $a[N]$ — всегда ошибка.

8.4 Размер массива и sizeof

Иногда нужно узнать длину массива. Но в C++ голый массив — это не объект с методом `.size()`, как `std::vector`. Нам нужна немного магии.

Если массив объявлен в этой же области видимости как `T a[N];`, то:

Фрагмент

```
std::size_t size = sizeof(a) / sizeof(a[0]);
```

- `sizeof(a)` — размер всего массива в байтах;
- `sizeof(a[0])` — размер одного элемента;
- их отношение = количество элементов.

Пример:

Полный пример

```
#include <cstdint> // std::size_t
#include <iostream>

int main() {
    int a[10]{};
    std::size_t n = sizeof(a) / sizeof(a[0]);

    std::cout << "n = " << n << "\n"; // выведет 10
}
```

Важный момент: это работает только пока `a` — именно массив, а не указатель. При передаче массива в функцию он «разваливается» до указателя (подробнее — в главе 10).

8.5 Цикл for по массиву через индексы

Классический способ пройти по массиву:

Фрагмент

```
int a[5] = {10, 20, 30, 40, 50};

for (int i = 0; i < 5; ++i) {
    std::cout << "a[" << i << "] = " << a[i] << "\n";
}
```

Чтобы не забыть размер массива, обычно выносят его в константу:

Фрагмент

```
const int N = 5;
int a[N] = {1, 2, 3, 4, 5};

for (int i = 0; i < N; ++i) {
    // ...
}
```

Так меньше шансов сделать промах «на один» и получить `a[N]`.

8.6 Циклы while и do while по массиву

Тот же обход можно сделать через while:

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};
const int N = 5;

int i = 0;
while (i < N) {
    std::cout << a[i] << " ";
    ++i;
}
```

Или через do while, если нам гарантированно нужно зайти в цикл хотя бы один раз:

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};
const int N = 5;

int i = 0;
do {
    std::cout << a[i] << " ";
    ++i;
} while (i < N);
```

На практике для массивов в 99% случаев используют for — в нём удобно держать в голове и индекс, и условие, и шаг.

8.7 Массивы разных типов и многомерные массивы

Массив может быть практически любого типа, который ты уже знаешь.

8.7.1 Массив `double`

Фрагмент

```
double temps[4] = {36.6, 37.2, 36.9, 38.0};

for (int i = 0; i < 4; ++i) {
    std::cout << temps[i] << "\n";
}
```

8.7.2 Массив `char`

Фрагмент

```
char hello[6] = {'h', 'e', 'l', 'l', 'o', '\0'};
```

Это уже почти C-строка — именно поэтому последний элемент `'\0'`. Такие «сырые» строки будем разбирать, когда будем говорить про `std::string` и почему лучше всё-таки его.

8.7.3 Двумерные массивы (матрицы)

Массивы могут быть «массивами массивов»:

Фрагмент

```
int m[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

С точки зрения компилятора это две строки по три элемента:

- `m[0]` — массив `int[3]` со значениями `{1, 2, 3}`;
- `m[1]` — `{4, 5, 6}`.

Обход матрицы — вложенный цикл:

Фрагмент

```
for (int i = 0; i < 2; ++i) {
    for (int j = 0; j < 3; ++j) {
        std::cout << m[i][j] << " ";
    }
    std::cout << "\n";
}
```

8.8 Указатель на массив: `int (*) [N]`

Иногда (особенно в задачах на перегрузки и в разборе «декея» массивов) всплывает тип `int (*) [N]` — **указатель на массив из N элементов**.

8.8.1 Разница между `a`, `&a` и `&a[0]`

Пусть есть массив:

Фрагмент

```
int a[5]{1, 2, 3, 4, 5};
```

Тогда:

- В большинстве выражений `a` **превращается** (“декейтся”) в `int*` и указывает на первый элемент: это почти то же самое, что `&a[0]`.
- Выражение `&a[0]` имеет тип `int*` (указатель на первый элемент).
- А вот `&a` имеет тип `int (*) [5]`: **указатель на весь массив как на одно целое**.

Это видно и по тому, как таким указателем пользоваться:

Фрагмент

```
#include <iostream>

int main() {
    int a[5]{1, 2, 3, 4, 5};
    int (*p)[5] = &a;          // p указывает на массив целиком

    std::cout << (*p)[0];      // разыменовали p (получили массив) и взяли [0]
}
```

8.8.2 Массивы в параметрах функций: куда именно “пропадает” размер

Правило “массив в параметре превращается в указатель” работает не только для одномерных массивов.

Фрагмент

```
void f1(int a[10]);           // на самом деле: void f1(int* a);
void f2(int a[5][5]);         // на самом деле: void f2(int (*a)[5]);
```

Почему так: `int a[5][5]` — это “массив из 5 элементов типа `int[5]`”. Когда он декейтся, остаётся указатель на первый “элемент”, то есть на `int[5]`. Поэтому получается `int (*) [5]`.

Ещё одна частая ловушка:

Фрагмент

```
void g(int* a[5]);           // на самом деле: void g(int** a);
```

Потому что `int* a[5]` — это “массив из 5 указателей на `int`”, а массив в параметре опять же превращается в указатель на первый элемент: указатель на `int*`, то есть `int**`.

Важный нюанс: Из-за этого нельзя “перегрузить по записи”. Например, объявления `void g(int**);` и `void g(int* [5]);` на самом деле задают один и тот же тип функции. Если попытаться определить обе версии, вы получите ошибку компиляции о переопределении функции.

8.9 Массивы и функции: почему внутри функции sizeof не спасёт

Немного важной магии.

Фрагмент

```
void print_array(int a[10]) { /* ... */ }
```

Выглядит как «функция, принимающая массив из 10 элементов». Но на самом деле параметр функции — это указатель на `int`, а не реальный массив:

Фрагмент

```
void print_array(int a[10]) {
    std::cout << sizeof(a) << "\n";    // размер указателя, а не массива!
}
```

При передаче массива в функцию он неявно превращается в `int*`. Размер «теряется». Поэтому сигнатура обычно пишется явно как:

Фрагмент

```
void print_array(const int* a, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {
        std::cout << a[i] << " ";
    }
}
```

И вызывается так:

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};
print_array(a, 5);
```

О том, что именно происходит с массивом при передаче (как он «декейится» до указателя), подробно поговорим в главе про указатели и ссылки.

8.10 Циклы по массиву через указатель

Имя массива в выражении почти всегда превращается в указатель на первый элемент:

Фрагмент

```
int a[5] = {10, 20, 30, 40, 50};

int* p = a; // то же самое, что &a[0]
```

`p` — указатель на `int`, который указывает на `a[0]`. Мы можем двигать указатель по массиву:

Фрагмент

```
int* p = a;           // на a[0]
int* q = a + 3;       // на a[3]
```

В лекциях показывали, как берут указатель на любой элемент массива и используют его так же, как индекс: `*(p + k)` и `p[k]` — это одно и то же.

8.10.1 Цикл по массиву через указатель

Классический пример:

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};

int* begin = a;           // &a[0]
int* end   = a + 5;       // один элемент ЗА последним

for (int* p = begin; p != end; ++p) {
    std::cout << *p << " ";
}
```

Здесь `p` бежит от начала до конца массива, а `*p` — текущий элемент. Важно:

- двигаем указатель только в пределах массива;
- `end` указывает «за концом», но не разыменовывается.

Такой стиль похож на то, как работают итераторы в стандартной библиотеке, поэтому полезно к нему привыкнуть.

8.11 Типичные ошибки при работе с массивами

Чтобы потом не искать баги по полдня, вот список классических граблей.

1. Выход за границы массива

Фрагмент

```
int a[10]{};
for (int i = 0; i <= 10; ++i) { // ОШИБКА: должно быть i < 10
    a[i] = i;
}
```

Последняя итерация пишет в `a[10]`, которого не существует — это UB.

2. Использование неинициализированных значений

Фрагмент

```
int a[10]; // мусор
int sum = 0;

for (int i = 0; i < 10; ++i) {
```

```
sum += a[i]; // собираем мусор
}
```

Если не хочешь получить «магические» числа, инициализируй массив: {} или явный список.

3. Непонимание, что массив в функцию передаётся как указатель

Фрагмент

```
void foo(int a[10]) {
    std::size_t n = sizeof(a); // размер указателя, а не 10 * sizeof(int)
}
```

Внутри функции `a` — уже не массив, а `int*`.

4. Перепутанные размеры Особенно больно в двумерных массивах:

Фрагмент

```
int m[2][3];

for (int i = 0; i < 3; ++i) // ОШИБКА: местами поменяли
    for (int j = 0; j < 2; ++j) // размеры по i и j
        m[i][j] = i + j;
```

5. Попытка сделать «динамический массив» через `int a[n]`; В стандарте C++ так нельзя. Это либо ошибка, либо нестандартное расширение компилятора. Для динамического размера нужен `std::vector`, о котором будет отдельная глава.

Массивы дают базовую механику, но в реальном C++ чаще живут более безопасные и удобные обёртки. Далее познакомимся с `std::string` и `std::vector` и научимся пользоваться ими без лишней магии.

Глава 9 `std::string` и `std::vector`

9.1 С-строки против `std::string`

9.1.1 С-строка: историческое наследие

Классический С-подход к строкам:

Фрагмент

```
char s[100] = "hello"; // массив char, в конце скрытый '\0'
```

Строка — это:

- массив `char`;
- в конце обязательно специальный символ `'\0'` (код 0) — терминатор строки.

Работа идёт через функции из `<cstring>`:

Фрагмент

```
#include <cstring>

char s[100] = "hello";
char t[100] = " world";

std::strcat(s, t);           // склеить
std::size_t n = std::strlen(s); // длина до '\0'
```

Проблемы:

- нужно не забывать выделять достаточно большой массив;
- легко выйти за границы: `strcat` вообще не знает, какой у массива размер;
- никакой информации о длине внутри типа — только через `strlen`, который каждый раз считает длину по символам.

Работать так можно, но это неудобно и небезопасно. Поэтому в С++ придумали класс `std::string`.

9.1.2 `std::string`: строка по-человечески

`std::string` — это класс (контейнер), который:

- сам хранит текст;
- **знает свою длину**;
- умеет копироваться, сравниваться, складываться, резаться на подстроки;
- автоматически управляет памятью.

Подключаем заголовок:

Полный пример

```
#include <string>

int main() {
```

```
std::string s = "hello";
}
```

Создание строк:

Фрагмент

```
std::string a;           // пустая строка ""
std::string b = "hello"; // из строкового литерала
std::string c("world");  // тоже из литерала
std::string d(5, 'x');   // "xxxxx"
std::string e = b + " " + c; // конкатенация: "hello world"
```

Строковые литералы `"..."` в C++ имеют тип `const char*`, но `std::string` умеет неявно строиться из них, так что запись `std::string s = "hi";` — нормальна.

9.2 Базовые операции с `std::string`

Представим, что нам дали:

Полный пример

```
#include <iostream>
#include <string>

int main() {
    std::string s = "hello";
}
```

Теперь пробежимся по основным операциям.

9.2.1 Длина строки

Фрагмент

```
std::string s = "hello";

std::size_t n1 = s.size();
std::size_t n2 = s.length(); // то же самое

std::cout << n1 << " " << n2 << "\n"; // 5 5
```

`size()` и `length()` возвращают количество элементов `char` (то есть байт) в строке. В ASCII это совпадает с числом «символов», но в UTF-8 один символ может занимать несколько байт.

Проверка на пустоту:

Фрагмент

```
if (s.empty()) {
    std::cout << "Пусто\n";
}
```

9.2.2 Индексация и доступ к символам

Почти как массив:

Фрагмент

```
std::string s = "hello";

char c0 = s[0]; // 'h'
char c1 = s[1]; // 'e'

s[0] = 'H';      // теперь "Hello"
```

Важно:

- индексация тоже с нуля;
- допустимые индексы: от 0 до `s.size() - 1`;
- `s[s.size()]` — ошибка (выход за границу, UB).

Важный нюанс: Иногда в «отладочных» сборках стандартной библиотеки можно встретить дополнительные проверки даже для `operator[]`. Это **не** гарантия стандарта: в релизной сборке доступ за границы по `[]` остаётся UB, поэтому на такие проверки нельзя полагаться.

Есть безопасный вариант — `at()`:

Фрагмент

```
char c = s.at(10); // если индекс за пределами, бросит std::out_of_range
```

Здесь «бросит `std::out_of_range`» означает: произойдёт **исключение** (exception) — стандартный способ сообщить об ошибке во время выполнения. Если исключение не перехватить, программа обычно завершится. Как перехватывать исключения через `try/catch` мы разберём дальше в отдельной главе про исключения. Для `std::out_of_range` нужен заголовок `<stdexcept>`.

`at()` чуть медленнее, но безопаснее: для отладки и учебных задач — хорошая идея, для суперпроизводительного кода можно вернуться к `operator[]`, когда уже уверен в себе.

9.2.3 Склеивание строк

Самый приятный бонус `std::string` — **конкатенация**:

Фрагмент

```
std::string name = "Alice";
std::string message = "Hello, " + name + "!";

std::cout << message << "\n"; // "Hello, Alice!"
```

Частая ловушка: `"A" + "B"` **не** делает строку. Литералы `"A"` и `"B"` имеют тип массива `const char[...]`, который в выражениях обычно превращается в `const char*`. «Склеивание» работает только если хотя бы один операнд — `std::string`.

Правильно:

Фрагмент

```
using namespace std::string_literals;

auto ok1 = std::string("A") + "B";
auto ok2 = "A"s + "B";
```

Можно использовать `+=`:

Фрагмент

```
std::string s = "abc";
s += "def";    // "abcdef"
s += '!';     // "abcdef!"
```

Или метод `append`:

Фрагмент

```
s.append("XYZ"); // "abcdef!XYZ"
```

9.2.4 Сравнение строк

Строки можно сравнивать обычными операторами:

Фрагмент

```
std::string a = "abc";
std::string b = "abd";

bool eq   = (a == b); // false
bool neq  = (a != b); // true
bool less = (a < b);  // true: "abc" < "abd" лексикографически
```

Сравнение идёт **лексикографически** (по символам слева направо, как в словаре).

9.2.5 Ввод и вывод строк

Вывод строки в `std::cout` — просто:

Фрагмент

```
std::string s = "hello";
std::cout << s << "\n";
```

С чтением есть два варианта.

9.2.5.1 Вариант 1: оператор »

Фрагмент

```
std::string word;
std::cin >> word;
```

Читает одно «слово» до первого пробела или перевода строки.

9.2.5.2 Вариант 2: `std::getline`

Если нужно прочитать всю строку целиком (с пробелами):

Фрагмент

```
std::string line;
std::getline(std::cin, line);
```

Осторожно с комбинацией:

Фрагмент

```
int x;
std::cin >> x;           // читает число, но оставляет '\n' в потоке
std::string line;
std::getline(std::cin, line); // тут сразу прочитается пустая строка
```

Частое решение — перед `getline` после `>>` сделать чтение лишнего символа:

Фрагмент

```
#include <limits>

int x;
std::cin >> x;
// Съедаем остаток текущей строки (включая '\n'), иначе следующий getline() может прочитать
// пустую строку.
// max() берём, чтобы <<промотать>> до конца строки при любом разумном размере буфера.
// Нужен заголовок: #include <limits>
std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');

std::string line;
std::getline(std::cin, line);
```

Этот `ignore` выкидывает оставшийся перевод строки после `>>`, чтобы `getline` читала следующую строку, а не пустую.

9.2.6 Подстроки и поиск

Минимальный набор, которого уже хватает для простых задач.

Подстрока: `substr(pos, count)`

Фрагмент

```
std::string s = "hello world";

std::string hello = s.substr(0, 5); // "hello"
std::string world = s.substr(6, 5); // "world"
```

Поиск подстроки: `find`

Фрагмент

```
std::string s = "abracadabra";

std::size_t pos = s.find("bra"); // позиция первого вхождения или std::string::npos

if (pos != std::string::npos) {
    std::cout << "нашли на позиции " << pos << "\n";
}
```

`npos` — специальная «невозможная» позиция: обычно это максимальное значение `size_t`.

9.2.7 Циклы по `std::string`

Можно ходить по строке как по массиву:

Фрагмент

```
std::string s = "hello";
for (std::size_t i = 0; i < s.size(); ++i) {
    std::cout << s[i] << "\n";
}
```

Можно в стиле «по элементам»:

Фрагмент

```
for (char c : s) {
    std::cout << c << "\n";
}
```

Это range-based for (расширенный for, появившийся в C++11): удобно и читаемо.

Есть и итераторы — это **объекты для обхода контейнера**, которые ведут себя «как указатель»: их можно сравнивать, сдвигать и разыменовывать. Пара `begin()/end()` задаёт полуинтервал «от первого элемента до позиции за последним». (Подробности об итераторах будут дальше, сейчас нам важна идея.)

Фрагмент

```
for (auto it = s.begin(); it != s.end(); ++it) {
    char c = *it;
    std::cout << c << "\n";
}
```

Итераторы работают примерно как указатели: `begin()` указывает на первый символ, `end()` — «за последним». Но итератор не обязан быть просто адресом в памяти (у некоторых контейнеров это отдельный объект-обёртка).

9.2.8 `std::string` и многобайтовые символы (коротко)

Важно понимать: `std::string` — это **контейнер байт** (фактически `std::basic_string<char>`).

В простых задачах (ASCII, латиница, простые символы) можно считать, что один символ = один байт. Но если использовать UTF-8 и небуквенные символы (кириллица, эмодзи и т. п.), один символ может занимать несколько байт, и тогда:

- `s.size()` — это количество **байт**, а не «человеческих» символов;
- `s[i]` даёт отдельный байт, не обязательно целый символ.

Для нашего курса обычно достаточно думать в терминах «один байт — один символ», чтобы не уходить в дебри Юникода. Но держи в голове, что в реальной жизни всё сложнее.

9.3 `std::vector`: динамический массив

Теперь переходим к `std::vector`.

Обычный массив:

Фрагмент

```
int a[10];
```

- размер фиксирован навсегда;
- компилятору надо знать его на этапе компиляции.

`std::vector` — это **динамический массив**:

- размер можно менять во время работы программы;
- элементы лежат подряд в памяти (как в обычном массиве);
- есть куча удобных методов.

Подключаем:

Фрагмент

```
#include <vector>
```

9.3.1 Создание `std::vector`

Базовые варианты:

Фрагмент

```
std::vector<int> v;           // пустой вектор
std::vector<int> v2(10);      // 10 элементов, все = 0
std::vector<int> v3(10, 42);  // 10 элементов, все = 42
std::vector<int> v4 = {1, 2, 3}; // список инициализации
```

Ещё пример с `double`:

Фрагмент

```
std::vector<double> temps = {36.6, 37.2, 38.0};
```

9.3.2 Размер и доступ по индексу

У вектора есть методы:

- `size()` — текущий размер (сколько элементов реально хранит);

- `empty()` — пустой ли;
- `operator[]` — доступ без проверки;
- `at()` — доступ с проверкой границ.

Пример:

Фрагмент

```
std::vector<int> v = {10, 20, 30};

std::size_t n = v.size(); // 3

v[0] = 100;
int x = v[1];           // 20

int y = v.at(2);        // 30, с проверкой

// v.at(3);             // выбросит std::out_of_range
```

Важно: как и с массивами, **индексы от 0 до `size()` - 1**.

9.3.3 Добавление и удаление элементов

Самое вкусное в `std::vector` — он может расти.

9.3.3.1 `push_back`: добавить в конец

Фрагмент

```
std::vector<int> v;

v.push_back(10); // v = {10}
v.push_back(20); // v = {10, 20}
v.push_back(30); // v = {10, 20, 30}
```

9.3.3.2 `pop_back`: удалить последний

Фрагмент

```
v.pop_back(); // v = {10, 20}
```

(Предполагается, что вектор не пустой.)

9.3.3.3 Ещё полезные методы

Фрагмент

```
v.clear(); // удалить все элементы, size() становится 0
```


Фрагмент

```
v.resize(10);      // изменить размер на 10 элементов
v.resize(10, 42);  // если увеличиваем, новые элементы = 42
```

9.3.4 `size()` против `capacity()`

У вектора есть два понятия:

- `size()` — сколько элементов **фактически** хранится;
- `capacity()` — сколько элементов он может хранить **без перераспределения памяти**.

Важный нюанс: Про `capacity()` и «внезапные» переезды. `std::vector` иногда перевыделяет память (обычно при росте), и тогда **все старые указатели/ссылки/итераторы на элементы могут стать недействительными**.

Если вы заранее знаете примерный размер, `reserve(n)` часто спасает от лишних переездов и ускоряет `push_back`.

При `push_back` вектор обычно:

- увеличивает `size()` на 1;
- если `size()` превысил `capacity()`, вектор **перевыделяет память** (создаёт новый массив побольше и копирует туда элементы).

Это важно для производительности и для указателей/итераторов (об этом ниже), но для начала можно просто помнить: иногда `push_back` дешёвый, а иногда дорогой (когда происходит перевыделение).

Можно заранее зарезервировать память:

Фрагмент

```
std::vector<int> v;
v.reserve(1000); // выделить место сразу под 1000 элементов
```

Но это уже тонкая оптимизация — для учебных задач можно и без неё.

9.4 Циклы по `std::vector`

По вектору можно ходить точно так же, как по массиву.

9.4.1 Цикл `for` с индексом

Фрагмент

```
std::vector<int> v = {10, 20, 30, 40, 50};

for (std::size_t i = 0; i < v.size(); ++i) {
    std::cout << "v[" << i << "] = " << v[i] << "\n";
}
```

Здесь `v.size()` — как «N» в предыдущей главе.

9.4.2 Range-based for (по элементам)

Удобный синтаксис:

Фрагмент

```
for (int x : v) {
    std::cout << x << " ";
}
```

Если нужно модифицировать элементы, можно пробежать по ссылке:

Фрагмент

```
for (int& x : v) {
    x *= 2; // удвоить каждый элемент
}
```

После этого `v` изменится.

9.4.3 Итераторы

Итераторы ведут себя почти как указатели.

Фрагмент

```
for (auto it = v.begin(); it != v.end(); ++it) {
    int x = *it;
    std::cout << x << " ";
}
```

- `v.begin()` — итератор на первый элемент;
- `v.end()` — итератор «за последним», его нельзя разыменовывать, но по нему удобно писать условия;
- `*it` — текущий элемент.

Итераторы очень похожи на указатели по духу (иногда даже являются настоящими указателями), поэтому всё, что ты видел в главе про массивы и указатели, отлично переносится и сюда.

9.5 std::vector и функции

Обычно вектор в функции передают **по ссылке**, чтобы:

- не копировать все элементы;
- при необходимости изменять вектор внутри функции.

Фрагмент

```
void print_vector(const std::vector<int>& v) {
    for (int x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";
}
```

```

}

int main() {
    std::vector<int> a = {1, 2, 3};
    print_vector(a); // передаём по константной ссылке
}

```

Если функция должна менять вектор:

Фрагмент

```

void add_one(std::vector<int>& v) {
    for (int& x : v) {
        ++x;
    }
}

```

В следующей главе, когда будем говорить про ссылки, мы к этим примерам вернёмся и разберёмся, как это всё работает под капотом.

9.6 Важные детали: память и сломанные указатели

Очень важный момент, про который легко забыть.

9.6.1 Перевыделение памяти

Когда вектору не хватает `capacity`, он:

1. выделяет новый кусок памяти побольше;
2. копирует туда все элементы;
3. освобождает старую память.

Все указатели, ссылки и итераторы на элементы старого массива становятся недействительными.

Пример:

Фрагмент

```

std::vector<int> v = {1, 2, 3};
int* p = &v[0];           // указатель на первый элемент

v.push_back(4);           // возможно, перевыделение памяти

// теперь p МОЖЕТ УЖЕ БЫТЬ НЕВАЛИДНЫМ указателем

```

Поэтому правило:

Не храни голые указатели/ссылки на элементы вектора, если собираешься дальше его модифицировать (особенно добавлять элементы).

Когда сильно надо — есть способы аккуратно с этим работать, но для начала лучше просто этого не делать.

9.6.2 shrink_to_fit и другие методы

Есть и другие методы, например `shrink_to_fit()`: он пытается уменьшить `capacity` до `size()` и освободить лишнюю память. Для базового курса достаточно знать:

- `size()` — сколько элементов;
- `clear()` — удалить всё;
- `push_back` / `pop_back` — работать с концом;
- `resize()` — изменить размер;
- `reserve()` — заранее зарезервировать память.

Остальное пригодится позже, когда станем делать более сложные проекты.

9.7 Вектор векторов: почти двумерные массивы

Частая задача: нужна таблица или матрица. Можно, конечно, объявить `int a[rows][cols];`, но размер статичен. `std::vector` позволяет сделать **динамическую матрицу**.

9.7.1 Прямоугольная матрица $rows \times cols$

Например, хотим матрицу 3×4 :

Фрагмент

```
int rows = 3;
int cols = 4;

std::vector<std::vector<int>> m(rows, std::vector<int>(cols, 0));
```

Теперь:

- `m.size() == 3` — количество строк;
- `m[i].size() == 4` — количество столбцов в строке `i`;
- элемент на позиции `(i, j)` — `m[i][j]`.

Обход:

Фрагмент

```
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        std::cout << m[i][j] << " ";
    }
    std::cout << "\n";
}
```

9.7.2 «Рваный» массив

Плюс `std::vector` в том, что строки могут быть разной длины:

Фрагмент

```
std::vector<std::vector<int>> v(3);
```

```
v[0] = {1, 2, 3};  
v[1] = {10, 20};  
v[2] = {42};
```

Это уже не матрица, а «список списков». В математике так не очень любят, а в программировании — очень даже полезно.

9.8 Сравнение: массив, `std::string`, `std::vector`

Давай подведём промежуточный итог и сравним всё, что у нас есть.

9.8.1 Обычный массив `T a[N];`

Плюсы:

- максимально простой и быстрый;
- использует ровно N элементов памяти;
- удобно использовать в низкоуровневых задачах.

Минусы:

- размер фиксирован при компиляции;
- сам не знает свой размер (`sizeof` работает только рядом с объявлением);
- легко выйти за границы, компилятор не всегда поможет.

Использовать, когда:

- размер действительно фиксирован и известен заранее;
- нужна совместимость с С-кодом или ОС.

9.8.2 `std::string`

По сути — «умный массив `char` + куча методов».

Плюсы:

- знает свой размер;
- умеет складываться, резаться, искать подстроки;
- безопаснее С-строк;
- совместим с С-API (можно получить `c_str()` — С-строку, если надо).

Минусы:

- всё ещё надо помнить, что это контейнер байт, а не «магия Юникода»;
- чуть тяжелее, чем голый `char[]`, но этого обычно даже не замечают.

Использовать почти всегда, когда работаешь со строками уровня задач по программированию.

9.8.3 `std::vector<T>`

«Умный массив» любого типа `T`.

Плюсы:

- динамический размер;
- компактное хранение подряд в памяти (как массив);

- знает свой `size()` и частично управляет `capacity()`;
- удобно итерироваться;
- безопаснее обычных массивов (есть `at()`).

Минусы:

- переназначение памяти может ломать указатели/ссылки/итераторы;
- чуть сложнее обычного массива.

Использовать:

- когда размер данных заранее неизвестен или меняется;
- почти всегда, когда хочется «массив, но удобно».

На этом мы заканчиваем знакомство с `std::string` и `std::vector`.

Контейнеры и функции постоянно работают с памятью и адресами. Чтобы понимать, что происходит под капотом (и не ловить UB), нужно освоить косвенный доступ: указатели и ссылки.

Глава 10 Указатели и ссылки

Всё это время мы работали с переменными «напрямую»:

- `int x;`
- `double y;`
- `std::string s;`

Но под капотом у любой переменной есть ещё одна важная штука — **адрес в памяти**. А C++ даёт к нему прямой доступ — через **указатели** и **ссылки**.

С их помощью можно:

- эффективно передавать большие объекты в функции;
- писать функции, которые меняют свои аргументы;
- работать с динамической памятью и динамическими структурами данных;
- делать красивые API наподобие `void foo(std::vector<int>& v)`.

В этой главе мы познакомимся с адресами в памяти и научимся получать их оператором `&`. Разберёмся, что такое указатель `T*`, как он связан с массивами, и какие ошибки с ним чаще всего приводят к UB.

Потом обсудим ссылки `T&` — чем они отличаются от указателей и почему их так любят в параметрах функций. В конце соберём самые частые грабли, чтобы работа с косвенным доступом была предсказуемой.

10.1 Адрес в памяти и оператор `&`

У любой переменной есть **адрес в памяти** — номер ячейки, где она живёт.

Полный пример

```
#include <iostream>

int main() {
    int x = 10;
    std::cout << "x = " << x << "\n";
    std::cout << "&x = " << &x << "\n";
}
```

Здесь:

- `x` — значение;
- `&x` — адрес переменной `x`.

Оператор `&` в этом контексте называется **оператор взятия адреса**. Адрес — это не «волшебная строка», а обычное значение особого типа, которое хранится в переменной-указателе.

10.2 Указатель: объявление, инициализация, `nullptr`

Указатель — это переменная, которая хранит **адрес** другого объекта.

Общий вид:

Фрагмент

```
тип_на_который_указывает* имя;
```

Примеры:

Фрагмент

```
int* p;           // указатель на int
double* q;        // указатель на double
std::string* s;   // указатель на std::string
```

Важно: на момент объявления указатель ни на что не указывает (в нём мусор). Разыменовывать такой указатель — UB и боль. Уточнение: «мусор» — это про локальные переменные автоматического хранения без инициализации. Глобальные и `static` переменные проходят нуль-инициализацию (для указателей это эквивалент `nullptr`), но всё равно лучше писать явную инициализацию ради читаемости.

10.2.1 Инициализация указателя адресом

Фрагмент

```
int x = 42;
int* p = &x; // p хранит адрес x

std::cout << "x = " << x << "\n";
std::cout << "p = " << p << "\n"; // адрес
```

Пока `x` жив, `p` указывает на него.

10.2.2 Пустой указатель и `nullptr`

Иногда нужно сказать: «указатель ни на что не указывает». Для этого есть специальное значение `nullptr` (C++11 и позже):

Фрагмент

```
int* p = nullptr; // p пока не указывает ни на что
```

Проверка:

Фрагмент

```
if (p == nullptr) {
    std::cout << "p пустой\n";
}
```

Нельзя разыменовывать `nullptr`:

Фрагмент

```
*p = 10; // если p == nullptr -> UB
```

Раньше вместо `nullptr` использовали `0` или `NULL`, но в современном C++ лучше писать именно `nullptr`: он имеет свой отдельный тип и меньше поводов для путаницы.

10.3 Разыменование: оператор *

Оператор * в контексте указателя — это **разыменование**, т.е. «сходить по адресу и взять то, что там лежит».

Фрагмент

```
int x = 10;
int* p = &x;

std::cout << *p << "\n"; // 10
```

Тут:

- `p` — указатель (адрес);
- `*p` — сам объект, на который указывает `p` (переменная `x`).

Через указатель можно и читать, и писать:

Фрагмент

```
*p = 20; // меняем x через указатель
std::cout << x << "\n"; // 20
```

Формально:

- тип `p` — `int*`;
- тип `*p` — `int&` (ссылка на `int`), но пока можно думать просто «`int`-значение по этому адресу».

Главное правило: **разыменовывать можно только валидный указатель**. Он либо указывает на существующий объект, либо на элемент массива внутри его границ. Все остальные случаи — `undefined behavior`.

10.4 Указатели и массивы

Из прошлой главы: имя массива в выражениях обычно «становится» указателем на первый элемент.

Фрагмент

```
int a[5] = {10, 20, 30, 40, 50};

int* p = a; // то же самое, что &a[0]
int* q = &a[2]; // указатель на третий элемент
```

10.4.1 Индексация через указатель

Всё, что мы делали с индексами `a[i]`, можно делать и через указатель:

Фрагмент

```
int a[5] = {10, 20, 30, 40, 50};
int* p = a; // &a[0]
```

```
std::cout << p[0] << "\n";    // 10 (оператор [] у указателя!)
std::cout << p[1] << "\n";    // 20
std::cout << *(p + 2) << "\n"; // 30
```

Запомнить удобно так: $a[i]$ — это синтаксический сахар для $*(a + i)$. То есть $p[i]$ и $*(p + i)$ — одно и то же.

10.4.2 Арифметика указателей

Если p — указатель на `int`, то:

- $p + 1$ — указатель на следующий `int` в памяти;
- $p + 2$ — через два `int` и т.д.

Компилятор сам умножает на `sizeof(int)` под капотом. Поэтому арифметику указателей можно использовать только для обхода массива:

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};

int* begin = a;        // &a[0]
int* end = a + 5;       // один элемент ЗА последним

for (int* p = begin; p != end; ++p) {
    std::cout << *p << " ";
}
```

Выглядит уже очень по-C++-шному — такой же стиль используют итераторы.

Нельзя прыгать указателем куда попало и выходить за границы массива (разыменовывать `end` нельзя).

10.5 Массивы и функции: «декей» к указателю

Ключевой момент: массив при передаче в функцию превращается в указатель на первый элемент.

Фрагмент

```
void foo(int a[10]) {
    // на самом деле параметр имеет тип int*
}
```

Эти три объявления эквивалентны:

Фрагмент

```
void foo(int a[10]);
void foo(int a[]);
void foo(int* a);
```

Поэтому внутри функции:

Фрагмент

```
void foo(int a[10]) {
    std::cout << sizeof(a) << "\n"; // размер указателя, НЕ всего массива
}
```

Чтобы знать, сколько элементов в массиве, размер обычно передают отдельно:

Фрагмент

```
void print_array(const int* a, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {
        std::cout << a[i] << " ";
    }
}

int main() {
    int a[5] = {1, 2, 3, 4, 5};
    print_array(a, 5); // имя массива -> int*
}
```

Понимание того, что «массив -> указатель», очень важно для всей остальной жизни в C++.

10.6 Указатели и const

С указателями и const есть любимая тема для путаницы, но нам нужен хотя бы минимум.

10.6.1 Указатель на константное значение

Фрагмент

```
int x = 10;
const int* p = &x; // указатель на const int
```

Что это значит:

- по `p` нельзя менять значение: `*p = 20;` — ошибка компиляции;
- сам `x` всё ещё неконстантный — его можно менять напрямую: `x = 30;;`
- можно перенаправить указатель: `p = nullptr;`, `p = &другая_переменная;`.

Читаем справа налево: `p` — указатель на `const int`.

Важный нюанс: Константность и указатели. Эти две записи означают разные вещи:

- `const int* p` — нельзя менять `*p`, но можно переставлять `p` на другой адрес;
- `int* const p` — нельзя переставлять `p`, но можно менять `*p`.

Читать удобно справа налево: `p` — *const* указатель на *int*, либо указатель на *const int*.

10.6.2 Константный указатель на неконстантное значение

Фрагмент

```
int x = 10;
int y = 20;

int* const p = &x;  // константный указатель на int
```

Теперь:

- `*p = 30;` — можно (меняем `x`);
- `p = &y;` — нельзя (сам указатель константный).

Читаем: `p` — `const` указатель на `int`.

10.6.3 Указатель на `const` и `const` указатель

Можно совместить оба:

Фрагмент

```
const int* const p = &x;
```

Тогда:

- нельзя менять `*p`;
- нельзя менять `p`.

В учебных задачах чаще всего нужен первый вариант — **указатель на константное значение**: `const T* p`. Он говорит: «я смотрю на объект, но обещаю его не менять».

10.6.4 Бокс: для тех, кто хочет пострадать — `const` и ссылки на указатели

Если базовые варианты

- `const int* p` — указатель на константу,
- `int* const p` — константный указатель,
- `const int* const p` — константный указатель на константу

уже не вызывают паники — можно посмотреть на ещё один уровень адской сложности: **ссылки на указатели с `const`**.

Зачем это вообще надо?

- иногда полезно передать в функцию «указатель, который можно поменять», то есть параметр типа `T*&`;
- но при этом важно не разломать систему константности.

10.6.4.1 Вспоминаем базу

Фрагмент

```
int x = 10;
const int cx = 20;

int*      p_x = &x;  // указатель на int
```

```
const int* p_cx = &cx; // указатель на const int
```

Можно:

- писать через `p_x` (`*p_x = 15;`) — это ок;
- читать через `p_cx`, но не писать (`*p_cx = 25;` — ошибка компиляции).

10.6.4.2 Ссылка на указатель: `T*&`

Тип `T*&` — это **ссылка на указатель**:

Фрагмент

```
int* p_x = &x;
int*& r_p_x = p_x; // r_p_x --- ссылка на указатель p_x
```

Через `r_p_x` можно:

- читать и писать в `*p_x`;
- переназначать сам указатель (`r_p_x = &другой_объект;`).

Это что-то вроде «передачи указателя по ссылке».

10.6.4.3 Почему `const int*&` и `int* const&` — минное поле

Фрагмент

```
int*      p_x  = &x; // указатель на неконстантный int
const int* p_cx = &cx; // указатель на const int

int* const cp_x = &x; // const-указатель на int
```

Теперь два подозрительных объявления ссылок:

Фрагмент

```
const int*& r1 = p_cx; // ОК: ссылка на указатель на const int
const int*& r2 = p_x;  // ошибка: пытаемся связать с int*
```

Почему вторая строка — ошибка?

Если бы компилятор её разрешил, можно было бы сделать так:

Фрагмент

```
int*      p_x  = &x;
const int*& r2 = p_x; // предположим, это скомпилировалось

const int cx = 100;
r2 = &cx;           // теперь p_x указывает на const int!

*p_x = 42;          // модифицируем cx через int* -> ломаем const
```

Мы получили способ **снять** `const` с объекта, объявленного как `const int cx`. Такое поведение запрещено правилами языка, поэтому компилятор не даёт привязать `const int*&` к `int*`. Аналогичная история в другую сторону:

Фрагмент

```
int*      p_x  = &x;
const int* p_cx = &cx;

int* const& r3 = p_x;    // ОК: ссылка на const-указатель int* (не переназначить p_x)
int* const& r4 = p_cx;    // ошибка: пытаемся привязать к const int*
```

Логика та же: если бы r4 было разрешено, можно было бы через него получить обычный int*, указывающий на const int, и снова сломать константность объекта.

10.6.4.4 Суть, которую важно вынести

Если не вдаваться в формальные правила:

Нельзя «расширять права» через ссылку: если у тебя был указатель на **неконстанту**, получив через ссылку указатель на **константу**, ты теоретически можешь потом «подсунуть» ему адрес константы и модифицировать её.

Компилятор видит потенциальную дыру в системе const и просто запрещает такие привязки на уровне типов.

Для повседневного кода запомни простое правило:

- никогда не пытайся писать что-то вроде const T*& или T* const&, если ты не очень хорошо понимаешь, зачем это делаешь;
- в 99% случаев тебе нужен либо T*, либо const T*, либо T&, либо const T&.

10.7 Ссылки: «кличка» для объекта

Теперь переключаемся на **ссылки** (& в объявлении типа). Ссылка — это ещё одно имя (алиас) уже существующего объекта.

Фрагмент

```
int x = 10;
int& r = x;    // r --- ссылка на x

r = 20;        // меняем x через ссылку
std::cout << x << "\n"; // 20
```

Особенности ссылок:

- должны быть инициализированы при объявлении — нельзя сделать «пустую ссылку»;
- не могут «переключаться» на другой объект: один раз привязалась — и всё;
- обращение к ссылке то же самое, что обращение к самому объекту (нет явного разыва).

Можно думать так: «ссылка — это переменная без собственных данных, просто другое имя».

10.7.1 Ссылка на `const`

Фрагмент

```
int x = 10;
const int& r = x;
```

Теперь:

- `r` нельзя использовать для изменения `x`: `r = 20;` — ошибка;
- `x` можно менять напрямую.

Такие ссылки часто используют в параметрах функций, чтобы не копировать большой объект и гарантировать, что функция его не изменит.

10.7.2 Ссылки и временные объекты

Интересный момент: `const T&` можно привязать к **временному объекту**:

Фрагмент

```
const std::string& ref = std::string("hello");
```

Временная строка продлевает свою жизнь до конца области видимости `ref`.

Детали этого поведения разберём позже, когда будем глубже обсуждать время жизни объектов. Пока главное: `const T&` **очень удобно принимать параметры функции**, потому что оно умеет привязываться и к обычным объектам, и к временным.

10.7.3 Категории выражений на пальцах

У выражений в C++ есть **категория значения** — грубо говоря, является ли оно «настоящим объектом» или «временным значением». Если пока не используешь `std::move` и `rvalue`-ссылки, можно держать в голове очень простую модель:

- **lvalue** — объект со стабильным адресом (обычно имеет имя), в него можно присваивать. Примеры: `x`, `*p`, `v[0]`.
- **rvalue** — временное значение, которое живёт недолго. Примеры: `42`, `x + y`, `std::string("hi")`.

В стандарте картинка чуть тоньше: `rvalue` делится на **prvalue** (чисто временные значения) и **xvalue** («истекающие» значения, из которых можно безопасно перемещать ресурсы). Эти термины начинают играть роль, когда появятся перемещение, `std::move` и `T&&` — к ним мы вернёмся в главе про жизненный цикл и перемещение объектов.

10.8 Параметры функций: указатели vs ссылки

Зачем нам вообще указатели и ссылки? Один из главных ответов: для **параметров функций**.

10.8.1 Передача по значению (копия)

Фрагмент

```

void inc(int x) {
    ++x; // меняем локальную копию
}

int main() {
    int a = 10;
    inc(a);
    std::cout << a << "\n"; // всё ещё 10
}

```

Здесь `a` копируется в `x`, и изменения внутри функции не видны снаружи.

10.8.2 Передача указателя

Фрагмент

```

void inc(int* p) {
    if (p != nullptr) {
        ++(*p);
    }
}

int main() {
    int a = 10;
    inc(&a);
    std::cout << a << "\n"; // 11
}

```

Функция получает указатель на `a` и меняет по нему оригинальную переменную. Минусы: нужно передавать `&a`, следить за `nullptr`, разыменовывать внутри `*p`.

10.8.3 Передача по ссылке

Более «C++-шный» вариант:

Фрагмент

```

void inc(int& x) {
    ++x; // меняем сам аргумент
}

int main() {
    int a = 10;
    inc(a);           // без &
    std::cout << a;   // 11
}

```

Снаружи выглядит как обычный вызов функции по значению, а внутри мы работаем с оригинальной переменной.

10.8.4 Константная ссылка для «чтения»

Если функция должна только читать объект, но не менять его:

Фрагмент

```
void print_vector(const std::vector<int>& v) {
    for (int x : v) {
        std::cout << x << " ";
    }
}
```

Плюсы:

- не копируем весь вектор (экономим время и память);
- в сигнатуре явно видно, что функция не меняет `v`.

10.8.5 Что выберут реальные люди

- нужно менять аргумент: `T&` или `T*` (в современном C++ чаще `T&`, если `nullptr` не нужен);
- нужно только читать большой объект: `const T&`;
- нужен «может быть, есть объект, а может, нет» — чаще указатель `T*` (или более продвинутые штуки вроде `std::optional<T>`).

10.8.6 Пример: три версии `swap`

Давай на одном и том же примере посмотрим, чем передача по значению отличается от указателей и ссылок. Классика жанра — функция `swap`, которая должна поменять местами два числа.

10.8.6.1 Версия 1. Передача по значению: не работает

Самый очевидный вариант:

Фрагмент

```
void swap(int x, int y) {
    int tmp = x;
    x = y;
    y = tmp;
}
```

И вызываем:

Фрагмент

```
int a = 10;
int b = 20;

swap(a, b); // ожидаем a = 20, b = 10
```

Проблема в том, что `a` и `b` копируются в параметры `x` и `y`. Внутри функции меняются только копии, а исходные переменные снаружи остаются прежними.

Можно представить себе стек кадра вызова:

- наверху лежат локальные `x` и `y` (копии `a` и `b`);
- мы меняем их местами;
- при выходе из функции этот кадр стека просто выбрасывается, вместе с изменениями.

В итоге после вызова `swap(a, b)`; значения `a` и `b` не меняются.

10.8.6.2 Версия 2. Через указатели: работает, но синтаксис шумный

Если мы хотим менять именно оригинальные переменные, можно передать в функцию их адреса:

Фрагмент

```
void swap(int* x, int* y) {
    int tmp = *x;
    *x = *y;
    *y = tmp;
}
```

Теперь вызов:

Фрагмент

```
int a = 10;
int b = 20;

swap(&a, &b); // передаём адреса a и b
```

Разбор:

- `&a` и `&b` — адреса переменных, тип `int*`;
- внутри функции `x` и `y` — указатели на `int`;
- разыменование `*x` даёт доступ к самой переменной `a`, `*y` — к `b`.

Эта версия уже меняет реальные `a` и `b`, всё честно. Но синтаксис получается шумный:

- при вызове нужно писать `&a`, `&b`;
- внутри функции — постоянно разыменовывать `*x`, `*y`.

Это «сишный» стиль: рабочий, но C++ даёт более приятный вариант.

10.8.6.3 Версия 3. Через ссылки: нормальный C++-шный вариант

В C++ можно сказать функции: «работай не с копиями, а сразу с самими переменными». Для этого используется передача по ссылке:

Фрагмент

```
void swap(int& x, int& y) {
    int tmp = x;
    x = y;
    y = tmp;
}
```

Вызов:

Фрагмент

```
int a = 10;
int b = 20;

swap(a, b);    // без & и без *
```

Что происходит:

- `x` становится ссылкой на `a`;
- `y` — ссылкой на `b`;
- все операции с `x` и `y` — это операции с исходными переменными `a` и `b`.

Снаружи выглядит как обычный вызов по значению, а изнутри это «как будто нам передали указатели и мы их разыменовываем», только без лишнего синтаксиса `*` и `&`.

10.8.6.4 Итого по трём версиям

- `void swap(int x, int y)` — не меняет аргументы, потому что работает с копиями.
- `void swap(int* x, int* y)` — меняет аргументы через указатели, но заставляет писать `&` при вызове, помнить про `nullptr` и постоянно разыменовывать `*x`, `*y`.
- `void swap(int& x, int& y)` — C++-шный «золотой стандарт»: вызов выглядит естественно (`swap(a, b);`), функция может менять переданные переменные и не нужно разыменовывать указатели.

Именно версию со ссылками дальше и будем считать «правильной» реализацией `swap` в стиле C++.

10.9 Возврат по ссылке

Функция может возвращать не только значение, но и **ссылку** на что-то:

Фрагмент

```
int& max_ref(int& a, int& b) {
    return (a > b) ? a : b;
}

int main() {
    int x = 10;
    int y = 20;

    int& m = max_ref(x, y);
    m = 100;    // меняем либо x, либо y

    std::cout << x << " " << y << "\n"; // 10 100
}
```

Здесь `max_ref` возвращает ссылку на один из аргументов. Меняя `m`, мы меняем исходную переменную.

Очень важное правило: нельзя возвращать ссылку на локальную переменную.

Фрагмент

```
int& bad() {
    int x = 10;
    return x; // ОПАСНО: x уничтожится при выходе из функции
}
```

После выхода из `bad` переменная `x` перестаёт существовать, а ссылка на неё становится висячей (dangling reference). Дальше — UB.

Важно: продление жизни временного работает, когда временный объект *непосредственно* привязан к `const T&` в том же выражении. Но если функция возвращает ссылку на временный объект, это всё равно висячая ссылка и UB.

Мини-пример: здесь продление жизни *не* спасает, потому что временный живёт только внутри `bad_ref`:

Фрагмент

```
const std::string& bad_ref() {
    return std::string("hello"); // UB: возвращаем ссылку на временный
}

int main() {
    const std::string& ok = std::string("ok"); // ok: жизнь временного продлена до конца обл
    // const std::string& r = bad_ref();          // UB: r станет висячей ссылкой
}
```

Возвращать по ссылке можно только то, что живёт дольше, чем функция (например, глобальные переменные, поля статических объектов) или аргументы, переданные в функцию по ссылке.

10.10 Указатели, ссылки и `std::vector/std::string`

Связка из предыдущих глав.

10.10.1 Ссылки на элементы

Фрагмент

```
std::vector<int> v = {1, 2, 3, 4, 5};

int& ref = v[2]; // ссылка на третий элемент
ref = 100;       // меняем v[2]

std::cout << v[2] << "\n"; // 100
```

Но аккуратно: если после этого вектор изменит свою память (например, через большой `push_back`), ссылка может стать висячей. В учебных задачах это редко всплывает, но в реальной жизни очень важно помнить.

10.10.2 Range-based for по ссылке

Чтобы менять элементы контейнера прямо в цикле, удобно писать:

Фрагмент

```
std::vector<int> v = {1, 2, 3, 4, 5};

for (int& x : v) { // x --- ссылка на элемент
    x *= 2;
}
```

После цикла `v` станет {2, 4, 6, 8, 10}.

С const:

Фрагмент

```
const std::vector<int> v = {1, 2, 3};

for (const int& x : v) {
    std::cout << x << " ";
}
```

10.10.3 Функции с `std::string` и `std::vector`

Типичные сигнатуры:

Фрагмент

```
void to_upper(std::string& s);           // менять строку
void print(const std::string& s);        // читать строку

void append_zero(std::vector<int>& v);    // менять вектор
void print(const std::vector<int>& v);    // читать вектор
```

Указатели по-прежнему полезны (особенно в низкоуровневом коде), но в обычном C++-коде на верхнем уровне чаще увидишь именно ссылки.

10.11 Хардкор: указатели на функции

Мы уже знаем, что указатель может хранить адрес переменной. Но в C++ (как и в C) можно хранить ещё и адрес функции — это **указатель на функцию**.

Звучит страшно, но идея простая:

Указатель на функцию — это переменная, в которой хранится «откуда начать выполнять код».

Зачем это нужно?

- передавать функцию как аргумент другой функции (например, компаратор для сортировки);
- реализовывать callbacks;
- строить простенькие таблицы обработчиков (как массив функций).

10.11.1 Указатель на функцию: синтаксис

Пусть есть функция:

Фрагмент

```
void print_int(int x) {
    std::cout << "x = " << x << "\n";
}
```

Тип этой функции можно читать так: «функция, принимающая `int` и возвращающая `void`».

Тип указателя на такую функцию:

Фрагмент

```
void (*pf)(int);
```

Читаем с середины:

- `pf` — имя переменной;
- `*pf` — «`pf` — указатель»;
- `void (*pf)(int)` — «`pf` — указатель на функцию, принимающую `int` и возвращающую `void`».

Инициализация:

Фрагмент

```
void (*pf)(int) = &print_int; // можно и без &: = print_int;
```

Вызов:

Фрагмент

```
pf(42);           // вызов через указатель
(*pf)(42);        // то же самое, более явно
```

Форма `pf(42)` работает, потому что компилятор сам подставляет разыменование.

10.11.2 Указатель на функцию как параметр другой функции

Сделаем функцию, которая будет применять другую функцию ко всем элементам вектора:

Фрагмент

```
void for_each(
    const std::vector<int>& v,
    void (*f)(int)           // указатель на функцию, принимающую int
) {
    for (int x : v) {
        f(x);                // вызов переданной функции
    }
}
```

Можно передать любую функцию с сигнатурой `void func(int)`:

Фрагмент

```

void print_int(int x) {
    std::cout << x << " ";
}

void print_square(int x) {
    std::cout << x * x << " ";
}

int main() {
    std::vector<int> v = {1, 2, 3};

    for_each(v, print_int);    // выведет: 1 2 3
    std::cout << "\n";

    for_each(v, print_square); // выведет: 1 4 9
    std::cout << "\n";
}

```

Так `for_each` получает «стратегию» в виде указателя на функцию и применяет её к каждому элементу.

10.11.3 Указатель на функцию-компаратор (например, для `std::sort`)

Очень частый сценарий — передать в алгоритм **компаратор**.

Пусть есть функция сравнения:

Фрагмент

```

bool my_less(int a, int b) {
    return a < b;
}

```

Её тип: «функция, принимающая два `int` и возвращающая `bool`».

Тип указателя:

Фрагмент

```

bool (*cmp)(int, int) = &my_less; // или просто = my_less;

```

Мы можем передать `my_less` прямо в `std::sort`:

Фрагмент

```

// std::sort находится в заголовке <algorithm>
#include <algorithm>

std::vector<int> v = {5, 2, 8, 1};

std::sort(v.begin(), v.end(), my_less); // третий аргумент -- функция-компаратор

```

Когда вы передаёте имя обычной функции, оно неявно превращается в указатель на функцию. Лямбды и функторы в общем случае передаются как вызываемые объекты; только лямбда без захвата может автоматически конвертироваться в указатель. Компаратор должен задавать строгий слабый порядок, иначе `std::sort` может вести себя как угодно.

10.11.4 А как же лямбды и функторы?

В современном C++ вместо «голых» указателей на функции часто используют:

- **лямбды**:

Фрагмент

```
std::sort(v.begin(), v.end(),
        [](int a, int b) { return a < b; });
```

- или **функторы** (объекты с перегруженным `operator()`).

Они могут быть гибче, чем указатель на функцию: умеют захватывать состояние, работать с шаблонами и т.п.

Но понимание указателей на функции полезно как минимум по трём причинам:

- это основа для понимания «как всё это вообще устроено под капотом»;
- многие старые, но живые библиотеки ожидают именно `T (*f)(...)`;
- про это любят задавать вопросы на собеседованиях.

10.12 Типичные ошибки с указателями и ссылками

Чтобы потом меньше ловить загадочные краши, соберём коллекцию «не делай так»:

1. Разыменование `nullptr` или неинициализированного указателя

Фрагмент

```
int* p;    // мусор
*p = 10;   // UB
```

Всегда инициализируй указатели:

Фрагмент

```
int* p = nullptr;
```

И проверяй перед использованием, если `nullptr` возможен.

2. Выход за границы массива через указатель

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};
int* p = a + 5; // указатель за концом

int x = *p;     // UB
```

`a + 5` — «за последним элементом», его можно использовать в сравнениях и циклах, но нельзя разыменовывать.

3. Висячие указатели

Фрагмент

```
int* p;
{
    int x = 10;
    p = &x;
} // x умер

*p = 20; // p указывает в никуда -> UB
```

Ситуация, когда указатель живёт дольше объекта, на который он указывает, опасна.

4. Висячие ссылки — тот же сценарий, но со ссылками:

Фрагмент

```
const int& foo() {
    int x = 10;
    return x;    // возвращаем ссылку на мёртвую переменную
}
```

Как только функция закончится, `x` уничтожается, и ссылка становится недействительной.

5. Попытка сделать «пустую ссылку»

Фрагмент

```
int& r;    // так нельзя: нужна инициализация
```

Ссылки обязаны на кого-то ссылаться с самого начала.

6. Переключение ссылки на другой объект

Фрагмент

```
int x = 10;
int y = 20;

int& r = x;
r = y;    // это не "r теперь ссылается на y"!
// Это "присвоить x значение y".
```

Ссылку нельзя «перепривязать». Она всегда остаётся связанной с тем объектом, с которым была инициализирована.

На этом базовое знакомство с указателями и ссылками заканчиваем. Дальше, когда будем говорить о динамической памяти, классах и более сложных структурах данных, указатели и ссылки будут появляться всё чаще. Но уже сейчас есть фундамент:

- указатели — «адреса переменных и элементов массива»;
- `*` и `&` — разыменование и взятие адреса;
- ссылки — удобный способ передать объект в функцию «по адресу», не меняя синтаксис вызова;
- `const T&` — любимчик всех, кто пишет функции, которые только читают данные.

Теперь у нас есть инструмент адресов и косвенного доступа. Далее мы начнём строить свои типы: упакуем связанные данные в одну сущность и посмотрим, как такие объекты реально лежат в памяти.

Часть 4

Собственные типы и ООП

В этой части мы начнём собирать свои типы — сначала `struct` (как устроены данные в памяти), потом `class` (как добавить поведение и инварианты), затем наследование и виртуальные функции (как писать код «по интерфейсу»). Здесь особенно важно держать в голове время жизни объектов и владение ресурсами: это сердце C++.

Глава 11 struct, выравнивание и работа с памятью

Представь, что у тебя есть три переменные:

Фрагмент

```
int x;  
int y;  
double length;
```

Ты таскаешь их вместе по всем функциям — это такой «разобранный конструктор». `struct` — способ сказать компилятору: «это всё одно логическое целое, дай мне свой тип».

В этой главе мы научимся объявлять собственные типы через `struct` и посмотрим, как структура реально лежит в памяти.

Разберёмся с выравниванием и паддингом, освежим в голове стек и кучу и потрогаем динамическую память, а по пути соберём коллекцию типичных граблей, связанных с памятью и раскладкой данных.

11.1 Свой тип данных: что такое `struct`

Базовый синтаксис:

Фрагмент

```
struct Point {  
    int x;  
    int y;  
};
```

`Point` — новый тип, как `int` или `double`. Теперь можно писать:

Фрагмент

```
Point p;    // переменная типа Point  
p.x = 10;  
p.y = 20;
```

Или сразу инициализировать (aggregate-инициализация):

Фрагмент

```
Point p{10, 20};
```

Структуры работают и в массивах/векторах:

Фрагмент

```
Point polyline[3] = {{0, 0}, {1, 2}, {5, 7}};  
std::vector<Point> points = {{1, 2}, {3, 4}};
```

У структуры могут быть методы:

Фрагмент

```

struct Point {
    int x;
    int y;

    void move(int dx, int dy) {
        x += dx;
        y += dy;
    }
};

Point p{0, 0};
p.move(5, -2); // p.x == 5, p.y == -2

```

11.1.1 struct против class

Синтаксически это одна и та же конструкция, главное отличие сейчас — уровни доступа по умолчанию:

Фрагмент

```

struct S {
    int x; // по умолчанию всё public
};

class C {
    int x; // по умолчанию всё private
};

```

В этой главе используем struct как удобную связку полей, к полноценным class вернёмся позже.

11.2 Размер и раскладка структуры в памяти

Компилятор обязан где-то разместить поля структуры. Интуитивно хочется думать:

Фрагмент

```

struct Foo {
    int a;
    int b;
    double c;
};

```

Размер Foo = 4 + 4 + 8 = 16 байт. В этом примере так и будет:

Фрагмент

```
std::cout << sizeof(Foo) << "\n"; // обычно 16 на 64 битах
```

Но поменяем порядок:

Фрагмент

```
struct Bar {
    int    a;
    double b;
    int    c;
};
```

и внезапно:

Фрагмент

```
std::cout << sizeof(Bar) << "\n"; // чаще всего 24
```

Разница появляется из-за выравнивания полей и паддинга.

11.3 Выравнивание и паддинг

Процессор любит, когда данные лежат по адресам, кратным их размеру (на типичной 64-битной системе: `int` — 4 байта, `double` — 8 байт).

11.3.1 Хороший порядок полей

Фрагмент

```
struct Foo {
    int    a; // 4
    int    b; // 4
    double c; // 8
};
```

Типичный расклад:

Фрагмент

```
offset 0:  a (4 байта)
offset 4:  b (4 байта)
offset 8:  c (8 байт)
итого: 16 байт
```

11.3.2 Порядок с дополнительным паддингом

Фрагмент

```
struct Bar {
    int    a; // 4
    double b; // 8
    int    c; // 4
};
```

Чтобы `b` начиналось с адреса, кратного 8, компилятор вставит невидимый паддинг:

Фрагмент

```
offset 0:  a (4 байта)
offset 4:  padding (4 байта)
offset 8:  b (8 байт)
offset 16: c (4 байта)
offset 20: padding (4 байта, чтобы размер кратен 8)
итого: 24 байта
```

Общие правила (чуть упрощённо):

- каждое поле выравнивается по своему любимому адресу (обычно по размеру типа);
- между полями может появляться **padding**, чтобы следующее поле начиналось правильно;
- размер структуры кратен выравниванию самого «толстого» поля, поэтому паддинг возможен и в хвосте.

11.3.3 Как уменьшить размер структуры

Иногда достаточно поменять порядок полей:

Фрагмент

```
struct Bad {
    char  c;
    double d;
    int   i;
};

struct Good {
    double d;
    int    i;
    char   c;
};
```

Good почти наверняка компактнее Bad именно за счёт более удачного выравнивания.

11.4 Пустые классы, sizeof и оптимизация пустой базы (ЕВО)

Иногда встречаются типы, у которых нет ни одного поля данных, но есть методы:

Фрагмент

```
struct A {
    void f(int) {}
};
```

Такой класс называют **пустым** (empty class): данных у него нет. Тем не менее, `sizeof(A)` **не может быть нулём**. Минимальный размер любого объекта в C++ — 1 байт: иначе два разных объекта могли бы иметь один и тот же адрес, а стандарт требует, чтобы у разных объектов были разные адреса.

Теперь сравним два случая:

Фрагмент

```
struct B {
    A a;      // ПУСТОЕ поле-объект
    char c;   // 1 байт
    int i;    // 4 байта (обычно выравнивание 4)
};

struct C : A { // ПУСТАЯ базовая часть
    char c;
    int i;
};
```

- В B поле a (даже пустое) обязано занимать место, поэтому оно обычно даёт хотя бы 1 байт плюс возможный паддинг из-за выравнивания следующих полей.
- В C компилятор **может** применить **empty base optimization** (ЕВО): пустая базовая часть не занимает места в объекте-наследнике.

Важный нюанс: ЕВО разрешена стандартом, но не обязана применяться. На практике почти все популярные компиляторы на обычных ABI (x86-64 и т.п.) её делают, и в задачах на sizeof обычно подразумевают именно такое поведение.

В C++20 есть ещё атрибут `[[no_unique_address]]`, который позволяет похожую оптимизацию для *полей-членов* пустого типа, но это уже более продвинутая тема.

11.5 Заглядываем внутрь: reinterpret_cast и байтики

Иногда ради эксперимента хочется посмотреть на память глазами компилятора. Учебный эксперимент: берём указатель на структуру и приводим его к `int*`, чтобы увидеть, что между полями есть байты паддинга.

Фрагмент

```
struct S {
    int x;
    double y;
    int z;
};

S s{5, 3.14, 7};

auto* pi = reinterpret_cast<int*>(&s);
std::cout << pi[0] << "\n"; // x (скорее всего 5)
std::cout << pi[1] << "\n"; // попадём в паддинг, будет мусор
```

Так делать в рабочем коде нельзя — это почти гарантированное `undefined behavior`. Это учебный эксперимент, чтобы убедиться, что паддинг реально существует.

Почему это UB: во-первых, объект `S` не является массивом `int`, а значит обращение `pi[1]` формально выходит за пределы единственного `int` внутри `x`; во-вторых, «читать память чужим типом» часто нарушает правила `aliasing`.

Правила aliasing (очень грубо): компилятор может считать, что указатели на «несвязанные» типы не указывают на одну и ту же область памяти, и на основе этого агрессивно оптимизировать код. Поэтому «прочитать объект как другой тип» может дать неожиданный результат или UB даже если «байты вроде совпадают». Исключения, которые обычно разрешены: доступ к представлению объекта через `unsigned char/std::byte`.

Если нужно безопасно смотреть байты объекта, используют `std::byte/unsigned char` или `std::memcpy`.

11.5.1 Как посмотреть байты без UB

Если хочется именно «байтовое представление», делайте так:

Фрагмент

```
#include <array>
#include <cstring> // std::memcpy

S s{5, 3.14, 7};
std::array<unsigned char, sizeof(S)> bytes{};
std::memcpy(bytes.data(), &s, sizeof(S)); // ок для trivially copyable
```

Дальше можно печатать `bytes[i]` в hex и изучать паддинг/выравнивание.

Trivially copyable (побайтно копируемый) — это тип, который стандарт разрешает копировать как «набор байт» (например, через `std::memcpy`). Для `char`, чисел, указателей и многих простых структур это обычно верно.

Endianness — порядок байтов в многобайтовых числах (какой байт считается «старшим/младшим»). **ABI** (Application Binary Interface) — соглашения о двоичном интерфейсе: как вызываются функции, как раскладываются структуры/классы в памяти, выравнивание и т.п. Поэтому байтовая диагностика полезна для учёбы/отладки, но это не формат данных, на который можно опираться между компиляторами/платформами.

В C++20 появился ещё один удобный инструмент: `std::bit_cast` (заголовок `<bit>`). Он позволяет «переложить биты» между типами одинакового размера, если оба типа `trivially copyable`. Это хороший вариант, когда хочется именно получить значение другого типа, а не просто посмотреть байты.

11.6 Где живёт структура: стек и куча

Коротко освежим модель памяти процесса: есть код, статические данные, стек и куча.

- **Стек** — как правило локальные переменные и параметры функций (в упрощённой модели). На практике компилятор может оптимизировать размещение (регистры, устранение временных и т.п.). Живут до выхода из своей области.
- **Куча (heap)** — динамическая память, которую выделяем вручную (`new/delete`) или через контейнеры.

Замечание. В современном C++ в прикладном коде почти не пишут голые `new/delete`: динамические массивы и строки хранят в `std::vector/std::string`, а если нужен именно указатель, используют `std::unique_ptr` или `std::shared_ptr`. Ниже мы разбираем ручное управле-

ние памятью, чтобы понимать механику; позже вернёмся к умным указателям как к безопасной обёртке.

11.6.1 Структура на стеке

Фрагмент

```
struct Point {
    int x;
    int y;
};

void foo() {
    Point p{10, 20};           // живёт на стеке
    std::cout << &p << "\n"; // адрес рядом с другими локальными
} // p уничтожается автоматически
```

11.6.2 Структура в динамической памяти

Фрагмент

```
Point* p = new Point{10, 20}; // выделили в куче
std::cout << p << "\n";      // адрес из другой области
delete p;                     // обязаны освободить
```

Правило: всё, что создали через new, нужно уничтожить через delete, иначе утечка.

11.7 new, delete и массивы структур

Структуры ведут себя с new как обычные типы.

11.7.1 Один объект

Фрагмент

```
struct Player {
    int hp;
    int mp;
    bool alive;
};

Player* p1 = new Player{100, 50, true};

p1->hp -= 10;
p1->alive = (p1->hp > 0);
(*p1).mp += 5;

delete p1;
```

Когда есть указатель на структуру, стрелка `->` — сахар над `(*p).field`.

11.7.2 Массив структур в куче

Фрагмент

```
std::size_t n = 1000;
Player* players = new Player[n]{}; // {} -> value-initialization (для простых полей даст нул
и)

players[0].hp = 42;
players[1].mp = 100;

delete[] players; // обязательно delete[]
```

Самая частая ошибка здесь — удалить массив через обычный `delete`. Тогда деструкторы элементов отработают неправильно, и дальше вы получите UB или утечку; запомните связку `new[]` $\leftrightarrow$ `delete[]`.

Вторая беда — потерять указатель `players` (например, перезаписать его), и тогда память останется висеть до конца программы.

11.8 Классические грабли работы с памятью

1. Выход за границы массива

Фрагмент

```
int arr[3] = {1, 2, 3};
arr[3] = 10; // UB: индексы 0..2
```

2. Ходим указателем мимо выделенной памяти

Фрагмент

```
int* p = new int{5};
int* q = p + 1; // one-past: сформировать можно (для сравнения/итерации), разыменовыва
ть нельзя
*q = 10; // UB: разыменовывание one-past
```

3. Потерянный указатель (утечка)

Фрагмент

```
int* p = new int{5};
p = nullptr; // адрес потерян, delete не вызвать
```

4. Использование удалённого объекта (dangling pointer)

Фрагмент

```
Player* p = new Player{100, 50, true};
delete p;
```

```
p->hp = 0; // UB: указатель висит
```

5. Возврат указателя/ссылки на локальный объект

Фрагмент

```
int* bad() {
    int x = 5;    // живёт на стеке
    return &x;    // x умрёт при выходе
}
```

6. Двойное освобождение

Фрагмент

```
int* p = new int{5};
delete p;
delete p; // UB
```

Хорошая привычка — после `delete` писать `p = nullptr`; и проверять указатель перед использованием.

11.9 Как думать о struct и памяти

Самое важное — перестать считать struct магией и видеть в ней:

«Набор полей, лежащих подряд в памяти, с возможным паддингом».

Отсюда следствия:

- размер структуры не всегда равен сумме размеров полей;
- порядок полей влияет на компактность;
- структура может жить на стеке, в куче или в статической области;
- всё, что `new`, должно быть `delete` — забыл, получил утечку.

Структуры уже позволяют собрать поля вместе. Следующий шаг — добавить поведение и правила использования: методы, инкапсуляцию и управление жизненным циклом объектов.

Глава 12 Классы, инкапсуляция и жизненный цикл

В прошлой главе про `struct` мы уже собирали данные в аккуратные «коробочки». Теперь время апгрейда: **class** — это уже не просто коробка, а гаджет: и данные внутри, и кнопки с экранчиком снаружи.

В этой главе мы разберёмся, чем класс смыслово отличается от структуры, и введём инкапсуляцию через `public/private/protected`.

Потом посмотрим на конструкторы (в том числе копирования) и деструктор, аккуратно разделим копирование и присваивание, дойдём до правила трёх и идеи RAII и соберём всё это в цельную картинку жизненного цикла объекта.

12.1 От `struct` к `class`: идея

С точки зрения языка C++:

Фрагмент

```
struct S {
    int x;
    void foo();
};

class C {
    int x;
    void foo();
};
```

отличаются ровно в двух местах:

- у `struct` по умолчанию всё `public`;
- у `class` по умолчанию всё `private`.

Но на уровне **смысла** договоримся так (ровно как в лекции):

`struct` — «просто данные», `class` — «данные + поведение».

Типичные примеры:

- `struct Point { double x, y; };` — просто координаты;
- `class String { /* поля + методы строки */ };` — уже инструмент.

Это не жёсткое правило языка, а стиль:

- для простых «пакетов полей» — `struct`;
- для полноценной абстракции с инвариантами, логикой, скрытием реализации — `class`.

12.2 Инкапсуляция и модификаторы доступа

Инкапсуляция — разделение «что можно делать с объектом» и «как он устроен внутри».

В C++ за это отвечают модификаторы доступа:

Фрагмент

```
class Account {
public:
    void deposit(int amount);
    void withdraw(int amount);
    int balance() const;

private:
    int balance_ = 0;
};
```

12.2.1 Три уровня доступа

1. `public` — наружный интерфейс:
 - доступен всем, кто видит этот класс;
 - то, чем пользуются.
2. `private` — внутренности:
 - виден только методам этого же класса (и друзьям, но это позже);
 - поля и вспомогательные функции, которые менять опасно.
3. `protected` — как `private`, но ещё видно в производных классах (наследование будет в следующей главе, пока просто запоминаем).

Типичная структура класса, рекомендованная в лекции:

- сначала идут `public` конструкторы, деструктор и публичные методы;
- в конце — `private` поля.

Так читающему проще: открыл файл, прочитал сверху: «ага, это класс `Stack`, он умеет `push`, `pop`, `top`, `empty`». И только если сильно надо — полез внутрь смотреть поля.

12.3 Устройство класса: поля, методы, this

Базовый пример:

Фрагмент

```
class Vector2 {
public:
    Vector2(double x, double y)
        : x_(x), y_(y) {}

    void move(double dx, double dy) {
        x_ += dx;
        y_ += dy;
    }

    double length() const {
        return std::sqrt(x_ * x_ + y_ * y_);
    }
};
```

```
private:
    double x_ = 0;
    double y_ = 0;
};
```

Здесь:

- `x_`, `y_` — поля (члены данных);
- `move`, `length` — методы-члены;
- слово `const` после `length()` значит: метод не изменяет объект.

12.3.1 mutable и логическая константность

`const`-метод запрещает менять состояние объекта. Но иногда есть поля, которые не меняют «смысл» объекта для пользователя: кэш, счётчик обращений, мьютекс.

Фрагмент

```
struct Vec {
    int sum() const {
        if (!cached) { cache = slow_sum(); cached = true; }
        return cache;
    }
private:
    int slow_sum() const;        // не важно как устроено
    mutable int cache = 0;      // можно менять даже в const-методе
    mutable bool cached = false;
};
```

Важно: `mutable` — инструмент, который нужно применять точно; он не «ломает `const`», а позволяет выразить идею «внешне объект не меняется».

Внутри методов у нас неявно есть указатель `this` на текущий объект:

Фрагмент

```
void move(double dx, double dy) {
    this->x_ += dx; // обычно this-> опускают
    this->y_ += dy;
}
```

Мы редко пишем `this->`, но полезно понимать, что под капотом каждое обращение к полю — это `this->field`.

12.4 Статические поля класса (static data member)

Статическое поле принадлежит **классу**, а не каждому объекту: оно одно на всех экземпляров.

Фрагмент

```
struct X {
```



```
static int n;           // объявление
};

int X::n = 0;           // определение (обычно в .cpp)
```

Начиная с C++17 можно делать **inline-переменные** и определять их прямо в классе:

Фрагмент

```
struct Y {
    inline static int n = 0;      // C++17: и объявление, и определение
    static constexpr int k = 10; // constexpr тоже ведёт себя как inline
};
```

Главное правило: статическое поле объявляют в классе, а определение либо выносят в один .cpp, либо (если оно inline/constexpr) оставляют в заголовке.

12.4.1 Статические методы

У класса могут быть и **статические методы**:

- вызываются как `Class::method()`;
- у них нет указателя `this`;
- они могут напрямую обращаться только к **статическим** членам класса.

Фрагмент

```
class Counter {
public:
    Counter() { ++alive_; }
    ~Counter() { --alive_; }

    static int alive() { // static member function
        return alive_;
    }

private:
    inline static int alive_ = 0; // C++17 inline static
};
```

12.5 Перегрузка операторов и friend (семинар + экзаменационный минимум)

Операторы в C++ — это просто *синтаксический сахар* над вызовами функций. Для своих типов мы можем перегружать многие операторы, чтобы класс выглядел «естественно» в использовании.

12.5.1 Главные ограничения

- Нельзя создать **новый** оператор: перегружать можно только существующие.

- Нельзя менять **приоритет** и **ассоциативность** оператора.
- Некоторые операторы перегрузить нельзя (например, `.`, `::`, `?:`, `sizeof`).

12.5.2 Член класса или свободная функция?

Бинарные операторы можно реализовать двумя способами:

- как метод: `T operator+(const T& rhs) const;`
- как свободную функцию: `T operator+(T lhs, const T& rhs);`

Важный нюанс: Для «симметричных» операторов (`+`, `==`, `<`, `<<`) часто удобнее свободная функция: так неявные преобразования могут применяться *и слева, и справа*.

12.5.3 Пример: Complex и паттерн += ++

Фрагмент

```
class Complex {
public:
    Complex(double re = 0.0, double im = 0.0)
        : re_(re), im_(im) {}

    Complex& operator+=(const Complex& rhs) {
        re_ += rhs.re_;
        im_ += rhs.im_;
        return *this;
    }

    double re() const { return re_; }
    double im() const { return im_; }

private:
    double re_;
    double im_;
};

Complex operator+(Complex lhs, const Complex& rhs) {
    lhs += rhs;           // переиспользуем +=
    return lhs;           // return by value (см. раздел про copy elision)
}
```

12.5.4 operator« и дружественные функции

Потоковый вывод обычно делают свободной функцией:

Фрагмент

```
#include <ostream>

class Complex {
```

```
// ...
friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
    return os << z.re_ << "+" << z.im_ << "i";
}
};
```

friend даёт функции доступ к приватным/защищённым полям класса. Это не «часть интерфейса для всех», а точно выданное право.

Частая ловушка: friend не нарушает инкапсуляцию сам по себе, но им легко злоупотребить. Хороший сигнал — если friend нужен только для оператора вывода/сравнения, это нормально. Если «друзей» стало много, возможно, дизайн требует пересмотра.

12.5.5 Операторы ++, сравнение и <=> (по желанию)

Префиксный ++x обычно возвращает ссылку:

Фрагмент

```
T& operator++(); // ++x
T operator++(int); // x++ (фиктивный int отличает сигнатуру)
```

Для сравнения в C++20 есть «космический корабль» <=>. Если тип — простой агрегат, можно попросить компилятор сгенерировать сравнение:

Фрагмент

```
#include <compare>

struct P {
    int x;
    int y;
    auto operator<=>(const P&) const = default;
};
```

12.5.6 Операторы преобразования

Иногда тип можно неявно/явно преобразовать к другому. Например, «проверка на валидность»:

Фрагмент

```
class File {
public:
    explicit operator bool() const { return ok_; }
private:
    bool ok_ = false;
};
```

Частая ловушка: Не делайте «слишком умные» неявные преобразования: они могут создавать неожиданные перегрузки и неоднозначности. Если преобразование потенциально опасно — делайте его explicit.

12.6 Конструкторы: старт объекта

Конструктор — специальная функция, которая создаёт и инициализирует объект.

- имя совпадает с именем класса;
- нет возвращаемого типа (даже `void` не пишется);
- вызывается автоматически, когда создаётся объект.

12.6.1 Конструктор по умолчанию

Если у класса нет ни одного конструктора, компилятор сгенерирует конструктор по умолчанию, который «ничего особенного не делает» (поля не инициализирует, если у них нет явных инициализаторов).

Мы можем задать его явно:

Фрагмент

```
class Counter {
public:
    Counter() = default; // "дефолтный" конструктор, сгенерированный компилятором

private:
    int value_ = 0;      // инициализация поля прямо в объявлении
};
```

12.6.2 Конструктор с параметрами

Фрагмент

```
class Counter {
public:
    explicit Counter(int initial)
        : value_(initial) {}

private:
    int value_ = 0;
};
```

12.6.3 Зачем нужен `explicit`

Конструктор с одним параметром может неявно превращать значение в объект класса. Иногда это удобно, но чаще приводит к «случайным» преобразованиям.

Фрагмент

```
struct Counter {
    explicit Counter(int x) : value(x) {}
    int value;
};
```

```
Counter a = 5;    // ошибка: copy-initialization запрещена для explicit
Counter b(5);    // ok
Counter c{5};    // ok
```

`explicit` делает преобразование только явным: читателю кода сразу видно, что тут создаётся объект класса.

12.6.4 `explicit operator bool()` и проверки в `if`

Иногда объект хочется проверять как «истина/ложь» (например, файловый поток, умный указатель).

Фрагмент

```
struct S {
    explicit operator bool() const { return ready; }
    bool ready = false;
};

S s{};
if (s) {           // ok: контекстное преобразование в bool
    // ...
}

bool b = s;        // ошибка: неявное преобразование запрещено
int x = s;         // ошибка
```

Так мы разрешаем только безопасный сценарий «проверить в условии», но не разрешаем случайно превратить объект в число.

Здесь появился список инициализации:

Фрагмент

```
Counter(int initial)
    : value_(initial)    // поле инициализируется здесь
{
    // тело конструктора
}
```

Ключевой момент: поля создаются и инициализируются **до входа в тело** конструктора. То, что вы напишете в теле, — уже после создания полей.

Поэтому правильно инициализировать поля именно в списке инициализации, особенно:

- `const`-поля;
- ссылки;
- сложные объекты без конструктора по умолчанию.

12.6.5 Порядок инициализации полей

Фокус: порядок определяется не списком инициализации, а порядком объявления полей.

Фрагмент

```
class A {
public:
    A(int x, int y)
        : y_(y), x_(x) // тут y_ первым написан, но...
    {}

private:
    int x_;
    int y_;           // ...сначала инициализируется x_, потом y_
};
```

Компилятор даже может выдать ворнинг, если порядок в списке не совпадает с порядком объявлений.

12.6.6 Конструктор копирования

Специальный конструктор:

Фрагмент

```
class Number {
public:
    Number(int value)
        : value_(value) {}

    Number(const Number& other) // конструктор копирования
        : value_(other.value_) {}
private:
    int value_;
};
```

Он вызывается, когда создаём **новый** объект как копию уже существующего:

Фрагмент

```
Number a{10};
Number b = a;           // конструктор копирования
Number c(a);            // тоже конструктор копирования
```

В лекции обсуждали, кто вызывается в разных случаях: конструктор копирования или оператор присваивания.

12.7 Деструктор: финал объекта

У любого «серьёзного» класса должен быть деструктор:

Фрагмент

```
class File {
public:
```

```
File(const char* path);
~File();           // деструктор

private:
    int fd_;        // условный файловый дескриптор
};
```

Признаки:

- имя — `~ИмяКласса`;
- не принимает аргументов;
- ничего не возвращает;
- вызывается автоматически, когда объект заканчивает жизнь:
 - вышли из блока, где он был локальной переменной;
 - вызвали `delete` для указателя на этот объект;
 - завершилась программа для статических объектов и т.п.

Порядок:

- сначала конструируются поля, потом весь объект;
- при разрушении вызывается деструктор объекта, после выхода из него уничтожаются поля в обратном порядке их объявления.

12.8 Жизненный цикл и где живут объекты

У жизни объекта два измерения:

1. где хранится память:
 - стек — локальные переменные;
 - куча — `new/delete`;
 - статическая область — глобальные и `static`.
2. когда начинается и когда заканчивается жизнь объекта.

Примеры:

Фрагмент

```
void foo() {
    Vector2 a{1, 2};    // автоматический объект на стеке
} // здесь вызывается деструктор a

Vector2* p = new Vector2{3, 4}; // динамический объект в куче
// ...
delete p; // здесь вызывается деструктор
```

Класс делает следующее:

При создании объекта запускает конструктор, при завершении жизни — деструктор.

Это основа подхода RAII (Resource Acquisition Is Initialization): ресурсы (память, файлы, мьютексы) привязываются к объекту, и деструктор гарантирует их освобождение.

12.9 Копирование и присваивание: две разные операции

Важно: копирование $\neq$ присваивание.

Фрагмент

```
class Number {
public:
    Number(int value)
        : value_(value) {}

    Number(const Number& other) // конструктор копирования
        : value_(other.value_) {}

    Number& operator=(const Number& other) { // оператор присваивания
        value_ = other.value_;
        return *this;
    }

private:
    int value_;
};
```

Примеры:

Фрагмент

```
Number a{10};
Number b = a;    // новый объект b создаётся как копия a -> конструктор копирования

Number c{0};
c = a;           // объект c уже существует -> оператор присваивания
```

Именно поэтому в классе, владеющем ресурсами, нужен и конструктор копирования, и оператор присваивания.

12.10 Когда дефолтное копирование ломает всё

Если у класса нет явного конструктора копирования и оператора присваивания, компилятор сгенерирует их автоматически: просто поэлементно скопирует все поля (memberwise copy).

Для простых типов — ок. Но если класс владеет динамической памятью:

Фрагмент

```
class String {
public:
    String(const char* s) {
        size_ = std::strlen(s);
        data_ = new char[size_ + 1];
        std::copy_n(s, size_ + 1, data_);
    }
};
```



```

    }

    ~String() {
        delete[] data_;
    }

private:
    char* data_ = nullptr;
    std::size_t size_ = 0;
};

```

Здесь и далее копируем буферы через `std::copy_n` из `<algorithm>`, а для `std::strlen` нужен заголовок `<cstring>`. Для `char` допустимо и `std::memcpy`, но привычка использовать стандартные алгоритмы безопаснее, если тип элементов внезапно окажется нетривиальным.

Про `std::memcpy`. Её можно применять только к **trivially copyable** типам (в частности: все фундаментальные числа, указатели и простые структуры/классы, которые язык разрешает копировать побайтно). Формальный критерий — `std::is_trivially_copyable<T>::value`. Если тип нетривиален, копирование байтов в лоб даёт UB. Для перекрывающихся областей используйте `std::memmove`, для строк можно `std::char_traits<char>::copy`. В учебных примерах с `char`/массивами `std::memcpy` формально легальна, но хорошая привычка — по умолчанию выбирать `std::copy`/`std::copy_n`.

и мы **не** написали конструктор копирования и оператор присваивания, компилятор сгенерирует «поверхностные» версии:

- `data_` и `size_` просто копируются;
- два объекта `String` начинают указывать на один и тот же буфер.

Получаем double free: сначала деструктор одного объекта удаляет память, затем деструктор второго пытается удалить её ещё раз.

Rule of Zero. В C++11+ хорошим стилем считается вообще не писать деструкторы/копирующие операции, если класс может хранить ресурсы в готовых RAII-обёртках (`std::string`, `std::vector`, умные указатели). Тогда компилятор сам сгенерирует корректные конструкторы и присваивания. К ручному управлению ресурсом (и, соответственно, к Правилу трёх) приходится обращаться лишь когда владеем сырым ресурсом напрямую.

С появлением перемещения в C++11 к этим трём специальным функциям добавились ещё две — конструктор перемещения и оператор перемещающего присваивания. Поэтому если класс вручную владеет ресурсом, обычно всплывает **Rule of Five**; а лучший способ не думать о пятёрке — завернуть ресурс в RAII-тип и вернуться к Rule of Zero.

12.10.1 `std::move`: что он делает на самом деле

`std::move(x)` не переносит данные сам. Это всего лишь каст: по сути `static_cast<T&&>(x)`, который делает выражение `x.value` и позволяет выбрать перемещающую перегрузку. Сам перенос происходит в move-конструкторе или перемещающем операторе присваивания. После перемещения объект остаётся **валидным**, но его состояние обычно **не специфицировано**: его можно уничтожать, присваивать, вызывать безопасные методы, но нельзя полагаться на конкретные значения полей.

12.11 Copy elision (RVO/NRVO) и возврат по значению

Когда функция возвращает объект **по значению**, компилятор часто может *не делать* лишних копирований/перемещений. Это называется **copy elision**.

12.11.1 Обязательная оптимизация в C++17

Начиная с C++17 в некоторых случаях копирование/перемещение **обязано быть устранено**. Например:

Фрагмент

```
struct X {
    X() { std::cout << "X()\n"; }
    X(const X&) { std::cout << "copy\n"; }
    X(X&&) { std::cout << "move\n"; }
};

X make() {
    return X{}; // C++17: copy/move может не вызываться вообще
}
```

Это полезно помнить, когда вы отлаживаете «какой конструктор вызывается»: иногда он *не вызывается*, потому что объект создаётся сразу «на месте».

12.11.2 NRVO (опционально, но почти всегда есть)

Другой частый случай — **Named Return Value Optimization**:

Фрагмент

```
X make2() {
    X x;
    // ...
    return x; // NRVO: компилятор может создать x сразу в месте результата
}
```

NRVO не обязателен стандартом, но практически все компиляторы делают его в простых случаях.

Частая ловушка: Не пишите `return std::move(x);` для локальной переменной `x`. Такой `std::move` может отключить NRVO и заставить компилятор делать лишнее перемещение там, где он мог вообще не делать ни копирования, ни перемещения. Правильный паттерн: `return x;`. Как правило, если NRVO невозможен, компилятор и так выберет перемещающий конструктор/присваивание (если они есть). Используйте `std::move` в `return` только когда вы сознательно «выносите» ресурс из объекта (например, возвращаете поле класса) и готовы к состоянию `moved-from` у исходного объекта.

Важный нюанс: Copy elision не отменяет необходимости корректного `copy/move`: перемещение всё равно нужно при передаче объектов, реаллокациях контейнеров и т.п. Но возврат «по значению» обычно остаётся эффективным и считается нормой современного C++.

12.12 Правило трёх

Отсюда классическое правило трёх (Rule of Three):

Если классу нужен нестандартный деструктор, конструктор копирования или оператор присваивания, то почти всегда нужно явно определить все три.

Важный нюанс: Современное дополнение (C++11+): если вы определяете эти «три» (и вообще объявляете любой из специальных методов вручную), компилятор может **не сгенерировать перемещающие операции** (`move ctor/move assignment`). В итоге контейнеры вроде `std::vector` при реаллокации будут вынуждены копировать элементы, а не перемещать. Поэтому в реальном коде часто стремятся к **Rule of Zero** (владеть ресурсом через `std::string/std::vector/std::unique_ptr` и не писать специальные методы), либо переходят к **Rule of Five** (добавляют `move ctor` и `move assignment`).

Почему?

- раз у класса нестандартный деструктор (он освобождает ресурс),
- значит копирование по полям, которое делает дефолтный конструктор копирования/оператор присваивания, почти точно ошибочно.

Делаем так:

Фрагмент

```
class String {
public:
    // 1. Конструктор
    String(const char* s = "")
        : size_(std::strlen(s))
        , data_(new char[size_ + 1])
    {
        std::copy_n(s, size_ + 1, data_);
    }

    // 2. Конструктор копирования
    String(const String& other)
        : size_(other.size_)
        , data_(new char[size_ + 1])
    {
        std::copy_n(other.data_, size_ + 1, data_);
    }

    // 3. Оператор присваивания
```

```
String& operator=(const String& other) {
    if (this == &other) {
        return *this;          // самоприсваивание
    }

    char* new_data = new char[other.size_ + 1];
    std::copy_n(other.data_, other.size_ + 1, new_data);

    delete[] data_;           // освобождаем старый ресурс

    data_ = new_data;
    size_ = other.size_;
    return *this;
}

~String() {
    delete[] data_;           // деструктор
}

private:
    char* data_ = nullptr;
    std::size_t size_ = 0;
};
```

Обратите внимание: в нашем операторе присваивания мы сначала выделяем новый буфер, и только потом удаляем старый, поэтому если `new` бросит исключение, объект останется в прежнем валидном состоянии.

Исключение (exception) — механизм, которым C++ сообщает об ошибке во время выполнения: управление «выпрыгивает» из текущего кода, пока не найдётся обработчик `try/catch` (или программа завершится). Когда говорят про **гарантии исключений**, имеют в виду, что будет с объектом, если на середине операции случилась ошибка. *Базовая гарантия*: объект остаётся валидным, но его состояние могло частично измениться. *Сильная гарантия*: если операция не удалась, объект остаётся как был («либо всё, либо ничего»). Именно к сильной гарантии часто ведёт подход *copy-and-swap*: сначала делаем временную копию, затем меняем содержимое местами (*swap*) — и если копирование/выделение памяти не удалось, исходный объект не тронут.

В лекции после наивной реализации оператор присваивания улучшали до *copy-and-swap* диомы, чтобы обеспечить более сильную гарантию исключений: создаём временную копию, меняем с ней содержимое местами (*swap*), временный объект удаляется, освобождая старый ресурс.

12.13 Умные указатели: RAII для динамической памяти

Если нужен объект в динамической памяти, почти всегда берут умный указатель вместо голого `new/delete`.

`std::unique_ptr<T>` реализует владение «один хозяин» и освобождает память автоматически. Создают его обычно через `std::make_unique`:

Фрагмент

```
auto p = std::make_unique<Point>(1, 2);
use(*p);
```

`unique_ptr` нельзя копировать, но можно перемещать, передавая владение:

Фрагмент

```
std::unique_ptr<Point> make_point() {
    return std::make_unique<Point>(1, 2);
}

auto q = make_point(); // владелец теперь q
```

`std::shared_ptr<T>` нужен, когда действительно требуется разделённое владение (есть счётчик ссылок). Он дороже, поэтому начинайте с `unique_ptr` и переходите на `shared_ptr` только при необходимости.

12.14 Инкапсуляция и инварианты

Зачем вообще всё это закрывать в `private` и мучаться с конструкторами?

У любого полезного класса есть **инварианты** — условия, которые должны быть истинны для «здорового» объекта:

- у `Account` баланс не должен быть отрицательным;
- у `String` `data_` либо `nullptr`, либо указывает на корректный буфер длиной `size_ + 1` с завершающим `'\0'`;
- у `Vector` `size_` не должен быть больше `capacity_`.

Чтобы инварианты соблюдались:

1. поля делаем `private` — снаружи их никто не трогает руками;
2. конструкторы приводят объект к валидному состоянию;
3. деструктор аккуратно завершает владение ресурсами;
4. публичные методы — единственная дверь, через которую можно изменить состояние; внутри них следим за инвариантами.

Типичный пример:

Фрагмент

```
class Account {
public:
    explicit Account(int initial_balance)
        : balance_(initial_balance)
    {
        if (balance_ < 0) {
            throw std::runtime_error("negative balance");
        }
    }

    void deposit(int amount) {
```

```

        if (amount < 0) throw std::runtime_error("negative deposit");
        balance_ += amount;
    }

    void withdraw(int amount) {
        if (amount < 0) throw std::runtime_error("negative withdraw");
        if (balance_ < amount) throw std::runtime_error("not enough money");
        balance_ -= amount;
    }

    int balance() const {
        return balance_;
    }

private:
    int balance_ = 0;
};

```

Если бы поле `balance_` было `public`, любой внешний код мог бы сделать:

Фрагмент

```
acc.balance_ = -1000000;
```

и разрушить все гарантии.

Теперь у нас есть базовые инструменты работы с объектами: конструкторы, деструкторы и правила времени жизни. Следующий шаг — научиться строить иерархии типов и вызывать код через базовые интерфейсы: это как раз то место, где появляются виртуальные функции.

Глава 13 Наследование и виртуальные функции

В прошлой главе мы научились делать **свои типы** всерьёз: с инкапсуляцией, конструкторами, деструкторами и всеми этими «взрослыми» штуками.

Теперь добавляем ещё один уровень магии: **классы могут строиться на базе других классов**. Это даёт нам:

- **наследование** — один класс «расширяет» другой;
- **полиморфизм** — работаем через указатель/ссылку на базовый тип, а подставляем объект любого «потомка»;
- **виртуальные функции** — механизм, который делает полиморфизм удобным и безопасным.

Всё это — фундамент классического ООП в C++.

13.1 Зачем нужно наследование

У тебя есть базовый тип:

Фрагмент

```
class Shape {
public:
    void move(double dx, double dy);
    void draw() const;
protected:
    double x_ = 0;
    double y_ = 0;
};
```

И тебе нужно добавить конкретные фигуры:

Фрагмент

```
class Circle {
public:
    void move(double dx, double dy);
    void draw() const;
private:
    double x_ = 0;
    double y_ = 0;
    double radius_ = 0;
};
```

И ещё:

Фрагмент

```
class Rectangle {
public:
    void move(double dx, double dy);
```

```

    void draw() const;
private:
    double x_ = 0;
    double y_ = 0;
    double width_ = 0;
    double height_ = 0;
};

```

Скопипастить `x_`, `y_`, `move` в каждый класс? Можно, но стыдно.

Идея наследования:

У нас есть общий класс `Shape`, а конкретные фигуры — «частные случаи»: `Circle`, `Rectangle`, `Triangle`, ...

То есть **круг — это фигура, прямоугольник — это фигура**. По-английски это называют отношение «is-a».

Мы хотим:

- один базовый код для общих частей (`x`, `y`, `move`);
- отдельные доп. поля и методы для потомков (`radius`, `width/height` и т.п.);
- иметь возможность работать через один общий интерфейс:

Фрагмент

```
void draw_all(std::vector<Shape*>& shapes);
```

и туда спокойно передавать круги, прямоугольники, треугольники...

13.2 Базовый синтаксис наследования

Базовое объявление:

Фрагмент

```

class Base {
    // ...
};

class Derived : public Base {
    // ...
};

```

Читается: `Derived` **наследует** `Base` с **публичным наследованием**. Что это значит:

- у объекта `Derived` внутри есть «подобъект» `Base`;
- `Derived` получает все `public` и `protected` члены `Base` (правила доступа сохраняются);
- `Derived` может добавлять свои поля и методы.

Простой пример с фигурами:

Фрагмент

```

class Shape {
public:

```



```

    void move(double dx, double dy) {
        x_ += dx;
        y_ += dy;
    }

protected:    // видно в наследниках
    double x_ = 0;
    double y_ = 0;
};

class Circle : public Shape {
public:
    Circle(double x, double y, double r) {
        x_ = x;           // доступ к protected-полям базового класса
        y_ = y;
        radius_ = r;
    }

    void set_radius(double r) { radius_ = r; }

private:
    double radius_ = 0;
};

```

Ключевые слова в списке наследования:

Фрагмент

```

class Derived : public Base;    // public-наследование
class Derived : protected Base; // protected-наследование
class Derived : private Base;  // private-наследование

```

В 99% случаев в нормальном коде используется **public наследование**: мы говорим «каждый Derived есть Base», и его можно использовать там, где ждут Base.

13.3 Что происходит с объектом при наследовании

Важно понимать: объект производного класса — это **базовый объект + расширение**.
Концептуально:

Фрагмент

```

class Base {
    int a_;
};

class Derived : public Base {
    int b_;
};

```

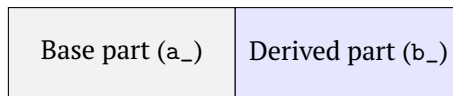


Рис. 13.1: Объект Derived состоит из базовой и производной частей

Объект Derived в памяти выглядит примерно как:

То есть у Derived есть внутри под-объект Base. Именно поэтому можно сделать:

Фрагмент

```
Derived d;
Base* pb = &d;           // ОК: Derived "is-a" Base
```

Такой переход называется **восходящее преобразование (upcast)** — безопасный и не требующий никакого cast:

Фрагмент

```
void foo(Base* b);

Derived d;
foo(&d);           // это нормально
```

Обратное направление (из Base* в Derived*) — уже не всегда законно, и для него нужен `dynamic_cast` (об этом в следующей главе).

13.4 Конструкторы и деструкторы при наследовании

Когда ты создаёшь объект производного класса, конструируется **сначала базовая часть, потом производная**:

Фрагмент

```
class Base {
public:
    Base() {
        std::cout << "Base()\n";
    }

    ~Base() {
        std::cout << "~Base()\n";
    }
};

class Derived : public Base {
public:
    Derived() {
        std::cout << "Derived()\n";
    }

    ~Derived() {
```

```

        std::cout << "~Derived()\n";
    }
};

```

Код:

Фрагмент

```

int main() {
    Derived d;
}

```

Типично выведет:

Фрагмент

```

Base()
Derived()
~Derived()
~Base()

```

То есть:

1. сначала вызывается конструктор Base;
2. затем конструктор Derived;
3. при разрушении — наоборот: сначала ~Derived, потом ~Base.

13.4.1 Вызов конструктора базового класса

Если у базового класса нет конструктора по умолчанию, или ты хочешь вызвать конкретный конструктор, это делается в **списке инициализации** производного класса:

Фрагмент

```

class Shape {
public:
    Shape(double x, double y) : x_(x), y_(y) {}

protected:
    double x_;
    double y_;
};

class Circle : public Shape {
public:
    Circle(double x, double y, double r)
        : Shape(x, y)      // тут вызывается конструктор базового класса
        , radius_(r)
    {}

private:
    double radius_;
};

```

Если ты не написал явный вызов `Shape(x, y)`, компилятор попытался бы вызвать конструктор по умолчанию `Shape()`. Если его нет — будет ошибка.

13.5 Перекрытие методов без virtual

Если в наследнике написать метод с таким же именем, как в базовом классе, ты **прячешь** (hides) базовую версию в контексте `Derived`:

Фрагмент

```
class Base {
public:
    void hello() const {
        std::cout << "Base::hello\n";
    }
};

class Derived : public Base {
public:
    void hello() const {
        std::cout << "Derived::hello\n";
    }
};
```

Смотрим, что будет:

Фрагмент

```
Derived d;

d.hello();           // Derived::hello

Base* pb = &d;
pb->hello();          // ВАЖНО: Base::hello (НЕ Derived), если нет virtual
```

Почему так? Потому что без `virtual` выбор функции идёт по статическому типу (`Base*`), а не по реальному типу объекта (`Derived`). Это называется **раннее связывание** (early binding).

13.6 virtual и полиморфизм

Делаем то же самое, но по-взрослому:

Фрагмент

```
class Base {
public:
    virtual void hello() const {
        std::cout << "Base::hello\n";
    }
};
```

```
class Derived : public Base {
public:
    void hello() const override {
        std::cout << "Derived::hello\n";
    }
};
```

И вызываем:

Фрагмент

```
Derived d;

d.hello();           // Derived::hello

Base* pb = &d;
pb->hello();          // теперь: Derived::hello (позднее связывание)
```

Слово `virtual` у базовой функции говорит компилятору: «выбирать реализацию нужно в момент вызова, глядя на фактический тип объекта, а не только на тип указателя/ссылки». Это и есть полиморфизм.

13.7 Как это устроено внутри (vtable, но без фанатизма)

В стандарте C++ ничего не сказано про конкретную реализацию, но подавляющее большинство компиляторов используют примерно такую схему:

- у каждого полиморфного объекта (у класса есть хотя бы одна виртуальная функция) хранится **указатель на таблицу виртуальных методов** — `vp_ptr`;
- таблица (`vtable`) содержит указатели на реальные функции для данного типа.

Схематично:

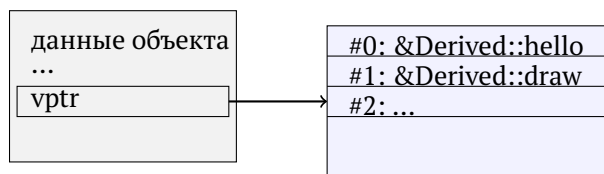


Рис. 13.2: `vp_ptr` указывает на таблицу виртуальных методов (`vtable`)

Когда мы делаем `pb->hello()`;, компилятор генерирует примерно:

1. взять `pb->vp_ptr`;
2. по фиксированному смещению найти указатель на нужную функцию;
3. вызвать её.

Важно понимать: это детали реализации, но на уровне модели полезно помнить: **виртуальная функция = ещё один уровень косвенности**.

13.8 `override` и почему его нужно писать всегда

Представь такую ситуацию:

Фрагмент

```
class Base {
public:
    virtual void draw() const;
};

class Circle : public Base {
public:
    void draw(); // хотели переопределить, но забыли const
};
```

С точки зрения компилятора это **не override**, а просто новая функция:

- Base::draw имеет тип void() const;
- Circle::draw имеет тип void().

Результат:

Фрагмент

```
Circle c;
Base* p = &c;
p->draw(); // вызывает Base::draw, а не Circle::draw
```

Чтобы такого не было, C++ даёт ключевое слово `override`:

Фрагмент

```
class Circle : public Base {
public:
    void draw() const override; // если сигнатура не совпадает --- ошибка компиляции
};
```

Современное правило: если ты переопределяешь виртуальную функцию — **всегда пиши** `override`. Это бесплатная страховка от глупых ошибок.

Важный нюанс: доступ и `virtual`

Доступность метода (`public/protected/private`) проверяется компилятором **в месте вызова** и по **статическому типу** выражения. А вот **какая реализация реально выполнится** при `virtual`-вызове выбирается уже во время выполнения.

Из-за этого возможна на первый взгляд странная ситуация:

Фрагмент

```
struct Base {
    virtual void f() { std::cout << "Base\n"; }
};

struct Derived : Base {
private:
    void f() override { std::cout << "Derived\n"; }
};
```

```
int main() {
    Derived d;

    // d.f();           // CE: f - private в Derived

    Base& b = d;
    b.f();              // OK: доступ проверяется по Base, а вызывается Derived::f
}
```

Это нормально: виртуальность и модификаторы доступа отвечают за разные вещи.

13.9 Виртуальный деструктор: must-have для полиморфной иерархии

Смотрим на такой код:

Фрагмент

```
class Base {
public:
    ~Base() {
        std::cout << "~Base\n";
    }
};

class Derived : public Base {
public:
    ~Derived() {
        std::cout << "~Derived\n";
    }
};

Base* p = new Derived;
delete p;    // UB: удаляем Derived через Base* без virtual-деструктора
```

Если деструктор Base **не виртуальный**, то delete p приводит к **undefined behavior**. На практике часто выглядит как «вызовется только ~Base(), а часть Derived останется «недоудалённой»», но стандарт не гарантирует даже этого: оптимизатор может сделать что угодно.

Правильная база для полиморфной иерархии:

Фрагмент

```
class Base {
public:
    virtual ~Base() { // виртуальный деструктор
        std::cout << "~Base\n";
    }
};
```

```
class Derived : public Base {
public:
    ~Derived() override {
        std::cout << "~Derived\n";
    }
};
```

Теперь:

Фрагмент

```
Base* p = new Derived;
delete p; // сначала ~Derived, потом ~Base
```

Золотое правило: если объект класса будут удалять через указатель на базу (например, `delete basePtr`), деструктор базового класса обязан быть виртуальным. Обычно это и есть причина, по которой полиморфным базам нужен `virtual ~Base()`.

13.10 Чисто виртуальные функции и абстрактные классы

Иногда базовый класс сам по себе не должен иметь объектов. Пример: `Shape`. Какая конкретно фигура у абстрактного `Shape`? Никакая. Но нам нужен общий интерфейс:

Фрагмент

```
class Shape {
public:
    virtual void draw() const = 0; // чисто виртуальная функция
    virtual double area() const = 0; // тоже
    virtual ~Shape() = default;

    void move(double dx, double dy) {
        x_ += dx;
        y_ += dy;
    }

protected:
    Shape(double x, double y) : x_(x), y_(y) {} // общий конструктор для наследников

    double x_ = 0;
    double y_ = 0;
};
```

`= 0` в объявлении виртуальной функции означает: **это чисто виртуальная функция, базовый класс не даёт её реализацию.**

Класс, у которого есть хотя бы одна чисто виртуальная функция, называется **абстрактным**:

- нельзя сделать `Shape s`; — будет ошибка компиляции;
- можно делать только указатели/ссылки на `Shape`.

Конкретные наследники обязаны переопределить все чисто виртуальные функции (иначе сами будут абстрактными):

Фрагмент

```
class Circle : public Shape {
public:
    Circle(double x, double y, double r)
        : Shape(x, y)
        , radius_(r)
    {}

    void draw() const override {
        std::cout << "Circle at (" << x_ << ", " << y_ << "), r = " << radius_ << "\n";
    }

    double area() const override {
        return 3.14159 * radius_ * radius_;
    }

private:
    double radius_;
};
```

Теперь можно (и лучше без утечек памяти, через RAII):

Фрагмент

```
#include <memory>

std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(0, 0, 1));
shapes.push_back(std::make_unique<Circle>(1, 1, 2));

for (const auto& s : shapes) {
    s->draw();           // вызывается draw() конкретного класса
    std::cout << s->area() << "\n";
} // при выходе из блока все объекты удалятся автоматически
```

13.11 Наследование интерфейса и реализации

С наследованием есть два разных сценария:

1. **Наследование интерфейса** — абстрактный базовый класс с чисто виртуальными методами.

Фрагмент

```
class Drawable {
public:
    virtual void draw() const = 0;
```

```
virtual ~Drawable() = default;
};
```

Наследники обязаны реализовать интерфейс, но сам код базового класса почти ничего не делает.

2. **Наследование реализации** — базовый класс даёт реальное поведение, а наследники его дополняют/переопределяют.

Фрагмент

```
class Shape {
public:
    void move(double dx, double dy) { x_ += dx; y_ += dy; }
    virtual void draw() const = 0;
    virtual ~Shape() = default;
protected:
    double x_ = 0;
    double y_ = 0;
};
```

На практике часто смешивается и то, и другое: часть функций чисто виртуальные, часть — обычные.

13.12 Срезка объектов (object slicing)

Важно помнить про **срезку** (slicing):

Фрагмент

```
class Base {
public:
    virtual void hello() const {
        std::cout << "Base\n";
    }
};

class Derived : public Base {
public:
    void hello() const override {
        std::cout << "Derived\n";
    }
};

void foo(Base b) {    // параметр по значению!
    b.hello();
}
```

Вызываем:

Фрагмент

```
Derived d;
foo(d);           // что будет?
```

В момент вызова `foo(d)` создаётся копия `Base` из объекта `d`: берётся только «базовая часть», производная часть отрезается. В итоге `b.hello()` вызывает `Base::hello`, даже если функция виртуальная.

Чтобы избежать срезки, полиморфные функции должны принимать **ссылки или указатели** на базовый класс:

Фрагмент

```
void foo(Base& b);    // ссылка
void foo(Base* b);    // указатель
```

и ни в коем случае не принимать базовый класс по значению, если предполагается полиморфизм.

13.13 Запрет переопределения и наследования: `final`

Иногда базовый класс спроектирован так, что дальше его расширять не надо и нельзя:

Фрагмент

```
class FinalClass final {
    // ...
};
```

`final` после имени класса запрещает дальнейшее наследование:

Фрагмент

```
class Derived : public FinalClass {}; // ошибка
```

То же самое работает и для виртуальных функций: `final` можно ставить на метод и запрещать дальнейшее переопределение.

Фрагмент

```
class Base {
public:
    virtual void foo();
};

class Mid : public Base {
public:
    void foo() override final; // дальше переопределять foo нельзя
};

class Child : public Mid {
public:
    void foo() override;      // ошибка: пытаемся override final-метод
};
```

Рекомендация по стилю: если вы переопределяете метод и хотите «закрыть» его дальше, удобно писать оба спецификатора:

Фрагмент

```
void foo() override final;
```

Так компилятор проверит, что вы действительно переопределяете базовый метод (override), и одновременно запретит дальнейшее переопределение (final).

13.14 Скрытие перегрузок и using Base::f (семинар)

Если в наследнике появляется метод с тем же именем, он может **скрыть весь набор перегрузок** из базового класса, даже если сигнатуры разные.

Фрагмент

```
struct Base {
    void f(int)    { std::cout << "Base::f(int)\n"; }
    void f(double) { std::cout << "Base::f(double)\n"; }
};

struct Derived : Base {
    void f(int) { std::cout << "Derived::f(int)\n"; } // скрывает ВСЕ Base::f(...)
};

int main() {
    Derived d;
    d.f(1.5); // вызовет Derived::f(int) после преобразования double -> int!
}
```

Чтобы «вернуть» перегрузки базового класса, используют using:

Фрагмент

```
struct Derived : Base {
    using Base::f;           // теперь Base::f(double) снова видна
    void f(int) { /* ... */ }
};
```

Частая ловушка: Если внезапно «пропала» перегрузка базового класса и начали происходить странные преобразования типов — проверьте, не забыли ли using Base::f.

13.15 Наследование конструкторов: using Base::Base

Конструкторы **не наследуются автоматически**. Если у базового класса нет конструктора по умолчанию, то и у наследника «по умолчанию» может не получиться сконструироваться.

Полный пример

```

struct Base {
    Base(int, int) {}
};

struct Derived : Base {
    // Derived() = default; // будет удалён: Base() не существует
};

int main() {
    // Derived d(1, 2); // ERROR: у Derived нет подходящего конструктора
}

```

Начиная с C++11 можно «унаследовать» конструкторы явно:

Фрагмент

```

struct Derived : Base {
    using Base::Base; // наследуем конструкторы Base
};

int main() {
    Derived d(1, 2); // OK
}

```

Важный нюанс: Унаследованные конструкторы не заменяют собой конструкторы копирования/перемещения и оператор присваивания (они всё равно генерируются/удаляются по обычным правилам).

13.16 Значения по умолчанию и virtual

Значения по умолчанию подставляются **в месте вызова** (в compile-time), поэтому для виртуальных функций они определяются **статическим типом** указателя/ссылки.

Фрагмент

```

struct Base {
    virtual void f(int x = 1) { std::cout << "Base " << x << "\n"; }
};

struct Derived : Base {
    void f(int x = 2) override { std::cout << "Derived " << x << "\n"; }
};

int main() {
    Derived d;
    Base* p = &d;
    p->f(); // напечатает: Derived 1
}

```

Частая ловушка: Виртуальность влияет на то, *какая функция будет вызвана*, но не на то, *какие значения по умолчанию подставляются*.

Важный нюанс: Практическое правило: **избегайте значений по умолчанию в виртуальных функциях**. Если нужно «удобное» API, обычно делают неvirtуальную обёртку без параметров, которая вызывает виртуальную функцию с явным аргументом, либо уносят «значение по умолчанию» в одно место (например, в базовый класс) и не переопределяют его в наследниках.

13.17 Виртуальные вызовы в конструкторах и деструкторах

Во время выполнения конструктора/деструктора объект ещё не является (или уже не является) «полным» объектом производного типа. Поэтому **виртуальный вызов из конструктора/деструктора не вызывает переопределение из наследника**.

Проще помнить так: пока выполняется конструктор Base, объект «ещё Base». Поэтому вызов `foo()` внутри `Base::Base()` ведёт себя так, как будто вы вызываете `Base::foo()`, даже если реально вы создаёте `Derived`.

Полный пример

```
#include <iostream>

struct Base {
    Base() { foo(); }           // вызовет Base::foo
    virtual void foo() { std::cout << "Base::foo\n"; }
    virtual ~Base() { foo(); }  // тоже вызовет Base::foo
};

struct Derived : Base {
    void foo() override { std::cout << "Derived::foo\n"; }
};

int main() {
    Derived d;
}
```

Важный нюанс: Почти всегда правило простое: **не вызывайте виртуальные функции из конструкторов/деструкторов**. Если нужно «достроить» объект, используйте фабричную функцию или отдельный метод и вызывайте его после конструирования.

13.17.1 Чисто виртуальная функция и «pure virtual call»

Если виртуальная функция чисто виртуальная (`= 0`), то **вызов такой функции через виртуальный механизм** — ошибка логики и обычно приводит к аварийному завершению (а по стандарту в ряде сценариев это UB).

При этом **чисто виртуальная функция может иметь реализацию**, и её можно вызвать явно, указав класс:

Фрагмент

```

struct A {
    virtual void f() = 0;
};
void A::f() { std::cout << "A::f\n"; }

struct B : A {
    void f() override { std::cout << "B::f\n"; }
};

int main() {
    B b;
    b.f();      // B::f
    b.A::f();   // явный вызов A::f
}

```

13.18 Множественное и виртуальное наследование (очень кратко)

Иногда класс наследуется от нескольких баз:

Фрагмент

```

struct A { int a; };
struct B { int b; };

struct C : A, B { int c; };

```

Тогда внутри C есть два базовых под-объекта, и адрес A* и B*, полученных из C*, может отличаться (указатель «сдвигается»).

Алмаз (diamond) возникает, если два базовых класса наследуются от общего предка. Чтобы общий предок был один, применяют **виртуальное наследование**:

Фрагмент

```

struct Granny { int x = 1; };

struct Mom : virtual Granny {};
struct Dad : virtual Granny {};

struct Child : Mom, Dad {}; // Granny одна, x не дублируется

```

Важный нюанс: В учебном коде множественное наследование встречается редко. Если оно используется, то чаще всего для наследования «интерфейсов» (классов без состояния).

Мы научились строить иерархии и пользоваться виртуальными методами — это базовый инструмент runtime-полиморфизма. Но иногда у нас на руках есть только ссылка или указатель на

базовый тип, и хочется безопасно понять, что там за объект на самом деле. Для таких редких случаев в C++ существует *RTTI* и инструменты вроде *dynamic_cast*.

Глава 14 Динамическое приведение типов и RTTI

В прошлой главе мы научились делать иерархии классов и пользоваться полиморфизмом через виртуальные функции. Логичный следующий вопрос: как, имея `Shape*`, понять, что там на самом деле лежит, и можно ли безопасно «спуститься» от `Base*` к `Derived*`?

Здесь на сцену выходят:

- **RTTI** — Run-Time Type Information, информация о типах во время выполнения;
- `dynamic_cast` — умное приведение типов с проверкой;
- `typeid` — способ спросить у объекта, кто он такой.

14.1 Зачем вообще нужны приведения в ООП

Классический полиморфный код:

Фрагмент

```
class Shape {
public:
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};

class Circle : public Shape {
public:
    void draw() const override {
        std::cout << "Circle\n";
    }

    double radius() const { return r_; }

private:
    double r_ = 1.0;
};

class Rectangle : public Shape {
public:
    void draw() const override {
        std::cout << "Rectangle\n";
    }
};
```

Работаем через базовый указатель:

Фрагмент

```
void render(Shape* s) {
    s->draw();          // полиморфный вызов
}
```

Полиморфизм даёт то, что обычно нужно: **не важно, какой конкретный наследник внутри**. Но иногда хочется узнать или использовать конкретный тип (если это `Circle` — вывести радиус и т.п.). Для этого в C++ уже есть встроенная инфраструктура: RTTI и `dynamic_cast`.

14.2 Напоминание: статический и динамический тип

Фрагмент

```
Shape* p = new Circle();
```

У переменной `p`:

- статический тип — `Shape*` (из кода);
- динамический тип — «указатель на объект типа `Circle`», то есть реальный тип объекта в памяти.

Статический тип известен на этапе компиляции и не меняется; динамический — во время выполнения. Виртуальные функции, `dynamic_cast` и `typeid` работают именно с динамическим типом (если есть RTTI).

14.3 Четыре оператора приведения в C++

Фрагмент

```
static_cast<T>(expr)
reinterpret_cast<T>(expr)
const_cast<T>(expr)
dynamic_cast<T>(expr)
```

Грубо:

- `static_cast` — умное приведение на этапе компиляции, без проверки в рантайме;
- `reinterpret_cast` — грубое переинтерпретирование (почти всегда подозрительно);
- `const_cast` — меняет cv-квалификацию. Это безопасно, если исходный объект **на самом деле не** `const`, а `const` появился из-за типа ссылки/указателя. Но если снять `const` и изменить объект, который был объявлен `const`, это `undefined behavior`;
- `dynamic_cast` — безопасное приведение по иерархии наследования с использованием RTTI.

Дальше говорим про последний.

14.4 RTTI: что это и когда оно есть

RTTI (Run-Time Type Information) позволяет:

- узнать реальный тип объекта (`typeid`);
- безопасно привести указатель/ссылку к нужному типу (`dynamic_cast`).

Важно: полноценная RTTI работает только для **полиморфных** типов — то есть у которых есть хотя бы одна виртуальная функция (часто это виртуальный деструктор). Если базовый класс не полиморфный, то `dynamic_cast` вниз по иерархии использовать нельзя.

14.5 dynamic_cast: безопасный даункаст

14.5.1 Синтаксис и основная идея

Фрагмент

```
Shape* s = get_shape(); // может вернуть Circle* или Rectangle*

Circle* c = dynamic_cast<Circle*>(s);
if (c != nullptr) {
    std::cout << "radius = " << c->radius() << "\n";
} else {
    std::cout << "not a circle\n";
}
```

Алгоритм:

1. dynamic_cast смотрит на динамический тип объекта, на который указывает s.
2. Если там реально Circle (или наследник), возвращает валидный Circle*.
3. Если нет — результат nullptr.

Важный нюанс: dynamic_cast выполняет проверку в **runtime** и поэтому не «бесплатен»; стоимость зависит от реализации, глубины/вида иерархии и может включать дополнительные вычисления и сдвиги указателей. В учебном коде это нормально, но в горячих местах лучше полагаться на виртуальные методы/паттерны проектирования, а не на частые проверки типа.

14.5.2 Требования к даункасту

- База должна быть полиморфной (есть virtual).
- Наследование должно быть публичным.

Иначе код не скомпилируется.

14.5.3 Приведение к ссылке

Фрагмент

```
void print_radius(Shape& s) {
    try {
        Circle& c = dynamic_cast<Circle&>(s);
        std::cout << "radius = " << c.radius() << "\n";
    } catch (const std::bad_cast&) {
        std::cout << "not a circle\n";
    }
}
```

При неудаче:

- к указателю — nullptr;
- к ссылке — исключение std::bad_cast.

14.6 Сравнение static_cast и dynamic_cast при наследовании

Фрагмент

```
Shape* s = get_shape(); // там new Circle(...)
Circle* c1 = static_cast<Circle*>(s); // без проверки
Circle* c2 = dynamic_cast<Circle*>(s); // с проверкой
```

static_cast верит на слово: если внутри окажется Rectangle, получим UB. dynamic_cast проверяет динамический тип и вернёт nullptr, если это не Circle. Правило: при даункасте, если нет 100% уверенности, используй dynamic_cast.

Uppcast (из наследника в базу) безопасен и без кастов:

Фрагмент

```
Circle* c = new Circle;
Shape* s1 = c; // upcast без кастов
Shape* s2 = static_cast<Shape*>(c); // пишут редко
```

14.7 dynamic_cast и разные направления

14.7.1 Вверх (upcast)

Фрагмент

```
Derived* d = new Derived;
Base* b = dynamic_cast<Base*>(d); // работает, но смысла нет
```

Uppcast и так безопасен.

14.7.2 Вниз (downcast)

Фрагмент

```
Base* b = get_base();

Derived* d1 = static_cast<Derived*>(b); // без проверки
Derived* d2 = dynamic_cast<Derived*>(b); // с проверкой
```

Здесь dynamic_cast делает то, что нужно.

14.7.3 Вбок (sidecast)

При множественном наследовании возможно приведение «вбок» через общего базового. dynamic_cast умеет корректно ходить по сложной иерархии (вверх/вниз/в сторону), а static_cast легко заведёт в UB.

Минимальный пример:

Cross-cast в множественном наследовании

```
#include <iostream>

struct Granny { virtual ~Granny() = default; };
struct Mother : Granny { void foo() { std::cout << "Mother\n"; } };
struct Father : Granny { void foo() { std::cout << "Father\n"; } };
struct Son : Mother, Father {};

int main() {
    Son s;
    Mother& m = s;

    dynamic_cast<Father&>(m).foo(); // ок: пришли в Father-subobject

    Son& ss = dynamic_cast<Son&>(m); // ок: downcast
    // ss.foo();                     // СЕ: неоднозначно, у Son два foo()
    ss.Mother::foo();                // ок: уточняем, откуда берём foo
}
```

Частая ловушка: Для `dynamic_cast` исходный тип должен быть **полиморфным** (иметь хотя бы одну `virtual` функцию, часто это виртуальный деструктор). Если убрать `virtual` из базового класса, `dynamic_cast<...>(m)` станет ошибкой компиляции.

14.8 typeid и std::type_info

Если `dynamic_cast` — это «попытка привести», то `typeid` — «узнать, кто ты».

14.8.1 Базовый синтаксис

Фрагмент

```
#include <typeinfo>

int main() {
    int x = 0;
    std::cout << typeid(x).name() << "\n";
}
```

`typeid(expr)` возвращает `const std::type_info&`, у которого есть `name()` и сравнение на равенство.

Замечание: строка из `name()` **не стандартизирована** — это часто «внутреннее» имя компилятора (может быть нечитабельным/манглированным). Поэтому `typeid(...).name()` используют в основном для отладки и логов, а не как часть логики программы.

14.8.2 Статический vs динамический тип в typeid

Неполиморфные типы: typeid всегда про статический тип:

Фрагмент

```
struct A {};  
A a;  
A& ra = a;  
A* pa = &a;  
  
std::cout << (typeid(a) == typeid(A)) << "\n";    // true  
std::cout << (typeid(ra) == typeid(A)) << "\n";    // true  
std::cout << (typeid(*pa) == typeid(A)) << "\n";    // true
```

Полиморфные типы через ссылку/указатель: typeid смотрит на динамический тип:

Фрагмент

```
class Base { public: virtual ~Base() = default; };  
class Derived : public Base {};  
  
Base* p = new Derived();  
  
std::cout << (typeid(*p) == typeid(Base)) << "\n";    // false  
std::cout << (typeid(*p) == typeid(Derived)) << "\n"; // true
```

Если написать typeid(p) (без *), сравнение будет по типу указателя Base*.

Частая ловушка: Если `p == nullptr` и вы делаете `typeid(*p)`, то для полиморфного типа будет выброшено исключение `std::bad_typeid`. Это не UB, но его нужно помнить (см. главу *Исключения*).

14.8.3 Практическое применение

Чаще всего:

- для отладочных сообщений: `std::cout << "type: " << typeid(*p).name();`
- для логирования;
- редко — для ветвления по типу (обычно это запах, лучше виртуальные методы или Visitor).

14.9 Когда RTTI не работает или опасен

14.9.1 Не полиморфная база

Фрагмент

```
class Base {}; // нет virtual  
class Derived : public Base {};
```

```
Base* b = new Derived;
// Derived* d = dynamic_cast<Derived*>(b); // ошибка компиляции
```

Решение — сделать базу полиморфной (добавить виртуальный деструктор или другой виртуальный метод).

14.9.2 Сломанный указатель

Если указатель/ссылка указывает не на валидный объект (use-after-free и т.п.), `dynamic_cast/typeid` не спасут: будет UB. Это не детектор мусора.

14.10 Где `dynamic_cast` и RTTI уместны, а где — тревожный звоночек

14.10.1 Где нормально

- Инфраструктура/фреймворки, где разные типы приходят по одному каналу (обработчики событий, сообщения).
- Отладка и логирование: печать реального типа, проверки ожиданий.
- Интероперабельность старого/нового кода, когда ещё нет чистой полиморфной архитектуры.

14.10.2 Где это тревожный сигнал (code smell)

Если везде писать цепочки `dynamic_cast` с `if/else if`, это типичный *code smell* — формально рабочий код, который намекает на проблемы в архитектуре.

Code smell здесь означает «подозрительный запах в коде»: само по себе это не ошибка компиляции, но часто признак, что дизайн можно сделать чище. А **Visitor** — паттерн, который позволяет вынести «ветвление по типам» из пользовательского кода в отдельный механизм обхода иерархии (если действительно нужно разное поведение для разных наследников).

Вероятно, её стоит пересмотреть:

- часто можно добавить виртуальную функцию в базовый класс;
- или применить паттерн Visitor.

Эвристика: если ты часто спрашиваешь объект «кто ты?» вместо того, чтобы просто вызвать у него метод, ты недоиспользуешь полиморфизм.

14.11 Главное по главе

В этой главе мы развели статический и динамический тип: статический виден компилятору из кода, а динамический — реальный тип объекта в памяти. RTTI появляется только у полиморфных классов с `virtual`, поэтому `dynamic_cast` вниз по иерархии работает только там; для указателей он возвращает `nullptr` при неудаче, для ссылок бросает `std::bad_cast`.

`static_cast` при даункасте ничего не проверяет и легко уезжает в UB, а `typeid` показывает статический тип для неполиморфных объектов и динамический — для полиморфных через

ссылку или указатель. Если же код постоянно строит цепочки `dynamic_cast` с ветвлениями, это обычно сигнал, что поведение лучше вынести в виртуальные методы или Visitor. На этом базовый набор по наследованию, виртуальным функциям и RTTI замыкается: есть и иерархии классов, и полиморфизм, и безопасные способы провалиться к конкретному типу, если это действительно нужно.

Мы уже умеем строить иерархии типов и работать с ними через интерфейсы. Но часто хочется обобщить код по типам без наследования: “то же самое, но для `int` и `double`”. Для этого в C++ есть шаблоны — к ним и переходим.

Часть 5

Введение в шаблоны

Если ООП — это обобщение через виртуальные функции и базовые интерфейсы, то шаблоны — это обобщение через параметры типов. Идея та же («хочу один алгоритм для разных типов»), но механизм другой: без RTTI и виртуальных вызовов, зато с компиляцией «на максималках».

Глава 15 Шаблоны: основы

Мы уже умеем строить иерархии типов и работать с полиморфизмом (виртуальные функции). Но часто полиморфизм нужен не во время выполнения, а во время компиляции: «один алгоритм для разных типов». Для этого в C++ есть **шаблоны** (templates).

15.1 Идея шаблонов

Шаблон — это «заготовка кода», параметризованная (обычно) типами. При использовании компилятор **инстанцирует** шаблон: подставляет конкретные типы и генерирует обычный код.

- **Плюсы:** нет виртуальных вызовов, всё решается в compile-time; обычно получается быстрый код.
- **Минусы:** сообщения об ошибках могут быть страшными; компиляция может быть дольше; иногда сложно читать.

15.2 Функции-шаблоны

15.2.1 Базовый синтаксис

Фрагмент

```
template <typename T>
T my_max(T a, T b) {
    return (a < b) ? b : a;
}

int main() {
    std::cout << my_max(10, 20) << "\n";    // T = int
    std::cout << my_max(1.5, 0.25) << "\n"; // T = double
}
```

Важное слово: typename (или class) вводит параметр типа.

15.2.2 Вывод параметров и явное указание типа

Если тип нельзя вывести однозначно, можно указать его явно:

Фрагмент

```
template <typename T>
T add(T a, T b) { return a + b; }

int main() {
    // add(1, 2.5);           // ERROR: T не один и тот же
    std::cout << add<double>(1, 2.5) << "\n"; // OK: T = double
}
```

```
}

```

Важный нюанс: Это одна из причин, почему в стандартной библиотеке часто используют `std::common_type_t` и перегрузки: «смешанные» типы — частая ситуация.

15.2.2.1 Параметры шаблона по умолчанию

У параметров шаблона могут быть значения «по умолчанию» — типы или константы. Это похоже на аргументы функции по умолчанию, только работает на уровне шаблона.

Пример (функция): параметр `T` нельзя вывести из аргументов, поэтому берётся значение по умолчанию.

Фрагмент

```
template <typename T = int>
T read(std::istream& in) {
    T x{};
    in >> x;
    return x;
}

int main() {
    int a = read(std::cin);           // T = int
    double b = read<double>(std::cin); // T = double
}
```

Важно:

- если параметр можно вывести из аргументов вызова, значение по умолчанию обычно не нужно — компилятор и так всё поймёт;
- у функций правила мягче: после параметра со значением по умолчанию могут идти параметры без него, если они выводятся из аргументов вызова.

15.2.3 Шаблон и не-шаблон: кто победит?

Если есть обычная перегрузка и шаблонная, то при прочих равных выигрывает **не-шаблон**. Например:

Фрагмент

```
void f(int) { std::cout << "non-template\n"; }

template <typename T>
void f(T) { std::cout << "template\n"; }

int main() {
    f(10);           // non-template
    f(10.0);         // template (подходящего не-шаблона нет)
}
```

Но можно «заставить» выбрать шаблон, указав параметры явно:

Фрагмент

```
f<int>(10); // вызовет template-версию
```

15.2.3.1 Перегрузки шаблонов: шаблон vs шаблон

Шаблонные функции тоже можно перегружать. Если подходят несколько шаблонных перегрузок, компилятор старается выбрать более «специализированную».

Фрагмент

```
template <typename T>
T maxx(T a, T b) {
    std::cout << "1\n";
    return (a < b) ? b : a;
}

template <typename T, typename U>
T maxx(T a, U b) {
    std::cout << "2\n";
    return (a < b) ? static_cast<T>(b) : a;
}

int main() {
    maxx(5, 7);           // выведет 1: подходит (T, T)
    maxx(5, 7.1);        // выведет 2: подходит (T, U)
    maxx<int, int>(5, 7); // выведет 2: явные параметры подходят только (T, U)
}
```

15.2.4 Пример: корректный swap

Фрагмент

```
template <typename T>
void my_swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
```

Частая ловушка: Шаблон работает корректно только если у типа есть копирование/присваивание. В современном коде чаще используют `std::swap`: он учитывает перемещение и специализации.

15.3 Классы-шаблоны

15.3.1 Базовый пример: Pair

Фрагмент

```
template <typename T, typename U>
struct Pair {
    T first;
    U second;

    Pair(const T& a, const U& b) : first(a), second(b) {}
};
```

Использование:

Фрагмент

```
Pair<int, double> p(10, 3.14);
```

15.3.2 Определение методов вне класса

Если метод класса-шаблона определять «снаружи», нужно повторить список параметров:

Фрагмент

```
template <typename T, typename U>
struct Pair {
    T first;
    U second;

    void swap();
};

template <typename T, typename U>
void Pair<T, U>::swap() {
    T tmp = first;
    first = static_cast<T>(second); // просто пример, обычно так не делают :)
    second = static_cast<U>(tmp);
}
```

Частая ловушка: Новички часто забывают `Pair<T, U>::` или список `template <...>` перед определением метода — и получают странные ошибки компиляции.

15.3.3 Шаблонные методы внутри класса-шаблона

Метод класса может сам быть шаблоном:

Фрагмент

```
template <typename T, typename U>
struct Pair {
    T first;
    U second;

    template <typename A>
```

```
void set_first(A x) {
    first = static_cast<T>(x);
}
};
```

15.4 Где «живут» шаблоны и почему они обычно в .h

Шаблон компилируется не как «готовая функция», а как **инструкция для генерации кода**. Поэтому компилятору нужно видеть **определение** шаблона там, где он инстанцируется.

Отсюда практическое правило:

- обычные функции: объявление в .h, определение в .cpp;
- шаблоны: **и объявление, и определение обычно в .h.**

15.5 Когда возникают ошибки

Факт, который сначала удивляет: многие ошибки в шаблонах проявляются **только при инстанцировании**.

Фрагмент

```
template <typename T>
void g(T x) {
    x.no_such_method(); // ошибка проявится только когда g(...) реально используют
}
```

Мы уже увидели главное: многие ошибки в шаблонном коде всплывают не при объявлении, а при инстанцировании — когда компилятор наконец генерирует конкретную версию. Дальше научимся управлять тем, какую именно версию шаблона выбирает компилятор (специализация и связанные с ней приёмы), а к вопросу «почему компилятор так поздно начинает ругаться» вернёмся, когда будем разбирать двухфазную проверку и зависимые имена.

Глава 16 Шаблоны: специализация, нетиповые параметры и метапрограммирование

В прошлой главе мы научились писать простые шаблоны. Теперь добавим инструменты, без которых шаблоны быстро становятся «взрослыми»: специализацию, нетиповые параметры, шаблоны от шаблонов, и вычисления на этапе компиляции.

16.1 Полная специализация

Иногда для конкретного типа нужна отдельная реализация.

16.1.1 Специализация функции-шаблона

Фрагмент

```
template <typename T>
void print(T x) {
    std::cout << x << "\n";
}

template <>
inline void print<bool>(bool x) {
    std::cout << (x ? "true" : "false") << "\n";
}
```

Важный нюанс: Явная специализация функции - это отдельная функция. Поэтому при многофайловой сборке важно помнить про ODR: если вы определяете её в заголовке, сделайте `inline` (как в примере) или определяйте её в одном `.cpp`.

Важный нюанс: явная специализация — не перегрузка

Явная специализация функции не добавляет «ещё одну перегрузку» в обычном смысле. Обычно происходит так:

1. компилятор выбирает, какую *перегрузку/шаблон вызывать* (как при обычной перегрузке);
2. если выбран именно тот шаблон, у которого есть подходящая явная специализация — берётся она.

Из-за этого специализация может «не сработать», если перегрузка выбрала другой шаблон:

Фрагмент

```
template <typename T>
void f(T) { std::cout << "1\n"; }

template <typename T>
```



```

void f(T*) { std::cout << "2\n"; }

template <>
void f<int*>(int*) { std::cout << "special\n"; } // специализация f(T) для T=int*

int main() {
    int x = 0;
    f(&x); // выведет 2, а не special: выбрана перегрузка f(T*)
}

```

Важный нюанс: Функции-шаблоны **нельзя** частично специализировать. Для «частных случаев» обычно используют перегрузки или (в более продвинутом коде) SFINAE, enable_if или concepts.

16.1.2 Специализация класса-шаблона

Фрагмент

```

template <typename T, typename U>
struct Pair {
    T first;
    U second;
};

template <>
struct Pair<void*, int> { // полная специализация
    void* first;
    int second;
    // ... особая логика
};

```

16.2 Частичная специализация (только для классов)

Классы можно специализировать частично. Это основа множества «type traits».

Фрагмент

```

template <typename T>
struct Rank { // по умолчанию: не массив
    static constexpr int value = 0;
};

template <typename T, std::size_t N>
struct Rank<T[N]> { // массив фиксированного размера
    static constexpr int value = 1 + Rank<T>::value;
};

template <typename T>

```

```
struct Rank<T[]> { // массив неизвестного размера
    static constexpr int value = 1 + Rank<T>::value;
};
```

16.3 Нетиповые параметры шаблона

Параметром шаблона может быть не только тип, но и значение, известное в compile-time:

Фрагмент

```
template <std::size_t N>
struct Buffer {
    int data[N];
};

Buffer<16> b; // N = 16
```

Частая ловушка: Нетиповые параметры должны быть compile-time константами соответствующего типа.

Ещё одна важная идея: значение нетипового параметра — часть типа. `Buffer<16>` и `Buffer<32>` — это два разных типа, поэтому компилятор может «поймать» несовместимые размеры ещё до запуска программы:

Фрагмент

```
void takes16(const Buffer<16>&) {}

int main() {
    Buffer<32> b;
    // takes16(b); // CE: разные типы -> разные размеры
}
```

16.4 Шаблоны от шаблонов: template template parameters

Иногда мы хотим передать шаблон как параметр, например выбрать «контейнер внутри контейнера».

Фрагмент

```
template <typename T,
        template <typename...> class Container>
struct Stack {
    Container<T> data;
    // ...
};
```

Такой приём работает с `std::vector`, `std::list` и многими другими контейнерами, потому что у них шаблонный интерфейс совместим с `template <typename...>` (первый параметр — тип элемента, остальные — обычно аллокатор и т.п.).

16.5 Compile-time вычисления: шаблонная рекурсия

Шаблоны позволяют вычислять значения в compile-time — это называют **метапрограммированием на шаблонах** (template metaprogramming).

Классический пример — Фибоначчи:

Фрагмент

```
template <int N>
struct Fib {
    static constexpr int value = Fib<N-1>::value + Fib<N-2>::value;
};

template <>
struct Fib<0> { static constexpr int value = 0; };

template <>
struct Fib<1> { static constexpr int value = 1; };

int main() {
    std::cout << Fib<10>::value << "\n"; // 55
}
```

Важный нюанс: Шаблонная рекурсия нагружает компилятор. Для многих задач в современном C++ проще и понятнее использовать constexpr функции, но знать механизм полезно: он лежит в основе многих стандартных инструментов.

16.6 <type_traits>: is_same, is_class, conditional и if constexpr

В стандартной библиотеке есть большой набор готовых «кирпичиков» для метапрограммирования (#include <type_traits>). В задачах и в реальном коде чаще всего всплывают:

- std::is_same<T, U> — «одинаковы ли типы»;
- std::is_class<T> — «является ли T классом/структурой»;
- std::conditional<cond, A, B> — «выбрать тип A или B по условию».

16.6.1 ::value и суффиксы _v, _t

Почти все type-traits — это **структуры с константой value**:

Фрагмент

```
static_assert(std::is_same<int, int>::value);
static_assert(!std::is_class<int>::value);
```

Начиная с C++17 есть удобные сокращения:

Фрагмент

```
static_assert(std::is_same_v<int, int>);
static_assert(!std::is_class_v<int>);
```

А для многих мета-шаблонов, которые возвращают *тип*, есть суффикс `_t`:

Фрагмент

```
// вместо: typename std::conditional<cond, A, B>::type
using X = std::conditional_t<cond, A, B>;
```

16.6.2 Выбор типа параметра: conditional

Частая цель: принять что-то по ссылке для «тяжёлых» типов и по значению для простых. Например: «если это класс — по ссылке, иначе — по значению»:

Фрагмент

```
#include <type_traits>

template <typename T>
using ParamT = std::conditional_t<std::is_class_v<T>, T&, T>;

template <typename T>
void f(ParamT<T> value) {
    // ...
}
```

Важно: `std::conditional<...>` — это **не тип**, а **шаблон-структура**. Чтобы получить именно тип, используйте `::type` или `_t`.

16.6.3 Почему обычный if не подходит и зачем нужен if constexpr

Если внутри шаблона вы пишете обычный `if`, то **компилятор обязан скомпилировать обе ветки**. Поэтому такой код часто падает с СЕ:

Фрагмент

```
template <typename T>
void bad(T x) {
    if (std::is_same_v<T, int>) {
        x *= 5;
    } else {
        x += "def"; // для int это не компилируется
    }
}
```

Правильный инструмент — `if constexpr` (C++17): неиспользуемая ветка **не компилируется**.

Фрагмент

```
#include <string>
#include <type_traits>

template <typename T>
```

```
using ParamT = std::conditional_t<std::is_class_v<T>, T&, T>;

template <typename T>
void good(ParamT<T> value) {
    if constexpr (std::is_same_v<T, int>) {
        value *= 5;          // изменится только локальная копия int
    } else {
        value += "def";      // для std::string это работает
    }
}
```

Важный нюанс: Если вам нужно «по ссылке только для `std::string`, а не для любого класса», то условие в `conditional_t` стоит писать через `std::is_same_v<T, std::string>`, а не через `is_class`.

16.7 Заметка про `std::vector<bool>`

Под капотом: `std::vector<bool>` — это историческая специализация, которая хранит биты упакованно. Из-за этого `operator[]` возвращает не `bool&`, а прокси-объект. В итоге выражения вроде `&v[0]` не работают так, как ожидаешь.

Практическое правило: если вам нужен «обычный» вектор булей с нормальными ссылками — используйте `std::vector<char>`/`std::vector<std::uint8_t>` или специализированные структуры вроде `std::bitset` (если размер фиксирован).

*Специализация и нетиповые параметры помогают сделать шаблоны точнее и безопаснее, но пока мы работали в основном с «фиксированным» числом параметров. Дальше добавим ещё одну степень свободы: пакеты параметров, *fold*-выражения и правила для зависимых имён — то, что делает современный шаблонный код компактным, но иногда пугающим на вид.*

Глава 17 Variadic templates, fold expressions и зависимые имена

Последний шаг в теме шаблонов — **variadic templates** (шаблоны с переменным числом параметров) и связанные с ними **fold expressions**. Здесь же разберём ключевую «боль» шаблонов: двухфазную проверку и зависимые имена (*typename/template*).

17.1 Variadic templates: параметры-пакеты

Фрагмент

```
template <typename... Ts>
void foo(Ts... args) {
    std::cout << sizeof...(Ts) << "\n"; // сколько типов в пакете
}
```

Ts... — это пакет типов.

args... — это пакет значений.

17.2 Рекурсивная обработка пакета

Классический стиль (до C++17) — рекурсия по параметрам:

Фрагмент

```
void print() { // базовый случай
    std::cout << "\n";
}

template <typename T, typename... Ts>
void print(T x, Ts... rest) {
    std::cout << x << " ";
    print(rest...);
}
```

Размер пакета: sizeof...(pack)

Для parameter pack есть специальный оператор sizeof...:

- sizeof...(Ts) — сколько типов в пакете Ts;
- sizeof...(args) — сколько аргументов в пакете args.

Фрагмент

```
template <typename... Ts>
void demo(Ts... args) {
    std::cout << "types = " << sizeof...(Ts) << "\n";
}
```

```
std::cout << "args  = " << sizeof...(args) << "\n";
}
```

17.3 Fold expressions (C++17)

Fold expression — это «свертка» пакета по оператору.

17.3.1 Примеры

Fold expression — это выражение специальной формы, и по синтаксису оно всегда пишется в скобках: $(xs + \dots)$ — корректно, а $xs + \dots$ — уже не fold expression.

Фрагмент

```
template <typename... Ts>
auto sum(Ts... xs) {
    return (xs + ...); // ((x1 + x2) + x3) + ...
}

template <typename... Ts>
bool all(Ts... xs) {
    return (... && xs); // x1 && x2 && x3 && ...
}
```

17.3.2 Печать: fold по оператору запятая

Для вывода через << удобен fold по запятой:

Фрагмент

```
template <typename... Ts>
void print_line(const Ts&... xs) {
    ((std::cout << xs << " "), ...);
    std::cout << "\n";
}
```

Важный нюанс: Запятая здесь — настоящий оператор запятая: он вычисляет левую часть, потом правую. Благодаря этому получаем гарантированный порядок вывода.

17.4 Двухфазная проверка шаблонов

Шаблоны проверяются в две фазы:

- на этапе определения шаблона проверяются **независимые** имена (не зависящие от параметров шаблона);
- на этапе инстанцирования проверяются **зависимые** выражения и имена.

Фрагмент

```
template <typename T>
void g(T t) {
    foo();           // Независимое имя: foo должен быть объявлен прямо сейчас
    t.bar();         // Зависимое: проверится только при инстанцировании
}
```

Важный нюанс: Из-за этого иногда кажется, что «шаблон компилируется, хотя внутри ерунда». На самом деле компилятор откладывает часть проверок до момента использования.

17.5 Зависимые имена: `typename` и `template`

17.5.1 `typename` для зависимого типа

Если имя зависит от параметра шаблона, компилятор не всегда знает, является ли оно типом или, например, статическим полем. Тогда нужно подсказать ключевым словом `typename`:

Фрагмент

```
template <typename T>
struct S {
    using A = int;
};

template <typename T>
void f() {
    typename S<T>::A x = 0; // без typename может не распарситься
}
```

17.5.2 `template` для зависимого шаблона

Если у зависимого объекта есть шаблонный метод, компилятор может попытаться распарсить `<` как оператор «меньше». Тогда нужно ключевое слово `template`:

Фрагмент

```
template <typename C>
void g(C& c) {
    c.template foo<5>(10); // подсказываем: foo - это шаблон
}
```

17.6 Зависимые базовые классы и `this->`

Ещё одна частая ловушка: члены зависимого базового класса не участвуют в обычном поиске имён на первой фазе.

Фрагмент

```
template <typename T>
struct Base {
    int x = 1;
};

template <typename T>
struct Derived : Base<T> {
    void f() {
        // std::cout << x;          // может не найти x
        std::cout << this->x;      // OK
        // или: std::cout << Base<T>::x;
        // или: using Base<T>::x;
    }
};
```

*На этом первый семестр заканчивается. Помни: компилятор действует по правилам, и если видеть границу между *compile-time* и *runtime*, многие «магические» места становятся предсказуемыми. Увидимся во втором семестре.*